Padrão de Projeto LabSUS - Manual de Referência Técnica

- Introdução
- Visão Geral da Arquitetura
  - Engenharia de Atributos (Feature Engineering)
  - Modelagem Avançada (Modeling & Analytics)
- Estrutura do Documento
- Módulos de Extração de Características (src/features/)
  - Arquitetura e Funcionamento
  - Descrição dos Módulos por Fonte de Dados
    - 1. Epidemiologia e Agravos (Fonte: SINAN)
    - 2. Estatísticas Vitais - Mortalidade (Fonte: SIM)
    - 3. Estatísticas Vitais - Nascimentos (Fonte: SINASC)
    - 4. Infraestrutura e Capacidade (Fonte: CNES)
    - 5. Morbidade Hospitalar (Fonte: SIH)
  - Orquestração
  - Utilidade e Propósito do Diretório src/features/
    - 1. Encapsulamento de Regras de Negócio (Lógica Epidemiológica)
    - 2. Normalização e Comparabilidade Demográfica
    - 3. Abstração da Complexidade de ETL (Extração, Transformação e Carga)
    - 4. Modularidade e Escalabilidade Horizontal
- Orquestrador de Integração de Dados (src/integrarindicadores.py)
  - Arquitetura e Responsabilidades
  - Fluxo de Processamento
  - Parâmetros de Execução (Interface CLI)
  - Artefato de Saída
- Orquestrador de Clusterização e Tipologia (src/modelagem/clusters.py)
  - Arquitetura e Metodologia
  - Componentes Chave
  - Parâmetros de Execução (CLI)
  - Saídas Geradas (Outputs)
- Módulo de Análise Exploratória Visual (src/modelagem/analiseexploratoriarisco.py)
  - Arquitetura e Dependências
  - Componentes de Visualização
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Análise de Fluxo de Pacientes (src/modelagem/analisefluxopacientes.py)
  - Arquitetura e Metodologia
  - Dependências Externas
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)

- Módulo de Inferência Causal (src/modelagem/analiseimpactocausal.py)
  - Arquitetura e Metodologia
  - Componentes Chave
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Análise de Redes Complexas (src/modelagem/analiseredehospitalar.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Ingestão e Mapeamento de Fluxos (ETL)
    - 2. Construção do Grafo e Cálculo de Métricas
    - 3. Classificação de Perfis
    - 4. Visualização e Persistência
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Análise de Risco Geográfico (src/modelagem/analiseriscogeografico.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Engenharia de Atributos Contextuais (Enriquecimento)
    - 2. Otimização e Pré-processamento
    - 3. Modelagem Preditiva (Supervisionada)
    - 4. Análise e Visualização Espacial
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Mineração de Sequências Clínicas (src/modelagem/analisesequenciaeventos.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Harmonização de Identificadores e Resolução de Entidades
    - 2. Definição de Coorte e Reconstrução Temporal (Backtracking)
    - 3. Mineração de Padrões e Frequência
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Análise de Sobrevida Infantil (src/modelagem/analisesobrevidainfantil.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Ingestão e Coorte Temporal
    - 2. Vinculação de Registros (Record Linkage)
    - 3. Enriquecimento e Engenharia de Atributos
    - 4. Modelagem Preditiva (Classificação)
    - 5. Explicabilidade do Modelo (XAI)
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)

- Módulo de Análise de Sobrevivência e Readmissão (src/modelagem/analisesobrevidareadmissao.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Ingestão e Estruturação Longitudinal
    - 2. Pré-processamento e Engenharia de Atributos
    - 3. Modelagem Multivariada (Regressão de Cox)
    - 4. Visualização e Análise Estratificada
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Modelagem de Tópicos em Texto (src/modelagem/analisetopicossintomasnlp.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Ingestão e Concatenação
    - 2. Pré-processamento de Texto (Limpeza)
    - 3. Vetorização
    - 4. Decomposição e Inferência (LDA)
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Auditoria Ambulatorial Inteligente (src/modelagem/auditoriaambulatorial.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Ingestão e Enriquecimento Cadastral (ETL)
    - 2. Motor de Regras de Negócio (Auditoria Clínica/Administrativa)
    - 3. Motor Estatístico (Detecção de Outliers Volumétricos)
    - 4. Consolidação e Reporte
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Inferência de Risco Individual (src/modelagem/forecastinternacoes.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Carregamento de Artefatos (Inicialização)
    - 2. Ingestão e Alinhamento de Esquema (Schema Enforcement)
    - 3. Inferência Probabilística
    - 4. Geração de Explicabilidade (SHAP)
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Geração de Painel Geográfico (src/modelagem/gerarpainelgeografico.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Ingestão e Padronização do IVS
    - 2. Mineração de Infraestrutura (CNES)

        - 3. Integração e Consolidação
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Predição de Abandono do Tratamento de Tuberculose (src/modelagem/modeloriscoabandonotb.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Filtragem de Escopo
        - 2. Pré-processamento e Engenharia de Atributos
        - 3. Treinamento e Balanceamento
        - 4. Avaliação e Explicabilidade (XAI)
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Predição de Risco Perinatal (src/modelagem/modeloriscoperinatal.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Consolidação de Dados
        - 2. Pré-processamento e Engenharia de Atributos
        - 3. Treinamento e Otimização
        - 4. Avaliação e Persistência
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Classificação de Risco de Readmissão (src/modelagem/modeloriscoreadmissaoclassificacao.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Otimização de Memória
        - 2. Construção da Coorte Longitudinal
        - 3. Enriquecimento e Preparação
        - 4. Treinamento e Interpretação
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Otimização de Referência e Contra-referência (src/modelagem/otimizadorreferenciapacientes.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Carregamento de Artefatos e Georreferenciamento
        - 2. Definição de Cenário (Origem e Destino)
        - 3. Motor de Simulação (Inference Loop)
        - 4. Ranking e Pontuação (Scoring)
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)

- Módulo de Priorização de Alocação de Recursos (src/modelagem/pipelinealocacaorecursosaps.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Ingestão e Fusão de Dados (Data Fusion)
    - 2. Normalização e Tratamento de Polaridade
    - 3. Agregação Dimensional e Ranking
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Análise de Jornada do Paciente Crônico (src/modelagem/pipelinedoencascronicas.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Ingestão e Otimização de Recursos (Memory Optimization)
    - 2. Harmonização e Identificação de Entidades
    - 3. Engenharia de Atributos Longitudinais
    - 4. Modelagem e Interpretação
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Pipeline de Readmissão Hospitalar (src/modelagem/pipelinereadmissao.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Aquisição e Persistência de Dados Brutos
    - 2. Engenharia de Atributos Aprimorada (Advanced Feature Engineering)
    - 3. Treinamento e Validação do Modelo
    - 4. Interpretabilidade e Serialização
  - Parâmetros de Configuração
  - Artefatos Gerados (Outputs)
- Módulo de Inferência de Risco Individual (src/modelagem/predictpacient.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Inicialização e Carregamento
    - 2. Ingestão e Alinhamento de Esquema (Schema Enforcement)
    - 3. Inferência Probabilística
    - 4. Geração de Artefatos de Decisão
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Inferência de Sobrevivência Individual (src/modelagem/preversobrevidaindividual.py)
  - Arquitetura e Fundamentação Metodológica
  - Fluxo de Processamento Detalhado
    - 1. Carregamento de Artefatos

        - 2. Vetorização e Alinhamento
        - 3. Predição da Função de Sobrevivência
        - 4. Extração de Métricas e Visualização
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Inferência de Risco de Óbito Infantil (src/modelagem/preversobrevidainfantil.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Carregamento de Artefatos
        - 2. Vetorização da Instância
        - 3. Inferência e Classificação
        - 4. Decomposição do Risco (SHAP Waterfall)
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Previsão de Custo de Internação (src/modelagem/previsaocustointernacao.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Integração de Fontes
        - 2. Pré-processamento e Engenharia de Atributos
        - 3. Treinamento e Avaliação
        - 4. Persistência e Explicabilidade
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Previsão de Demanda Estratificada (src/modelagem/previsaodemandaestratificada.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão de Série Histórica
        - 2. Estratificação de Risco (Batch Scoring)
        - 3. Agregação e Modelagem de Séries Temporais
        - 4. Consolidação e Visualização
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Requalificação de Causa de Óbito via NLP (src/modelagem/requalificadorcausaobitonlp.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Estruturação do Corpus
        - 2. Pré-processamento e Vetorização
        - 3. Treinamento Supervisionado
        - 4. Inferência e Filtragem por Confiança
    - Parâmetros de Execução (CLI)

    - Artefatos Gerados (Outputs)
- Módulo de Simulação de Impacto de Políticas Públicas (src/modelagem/simuladorimpactopoliticas.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Carregamento e Definição de Linha de Base (Baseline)
        - 2. Definição da Intervenção (Perturbação de Dados)
        - 3. Re-inferência (Cenário Simulado)
        - 4. Cálculo de Impacto
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Simulação de Custo de Internação (src/modelagem/simularcustointernacao.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Inicialização e Carregamento de Artefatos
        - 2. Vetorização e Tipagem Estrita
        - 3. Inferência Numérica
        - 4. Geração de Explicabilidade (Waterfall Plot)
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Calculadora de Risco Perinatal (src/modelagem/usarcalculadorarisco.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Carregamento de Artefatos
        - 2. Ingestão e Vetorização
        - 3. Inferência Probabilística
        - 4. Geração de Explicabilidade Visual
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Validação Cruzada de Desfechos (src/modelagem/validarmodeloperinatalcomsobrevida.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Carregamento de Artefatos (Modelo)
        - 2. Construção do Dataset de Validação (ETL de Sobrevida)
        - 3. Inferência Cruzada
        - 4. Avaliação de Performance
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Previsão de Tempo de Permanência - LOS (src/modelagem/previsaotempopermanencia.py)
    - Arquitetura e Fundamentação Metodológica

    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Enriquecimento
        - 2. Engenharia de Atributos Avançada
        - 3. Modelagem Segmentada (Loop de Treinamento)
        - 4. Inferência Híbrida e Avaliação
        - 5. Explicabilidade Semântica (SHAP)
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Simulação de Capacidade Hospitalar (src/modelagem/simuladorcapacidadeleitos.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Definição de Cenários
        - 2. Execução da Simulação (Núcleo Estocástico)
        - 3. Consolidação de Métricas
        - 4. Análise de Trade-off
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Detecção de Anomalias Financeiras (src/modelagem/detectoranomaliasfaturamento.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Limpeza
        - 2. Engenharia de Atributos para Auditoria
        - 3. Detecção Não Supervisionada
        - 4. Explicação Heurística
        - 5. Visualização (Redução de Dimensionalidade)
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Otimização de Alocação de Recursos (src/modelagem/otimizadoralocacaorecursos.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Normalização de Dados
        - 2. Formulação do Problema (Modelagem)
        - 3. Resolução (Solving)
        - 4. Consolidação e Relatório
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Detecção de Surtos e Anomalias (src/modelagem/pipelinedeteccaosurtos.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado

        - 1. Ingestão e Filtragem Geoespacial
        - 2. Construção de Séries Temporais
        - 3. Modelagem e Inferência (Prophet)
        - 4. Visualização e Reporte
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Fenotipagem de Pacientes Complexos (src/modelagem/clusterizacaopacientescomplexos.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Filtragem de Escopo
        - 2. Pré-processamento Híbrido
        - 3. Modelagem (PCA + K-Means)
        - 4. Perfilamento (Profiling) e Interpretação
        - 5. Visualização em Espaço Reduzido
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Mineração de Comorbidades Clínicas (src/modelagem/mineracaocomorbidades.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Extração de Transações
        - 2. Vetorização (One-Hot Encoding)
        - 3. Mineração de Regras (Apriori)
        - 4. Modelagem e Visualização de Redes
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Simulação de Fluxo de Pronto Socorro (src/modelagem/simuladorfluxops.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Geração de Demanda (Chegadas)
        - 2. Triagem e Classificação
        - 3. Atendimento Médico
        - 4. Decisão Clínica e Desfecho
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Módulo de Análise Espacial e Detecção de Clusters (src/modelagem/analiseespaciallisa.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Vinculação Geoespacial (Spatial Join)
        - 2. Cálculo de Autocorrelação (Modelagem)
        - 3. Renderização e Persistência
    - Parâmetros de Execução (CLI)

    - Artefatos Gerados (Outputs)
- Módulo de Inferência Causal e Pareamento (src/modelagem/analiseimpactocausal2.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Ingestão e Definição do Modelo Causal
        - 2. Estimação do Escore de Propensão
        - 3. Pareamento e Estimativa do Efeito
        - 4. Validação de Suposições (Common Support)
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
- Prova de Conceito: Módulo de Assistente de Protocolos Clínicos - RAG Lite (src/modelagem/assistenteprotocolosrag.py)
    - Arquitetura e Fundamentação Metodológica
    - Fluxo de Processamento Detalhado
        - 1. Construção do Índice Vetorial (Em Memória)
        - 2. Processamento da Consulta (Query)
        - 3. Recuperação e Ranking (Retrieval)
    - Parâmetros de Execução (CLI)
    - Artefatos Gerados (Outputs)
    - Limitações do Protótipo

Introdução

Este documento consolida as especificações técnicas, arquiteturais e funcionais dos módulos computacionais que compõem o **LabSUS** — Laboratório de Inteligência e Análise de Dados em Saúde. A plataforma foi concebida para operacionalizar o ciclo de vida completo dos dados de saúde pública brasileira (SUS), transcendendo a análise descritiva tradicional para oferecer capacidades preditivas e prescritivas avançadas.

O **LabSUS** atua como um motor analítico integrado, abrangendo desde a ingestão de microdados administrativos brutos até a execução de algoritmos complexos para suporte à decisão estratégica. Esta documentação destina-se a engenheiros de dados, cientistas de dados e arquitetos de soluções, servindo como referência central para a manutenção, escalabilidade, auditoria e governança dos algoritmos implementados.

Para explicar a arquitetura atual do LabSUS, precisamos voltar à gênese do projeto. A plataforma não nasceu como um sistema distribuído complexo; ela nasceu de uma pergunta epidemiológica: É possível automatizar matematicamente o Diagnóstico Situacional de Saúde de municípios aplicando o algoritmo KMeans?

A resposta foi nosso primeiro modelo: o Módulo de Clusterização

Como alguém com um paradigma em ciências da saúde, a escolha de indicadores de saúde na engenharia de features pareceu uma escolha óbvia. No entanto, existem motivos para isso: do ponto de vista matemático temos a eliminação do viés populacional através da normalização dos dados; mas, mais importante, do ponto de vista epidemiológico, os indicadores não são dados

frios: são conhecimento, são conceitos de saúde pública, e isso garante que os clusters tenham relevância clínica e não sejam apenas uma coincidência matemática.

Porém, definir que usaríamos indicadores foi apenas o primeiro passo. O desafio seguinte foi a engenharia de dados: como extrair essa informação de forma programática e escalável?
Nós sabíamos que os dados necessários existiam no DATASUS, mas o acesso era totalmente complicado. O DATASUS usa um banco de dados da década de 80 espalhado por servidores FTP, dividos em diretórios por estados, anos, meses ou agravos. Para alimentar o KMeans nós precisávamos transformar esse caos em uma matriz estruturada. Fazer isso manualmente ou através de scraping para cada base seria impossível.

Foi aí que o PySUS se tornou o alicerce do projeto. Ele foi desenvolvido pela Escola de Matemática Aplicada da FGV em parceria com a Fiocruz como parte do projeto Alerta Dengue. Ele resolveu três problemas críticos:
Abstração de acesso: Nós não precisávamos mais saber a estrutura do FTP. O PySUS encapsulou essa complexidade
Decodificação: O PySUS converte os arquivos .dbc em DataFrames manipuláveis, o que exigiria engenharia reversa complexa
Escalabilidade: Ele permite baixar dados de todo o Brasil em batch/lote

Mas a importância do PySUS transcendeu completamente o KMeans.

Ao validar que conseguíamos ingerir dados do DATASUS programaticamente para fazer a clusterização, percebemos que havíamos encontrado o Padrão de Ingestão para toda a plataforma. O PySUS se tornaria a Camada de Infraestutura de Dados de toda a arquitetura do LabSUS: ele padronizou a entrada de dados, atuando como o motor de ETL e permitindo que o foco permanecesse na regra de negócio epidemiológica.

Para construir as features, sabíamos que não poderíamos fazer tudo em um código só ou tratar cada base de dados como um problema isolado. Isso geraria um código caótico, difícil de manter e de escalar. Então, adotamos uma abordagem de Arquitetura Modular. Desenhamos um template padrão com o fluxo de ETL, que seria a fábrica de indicadores. Cada script do diretório features/ segue rigorosamente o mesmo fluxo lógico:
Normalização do denominador: criamos um módulo utilitário que carrega a população oficial do IBGE para cada município; isso assegura que todos os indicadores usem o mesmo denominador, garantindo a comparabilidade estatística
Abstração do numerador: por exemplo, no caso da TMI, não existe uma base de dados de mortalidade infantil. A solução nesse caso foi filtrar a coluna IDADE do SIM para valores menores que 1
Cálculo da taxa/padronização: cruza o numerador com o denominador e aplica o fator de multiplicação

Com os indicadores definidos modularmente, outro problema de engenharia surgiu: como unir dezenas de scripts independentes em uma única Matriz Analítica sem criar um código rígido que precisasse ser editado toda vez em que criássemos um novo indicador?
Para resolver isso, desenvolvemos o orquestrador de indicadores, que atua como o coração do ETL, usando dois preceitos:
Descoberta dinâmica: em vez de importar manualmente cada script de indicadores, utilizamos a

biblioteca importlib para criar um sistema de descoberta automática de forma que, se adicionássemos um script arboviroses.py na pasta hoje, o sistema o reconheceria e o incorporaria ao painel na próxima execução sem que fosse necessário alterar uma única linha de código
Resolver a esparsidade: se um município não teve nenhum caso de dengue, ele simplesmente não existe no arquivo original; se fizéssemos uma junção simples, perderíamos os municípios saudáveis da análise. O orquestrador resolve isso criando primeiro um "esqueleto" com todos os municípios e populações para depois acoplar os indicadores usando Left Joins e tratar os valores ausentes (Null) como Zero, de forma que o KMeans enxergue a diferença entre um Dado Ausente (erro) e Zero Casos (sucesso sanitário)

Com o orquestrador de indicadores rodando, tínhamos em mãos uma Matriz Analítica completa, com várias colunas cobrindo diversos indicadores.

A tentação mais óbvia era jogar tudo no KMeans, na esperança de encontrar um "padrão universal de saúde", mas com isso cairíamos num erro metodológico clássico: o ruído encobriria o sinal, criando clusters matematicamente válidos, mas sem sentido epidemiológico.

Notamos, então, que o Diagnóstico Situacional não é único: ele depende da pergunta que está sendo feita.

Para resolver isso, refatoramos o modelo de clusterização para que ele funcionasse como uma Lente Ajustável através da Linha de Comando (CLI): ele aceita uma lista de argumentos "--indicadores" que define o recorte da análise.

Assim, o mesmo código agora permitiria rodar numa sessão uma análise focada exclusivamente em Saúde Materna, e depois outra análise focada em Vigilância, usando exatamente a mesma infraestrutura de dados e o mesmo algoritmo.

Com o algoritmo rodando, tínhamos grupos matematicamente perfeitos, mas semanticamente vazios. Se rodássemos uma análise de Saúde Materna, o 'Cluster 0' poderia ser o grupo de Alto Risco; se rodássemos uma análise de Arboviroses, o 'Cluster 0' poderia ser o grupo de Baixo Risco, porque o KMeans é cego (não-supervisionado): seria necessário abrir o Excel e interpretar as médias manualmente a cada execução.

Para resolver esse problema, criamos uma camada híbrida de aprendizado supervisionado que nomeamos de Classificação por Arquétipos.

Em vez de tentar fazer a máquina "adivinhar" o nome do cluster, nós inserimos o conhecimento epidemiológico no sistema.
Criamos um mecanismo onde nós definimos Perfis Teóricos de Referência (os Arquétipos) em um arquivo CSV simples. Por exemplo, definimos o que seria um município de "Alta Vulnerabilidade": TMI alta, Cobertura de Pré-natal baixa etc. Definimos o que seria um "Polo de Atendimento": Alta complexidade, muitos médicos etc.

No código, implementamos uma rotina de pós-processamento do KMeans:
O algoritmo calcula o Centroide (ponto médio) de cada cluster real encontrado nos dados
Em seguida, ele calcula a Distância Euclidiana entre esse centroide real e todos os Arquétipos teóricos que definimos

O sistema então rotula o cluster automaticamente com o nome do Arquétipo mais próximo matematicamente

Assim, o 'Cluster 0' deixa existir. O sistema entrega, por exemplo, 'Cluster: Alta Vulnerabilidade Materna'

Com isso, resolvemos o problema da Caixa Preta, criando um mecanismo de controle para garantir que a saída faça sentido clínico. Além disso, trouxemos flexibilidade: se o critério mudar, basta alterar o CSV de Arquétipos sem precisar alterar uma linha de código.

Com o modelo validado, o desafio passou a ser a entrega. Como colocar isso na mão do usuário?

Nossa primeira hipótese foi o Streamlit, mas ele se mostrou muito limitado para a complexidade do que estávamos projetando.

Então, adotamos o Dash. E, por um bom tempo, ele foi a primeira casa do LabSUS. Foi no Dash que conseguimos construir a primeira versão funcional do painel de clusterização: a ferramenta funcionava para prototipagem e validação da lógica.

Porém, à medida em que adicionávamos novos modelos e refinávamos as visualizações - adicionando mapas vetoriais pesados, filtros dinâmicos, interatividade entre gráficos etc - chegamos ao teto de performance.

O Dash gerenciava o estado da aplicação no server-side: cada clique num filtro, por exemplo, disparava um recálculo no Python e uma nova renderização no HTML. Não foi possível implementar uma camada de MLOps, sistema de usuários, gerenciamento de arquivos, a interatividade dos gráficos era limitada. Com o volume de dados e a complexidade das visualizações, a aplicação se tornou lenta e pesada. A experiência de usuário degradou.

Percebemos, então, que estávamos tentando forçar uma ferramenta de script de dados a se comportar como um Sistema Web complexo. Foi esse gargalo que nos obrigou a tomar a decisão final de engenharia: abandonar o framework all-in-one e desacoplar o sistema.

Precisávamos de um Frontend real - que viria a ser o React - para lidar com as visualizações no cliente e operação da camada de MLOps, e de uma API robusta - que viria a ser o Django REST framework - para servir apenas os dados. Foi a morte do Dash que deu vida à arquitetura atual do LabSUS.
Quando assumimos a decisão de abandonar o Dash, compreendemos que o problema raiz não era apenas a ferramenta de visualização, mas a arquitetura monolítica. O mesmo processo calculava a matemática pesada dos modelos e renderizava a interface.

Para superar isso, projetamos uma nova arquitetura baseada no Desacoplamento de Responsabilidades. Dividimos o que se tornaria o ambiente de produção do LabSUS em camadas independentes que operam em sintonia:

O Backend - a API como "sistema nervoso": primeiro, isolamos toda a lógica de negócio, acesso a dados e segurança em uma API RESTful utilizando o Django REST Framework. O DRF não sabe o que é um gráfico ou um mapa. Ele não gasta processamento gerando HTML. Ele atua estritamente como um guarda, recebendo requisições, validando permissões e entregando dados puros estruturados em formato JSON. Isso garantiu a padronização que o sistema precisava.

O Frontend - a experiência de usuário: na ponta, contruimos uma Single Page Application usando React. Diferentemente do Dash, onde o servidor desenhava as visualizações, aqui invertemos a lógica: o servidor manda apenas os números. O navegador do usuário (client-side) utiliza a biblioteca Plotly.js para desenhar as visualizações. Isso distribuiu o processamento: o servidor cuida da inteligência de dados e o dispositivo do usuário cuida da renderização visual.
O Motor Assíncrono - resolvendo o gargalo do processamento: modelos de Machine Learning e IA são intensivos no uso de processamento. Se nós rodássemos o modelo durante uma requisição HTTP tradicional (síncrona), o ambiente travaria esperando a resposta, criando uma experiência de usuário inaceitável. Para solucionar isso, implementamos uma Fila de Tarefas Assíncrona usando Celery e Redis:
Quando o usuário pede uma análise, a API não a executa; ela coloca um "ticket" no Redis - o gerenciador de filas - e devolve um "OK" imediato para a interface
Em segundo plano, os trabalhadores do Celery pegam esse ticket, rodam o processamento pesado de forma isolada e salvam o resultado. Isso garante que o ambiente permaneça fluido e responsivo, mesmo que o servidor esteja processando milhões de registros de internações
O Banco de Dados Híbrido - robustez e flexibilidade: para amarrar tudo isso, utilizamos o PostgreSQL, mas com uma decisão estratégica de design: o uso de campos JSONB. Como a Ciência de Dados é experimental, os parâmetros, variáveis e saídas dos modelos podem mudar o tempo todo. O JSONB nos permitiu ter a flexibilidade de um banco NoSQL dentro da segurança transacional e robustez de um Banco Relacional.

No entanto, antes mesmo de chegarmos a essa infraestrutura robusta, tomamos uma decisão de arquitetura fundamental: a separação total entre Pesquisa e Produção.

Transformamos o diretório src/ no nosso Ambiente de Pesquisa e Desenvolvimento (R&D). Ali, o sistema opera de forma desacoplada do banco de dados complexo. Trabalhamos com arquivos planos (CSVs e Parquet) e scripts isolados.

Isso é vital porque a Ciência de Dados é experimental. Nós precisamos testar dez hipóteses para uma funcionar. Se nós tivéssemos que migrar o banco de dados ou alterar a API para cada teste de modelo, o desenvolvimento pararia.

No Ambiente de Pesquisa e Desenvolvimento, nós temos a liberdade de testar, errar e ajustar hiperparâmetros rapidamente usando arquivos locais. Somente quando um modelo é estatisticamente validado e provado útil, ele é "promovido" para o Ambiente de Produção.
Nesse momento, ocorre a Refatoração: o código sai do laboratório, é otimizado e integrado à arquitetura de produção, onde ele ganha escala, segurança e persistência no banco de dados oficial.

Essa estrutura garante que a Inovação (no src/) nunca comprometa a Estabilidade (na API).

O KMeans foi um exemplo para ilustrar as decisões de arquitetura, mas ele é apenas a ponta do iceberg. A decisão de separar o Ambiente de Pesquisa do Ambiente de Produção nos permitiu escalar a experimentação de forma exponencial.

Hoje, o diretório src/ opera como um Laboratório de Ciência de Dados independente, abrigando quase 40 modelos em diferentes estágios de maturação. Esse ambiente funciona como uma incubadora de algoritmos, onde testamos hipóteses epidemiológicas fora da complexidade do Ambiente de Produção.

Esses modelos cobrem diversos tipos de modelagem, entre elas:

Modelagem Descritiva
Temos os extratores de indicadores, que garantem que a base de dados seja limpa, normalizada e comparável. Eles alimentam certos modelos e também são a base para gerar diversas visualizações no Frontend.
Modelagem Preditiva
Classificação: Usamos LightGBM e Regressão Logística para prever riscos binários (Vai a óbito? Vai reinternar?).
Séries Temporais: Utilizamos o Prophet para decompor tendências de doenças sazonais (arboviroses) e prever surtos. Também usamos o Prophet para a análise de séries temporais do número de internações por CID.
Análise de Sobrevivência: Aplicamos modelos de Cox e Kaplan-Meier não apenas para saber se um evento vai ocorrer, mas quando (tempo até readmissão).
Modelagem de Sistemas Complexos
Grafos: Mapeamos a rede de referência hospitalar para entender fluxos de pacientes e centralidade.
Mineração de Sequências: Algoritmos que reconstroem a jornada do paciente crônico entre SIA e SIH.
NLP (Processamento de Linguagem Natural): Modelos que leem textos livres para corrigir dados ou buscar padrões.
Modelagem Prescritiva
Otimização: Modelos de Programação Linear (com solvers CBC) que alocam recursos finitos de forma matematicamente ótima.
Simulação: Modelos de Monte Carlo e Simulação de Eventos Discretos (Gêmeos Digitais) para testar cenários de estresse em UTIs e Prontos-Socorros.

O Ambiente de Pesquisa e Desenvolvimento funciona como um funil de validação científica, com o seguinte fluxo:
Prototipagem: Criamos um script rápido para testar uma ideia (ex: 'Existe correlação entre acidente de trabalho e transtorno mental?').
Validação: Testamos as métricas estatísticas e a relevância clínica.
Promoção: Apenas os modelos que passam nesse crivo rigoroso são refatorados e integrados à API de Produção.

Essa estratégia garante que o LabSUS seja um sistema vivo, com um fluxo contínuo de novas inteligências sendo desenvolvidas, testadas e, eventualmente, entregues.

Visão Geral da Arquitetura

O **LabSUS** opera sob uma arquitetura modular e desacoplada, orientada a *pipelines* de processamento de dados. A estrutura lógica organiza-se em camadas hierárquicas de transformação de informação, evoluindo de dados brutos para inteligência analítica de alta complexidade.

A topologia do sistema divide-se em dois domínios principais documentados neste volume:

Engenharia de Atributos (*Feature Engineering*)

Localizados no diretório `src/features/`, estes módulos constituem a camada de **ETL (Extração, Transformação e Carga)** do LabSUS. São responsáveis por:

- **Abstração das Fontes de Dados:** Interface robusta com os sistemas governamentais (SIM, SINASC, CNES, SIH, SINAN), isolando a complexidade dos formatos originais.
- **Normalização Epidemiológica:** Padronização de nomenclaturas, tratamento de chaves geográficas e cálculo de taxas ponderadas pela população, garantindo a comparabilidade estatística entre territórios.
- **Consolidação:** Orquestração e fusão de indicadores heterogêneos em painéis analíticos unificados (*Master Datasets*).

Modelagem Avançada (*Modeling & Analytics*)

Localizados no diretório `src/modelagem/`, estes módulos constituem o núcleo de **Inteligência Artificial e Pesquisa Operacional** do LabSUS. Esta camada consome os dados preparados para executar tarefas de:

- **Aprendizado Supervisionado:** Classificação de risco clínico (ex: óbito, readmissão) e regressão de custos/tempo.
- **Aprendizado Não Supervisionado:** Detecção de anomalias em faturamento e segmentação (clusterização) de perfis de pacientes.
- **Analytics Espacial e de Redes:** Mapeamento de fluxos de referência hospitalar e detecção de *clusters* geográficos de risco (LISA).
- **Simulação e Otimização:** Modelagem estocástica de capacidade hospitalar e programação linear para alocação ótima de recursos orçamentários.
- **Processamento de Linguagem Natural:** Extração de conhecimento estruturado a partir de textos clínicos livres.

Estrutura do Documento

As seções subsequentes detalham cada módulo do **LabSUS** individualmente. Para cada componente, são apresentados: a fundamentação metodológica, o fluxo detalhado de processamento, os parâmetros de interface (CLI) e os artefatos gerados. A documentação enfatiza a reprodutibilidade técnica e a justificativa das escolhas algorítmicas adotadas no desenvolvimento da plataforma.

Módulos de Extração de Características (`src/features/`)

O diretório `src/features/` contém os módulos responsáveis pela engenharia de dados e construção de indicadores de saúde. Cada script atua como uma unidade de ETL (Extração, Transformação e Carga) especializada em um domínio específico do DATASUS.

Arquitetura e Funcionamento

Os scripts seguem um padrão de projeto (design pattern) uniforme para permitir a orquestração automatizada:

1. **Interface Padronizada:** Cada módulo expõe uma função principal `processar_dados(ufs, anos, arquivo_populacao, ...)` que serve como ponto de entrada.
2. **Definição do Denominador:** O processo inicia-se com o carregamento das estimativas populacionais municipais via `src.utils.dataloaders`, estabelecendo a base para o cálculo de taxas.
3. **Extração do Numerador:** Utiliza-se a biblioteca `PySUS` para o download e processamento dos microdados brutos dos sistemas de origem (SIM, SINASC, CNES, SIH ou SINAN).
4. **Aplicação de Regras de Negócio:** Os dados brutos são filtrados conforme critérios epidemiológicos (ex: códigos CID-10 específicos, CBOs de profissionais, faixas etárias).
5. **Agregação e Cálculo:** Realiza-se o agrupamento dos dados por município (`CODMUN`) e o cálculo matemático do indicador (Numerador / Denominador * Fator de Multiplicação).
6. **Retorno:** O output é um *DataFrame* pandas estruturado, pronto para integração.

Descrição dos Módulos por Fonte de Dados

1. Epidemiologia e Agravos (Fonte: SINAN)

Scripts focados no monitoramento de doenças de notificação compulsória.

- `incidencia_arboviroses.py`
  - **Função:** Calcula a taxa de incidência (por 100.000 habitantes) para Dengue, Zika e Chikungunya.
  - **Metodologia:** Filtra notificações com classificação final confirmada no banco de dados do SINAN.
- `hanseniase.py`
  - **Função:** Calcula a taxa de detecção de casos novos de Hanseníase (por 100.000 habitantes).
  - **Metodologia:** Contabiliza apenas os casos marcados como "Caso Novo" na notificação.
- `indicadores_tuberculose.py`
  - **Função:** Gera múltiplos indicadores: Taxa de Incidência, Taxa de Abandono de Tratamento e Taxa de Cura.
  - **Metodologia:** Analisa o campo de situação de encerramento (`SITUA_ENCE`) do SINAN-TB.

2. Estatísticas Vitais - Mortalidade (Fonte: SIM)

Scripts dedicados à análise dos dados do Sistema de Informação sobre Mortalidade.

- `mortalidade_infantil.py`
  - **Função:** Calcula a Taxa de Mortalidade Infantil (TMI) por 1.000 nascidos vivos.
  - **Metodologia:** Realiza o cruzamento de óbitos de menores de 1 ano (SIM) com o total de nascidos vivos (SINASC).
- `mortalidade_causas_especificas.py`

    - **Função:** Calcula taxas de mortalidade por grupos específicos da CID-10 (Circulatório, Neoplasias, Respiratórias, Diabetes, Causas Externas e COVID-19).
    - **Metodologia:** Filtra a Causa Básica do óbito (`CAUSABAS`).
- `mortalidade_prematura_dcnt.py`
    - **Função:** Calcula a mortalidade prematura (faixa etária de 30 a 69 anos) causada por Doenças Crônicas Não Transmissíveis.
    - **Metodologia:** Aplica filtros de idade e utiliza a lista de CIDs definida pela OMS para DCNTs.
- `causas_mal_definidas.py`
    - **Função:** Calcula a proporção de óbitos classificados no Capítulo R da CID-10 (Sintomas, sinais e achados anormais).
    - **Relevância:** Atua como *proxy* para a qualidade da assistência e do sistema de vigilância local.

3. Estatísticas Vitais - Nascimentos (Fonte: SINASC)

Scripts que analisam as Declarações de Nascido Vivo para indicadores materno-infantis.

- `pre_natal.py`
    - **Função:** Calcula o percentual de gestantes com acesso a pré-natal adequado (7 ou mais consultas).
- `partos_cesareos.py`
    - **Função:** Calcula a proporção de partos cesáreos em relação ao total de partos.
- `maes_adolescentes.py`
    - **Função:** Calcula a proporção de nascidos vivos de mães com idade inferior a 20 anos.

4. Infraestrutura e Capacidade (Fonte: CNES)

Scripts para análise da capacidade instalada e recursos humanos.

- `medicos_por_mil.py`
    - **Função:** Calcula a densidade de médicos ativos por 1.000 habitantes.
    - **Metodologia:** Contabiliza CPFs únicos com CBO da família 225 (Médicos) na base de Profissionais (`PF`).
- `equipes_esf_taxa.py`
    - **Função:** Calcula a cobertura de Equipes de Saúde da Família (eSF).
    - **Metodologia:** Filtra equipes do tipo '70' na base de Equipes (`EP`).

5. Morbidade Hospitalar (Fonte: SIH)

Scripts que processam dados de internações hospitalares do SUS.

- `internacoes_gerais.py`
    - **Função:** Calcula a taxa geral de internações hospitalares por 1.000 habitantes.
- `doencas_cronicas.py`
    - **Função:** Calcula a taxa de Internações por Condições Sensíveis à Atenção Primária (ICSAP), com foco no grupo de crônicas (Hipertensão, Diabetes, Asma/DPOC).

- **Relevância:** Indicador indireto da efetividade da Atenção Primária na prevenção de agravos.

Orquestração

A execução destes módulos é gerenciada pelo script `src/integrar_indicadores.py`. Este orquestrador realiza a importação dinâmica dos módulos, executa o processamento sequencial e unifica os resultados em um único painel de dados (Dataset Mestre), que serve de insumo para os modelos de Machine Learning subsequentes.

Utilidade e Propósito do Diretório src/features/

A utilidade deste componente baseia-se em quatro pilares fundamentais:

1. Encapsulamento de Regras de Negócio (Lógica Epidemiológica)

Os dados originais (SIM, SINASC, SIH, CNES) são compostos por códigos técnicos e administrativos que não possuem significado analítico imediato. Os scripts deste diretório encapsulam o conhecimento de domínio necessário para interpretar esses dados.

- **Função:** Traduzir critérios técnicos (ex: "Registros no SIM com IDADE < 1") em conceitos analíticos (ex: "Mortalidade Infantil").
- **Benefício:** Centraliza as definições de CIDs (Classificação Internacional de Doenças), CBOs (Classificação Brasileira de Ocupações) e procedimentos em módulos específicos, garantindo consistência metodológica em todo o sistema.

2. Normalização e Comparabilidade Demográfica

A análise de dados absolutos em saúde pública pode gerar distorções analíticas devido à variação populacional entre os municípios.

- **Função:** Realizar o cruzamento sistemático dos dados de eventos (numerador) com as estimativas populacionais (denominador), gerando taxas padronizadas (por 1.000 ou 100.000 habitantes).
- **Benefício:** Torna estatisticamente comparáveis territórios com demografias distintas (ex: pequenos municípios versus grandes metrópoles), pré-requisito essencial para o treinamento de modelos de Machine Learning não enviesados.

3. Abstração da Complexidade de ETL (Extração, Transformação e Carga)

As fontes de dados governamentais apresentam heterogeneidade em formatos, nomenclaturas de colunas e tipagem de dados ao longo dos anos.

- **Função:** Atuar como uma camada de abstração que resolve inconsistências, trata valores nulos, padroniza códigos de município (6 vs 7 dígitos) e harmoniza nomes de variáveis.
- **Benefício:** Isola a complexidade do tratamento de dados brutos. As camadas superiores do sistema (Orquestração e Modelagem) consomem dados limpos e padronizados, independentemente das especificidades ou alterações nos formatos dos arquivos originais.

4. Modularidade e Escalabilidade Horizontal

A arquitetura baseada em scripts independentes para cada indicador permite a expansão do sistema sem refatoração do código núcleo.

- **Função:** Permitir a adição de novos indicadores de saúde através da criação de novos arquivos .py no diretório, que são automaticamente descobertos pelo orquestrador.
- **Benefício:** Facilita a manutenção e a evolução contínua do projeto. A inclusão de um novo monitoramento (ex: Taxa de Suicídio ou Cobertura Vacinal) não exige alterações na infraestrutura de ingestão ou nos modelos preditivos existentes.

Na topologia do sistema, src/features/ atua como o **processador intermediário** que converte a "matéria-prima" (microdados do DATASUS) em "produtos acabados" (Indicadores de Saúde), alimentando tanto o Data Warehouse (arquivos CSV integrados) quanto os Pipelines de Machine Learning.

Orquestrador de Integração de Dados (`src/integrar_indicadores.py`)

O script `integrar_indicadores.py` atua como o **motor central de processamento** (*pipeline runner*) do sistema. Sua função primordial é orquestrar a execução sequencial dos módulos de extração de características (localizados em `src/features/`), consolidar os resultados heterogêneos provenientes de diversas fontes do DATASUS e estruturar o **Painel de Dados Mestre** (*Master Dataset*). Este painel consolidado serve como a fonte única da verdade para as etapas subsequentes de modelagem preditiva e análise estatística.

Arquitetura e Responsabilidades

O componente foi desenhado sob uma arquitetura de **extensão dinâmica**, eliminando a necessidade de codificação rígida (*hardcoding*) para a inclusão de novos indicadores. Suas responsabilidades técnicas incluem:

1. **Descoberta Automática de Módulos (Introspecção):**
   - Utiliza a biblioteca `importlib` para varrer o diretório `src/features/` em tempo de execução.
   - Identifica arquivos Python válidos e importa dinamicamente suas funções `processar_dados`.
   - **Utilidade:** Permite a expansão horizontal do sistema. A adição de um novo script de indicador na pasta `features` é automaticamente reconhecida pelo orquestrador na próxima execução, sem necessidade de refatoração do código central.
2. **Gerenciamento da Integridade Territorial (Backbone Populacional):**
   - Antes de processar qualquer indicador de doença, o script gera um *DataFrame* base contendo a chave primária composta (Município, Ano, UF) e os dados populacionais para todas as combinações solicitadas.
   - **Utilidade:** Garante que o painel final contenha **todos** os municípios da abrangência geográfica, inclusive aqueles sem ocorrências de agravos. Isso evita o viés de sobrevivência nos dados (onde municípios com zero casos desapareceriam do dataset).

3. **Execução Isolada e Tratamento de Exceções:**
   - Itera sobre a lista de indicadores, executando cada módulo em um contexto isolado.
   - Implementa tratamento de erros (`try/except`) granular.
   - **Utilidade:** Assegura a robustez do pipeline. A falha no download ou cálculo de um único indicador (ex: instabilidade no FTP do DATASUS para o SINAN) não compromete a geração do painel com os demais indicadores (ex: SIM e SIH).
4. **Fusão de Dados e Normalização (*Merge & Fillna*):**
   - Realiza a união dos dados calculados ao *DataFrame* base utilizando a operação *Left Join* nas chaves `['cod_mun_ibge_6', 'ANO', 'UF']`.
   - Aplica regras de limpeza pós-processamento, convertendo valores nulos (`NaN`) resultantes do *join* em `0` (zero).
   - **Utilidade:** Materializa a premissa epidemiológica de que a ausência de registro de notificação para um município específico (preservado pelo *Left Join*) implica ausência do evento naquele período.

Fluxo de Processamento

A execução do script obedece ao seguinte fluxo lógico de ETL (Extract, Transform, Load):

1. **Inicialização:** *Parsing* dos argumentos de linha de comando (CLI) e configuração de diretórios.
2. **Mapeamento:** Identificação de todas as *features* disponíveis.
3. **Construção do Esqueleto:** Carregamento da malha populacional para as UFs e Anos alvo.
4. **Ciclo de Integração:**
   - Chamada da função `processar_dados` do módulo específico.
   - Recebimento do *DataFrame* parcial.
   - Junção ao *DataFrame* Mestre.
5. **Persistência:** Exportação do resultado final para formato CSV (`output_csv_path`).

Parâmetros de Execução (Interface CLI)

O script deve ser invocado via terminal e aceita argumentos para definir o escopo do *dataset* a ser gerado:

- `--ufs` : Lista de siglas das Unidades Federativas (ex: `TO SP`).
- `--anos` : Lista de anos de referência (ex: `2022 2023`).
- `--indicadores` : (Opcional) Lista de nomes específicos dos módulos a serem executados. Se omitido, executa todos os disponíveis.
- `--arboviroses` : Parâmetro específico para o módulo de arboviroses.
- `--mortalidade` : Parâmetro específico para grupos de causas de óbito.
- `--output_csv_path` : Caminho de destino do arquivo consolidado.
- `--pop` : Caminho do arquivo de estimativas populacionais.

Artefato de Saída

A utilidade final do script é a geração de uma **Tabela Analítica Mestre** (formato *Wide*), onde:

- **Linhas:** Representam a unidade de análise (Município X no Ano Y).
- **Colunas:** Contêm os metadados (População, UF) seguidos por todas as *features* calculadas (ex: `TAXA_MORT_INFANTIL`, `COBERTURA_PRENATAL`, `TAXA_INCID_DENGUE`).

Este artefato é o pré-requisito obrigatório para a execução dos módulos de Clusterização, Previsão de Surtos e Otimização de Recursos.

Orquestrador de Clusterização e Tipologia (`src/modelagem/clusters.py`)

O módulo `clusters.py` implementa um pipeline de **Aprendizado de Máquina Não Supervisionado** destinado à segmentação e tipologia de municípios. Sua função principal é identificar padrões latentes nos indicadores de saúde e agrupar territórios com características semelhantes (clusters), atribuindo-lhes rótulos semânticos baseados em arquétipos de referência pré-definidos.

Arquitetura e Metodologia

O script adota uma abordagem híbrida que combina a exploração não supervisionada (K-Means) com regras de negócios supervisionadas (Arquétipos) para garantir a interpretabilidade dos resultados. O fluxo metodológico consiste em:

1. **Ingestão e Filtragem:** Carrega o *Painel de Dados Mestre* (gerado por `integrar_indicadores.py`), aplicando filtros temporais (Anos) e geográficos (UFs).
2. **Seleção de Atributos (*Feature Selection*):** Permite a seleção explícita de indicadores via argumentos de linha de comando ou realiza a inferência automática de colunas numéricas relevantes presentes no dataset.
3. **Pré-processamento e Normalização:**
   - Limpeza de dados (remoção de registros com valores nulos nos indicadores selecionados).
   - **Padronização (Z-Score):** Aplicação do `StandardScaler` para transformar os dados para média 0 e desvio padrão 1. Esta etapa é crítica para o algoritmo K-Means, impedindo que indicadores com escalas de magnitude maiores (ex: População) dominem o cálculo da distância em detrimento de taxas menores (ex: Taxas de incidência).
4. **Algoritmo K-Means:** Executa o algoritmo de agrupamento para particionar os municípios em k grupos distintos (`n_clusters`), minimizando a inércia (soma dos quadrados das distâncias intra-cluster).
5. **Classificação por Arquétipos (*Matching*):**
   - Calcula os centroides (médias) de cada cluster gerado.
   - Carrega um arquivo de referência de "Arquétipos" (perfis teóricos de saúde, ex: "Alta Complexidade", "Vazio Assistencial").
   - Calcula a **Distância Euclidiana** entre o centroide de cada cluster e os arquétipos.
   - Atribui ao cluster o rótulo do arquétipo mais próximo matematicamente.
6. **Persistência:** Exporta os dados rotulados e os perfis dos centroides para análise posterior.

Componentes Chave

- `carregar_arquetipos`**:** Função responsável por ler o arquivo CSV de referência contendo os perfis esperados e suas respectivas cores hexadecimais para visualização. Valida se as colunas dos arquétipos correspondem aos indicadores presentes nos dados.
- `classificar_perfis_por_similaridade`**:** Implementa a lógica de "Matching". Recebe os centroides calculados pelo K-Means e retorna um dicionário mapeando o ID do cluster (0, 1, 2...) para um nome semântico ("Perfil X").
- `executar_kmeans_personalizado`**:** Núcleo de processamento que integra a normalização, o treinamento do modelo `KMeans` do scikit-learn e a aplicação da classificação semântica.

Parâmetros de Execução (CLI)

O script é executado via terminal e aceita argumentos para customização da modelagem:

- `--input_data_csv_path` : Caminho para o arquivo CSV de entrada (Painel Mestre).
- `--ufs` : Lista de UFs para filtragem (ex: `TO PA`).
- `--anos` : Lista de anos para filtragem.
- `--indicadores` : (Opcional) Lista explícita de colunas a serem usadas no agrupamento.
- `--n_clusters` : Número de grupos (k) a serem formados (padrão: 3).
- `--archetypes_path` : Caminho para o arquivo CSV de definição dos arquétipos de referência.
- `--output_dir` : Diretório para salvamento dos artefatos gerados.

Saídas Geradas (Outputs)

O módulo gera dois artefatos principais no diretório de saída:

1. **Dados Rotulados (** `dados_com_clusters_personalizado.csv` **):** O dataset original enriquecido com as colunas:
   - `cluster_num` : O ID numérico do grupo (0, 1, 2...).
   - `perfil` : O rótulo semântico atribuído (ex: "Vulnerabilidade Social").
   - `cor` : O código hexadecimal para plotagem consistente em dashboards.
2. **Centroides (** `centros_dos_clusters_personalizado.csv` **):** Tabela contendo os valores médios (na escala original, não padronizada) de cada indicador para cada cluster. Essencial para a interpretação clínica e epidemiológica do perfil de cada grupo.

Módulo de Análise Exploratória Visual (`src/modelagem/analise_exploratoria_risco.py`)

Este módulo implementa rotinas de **Análise Exploratória de Dados (EDA)** voltadas para a investigação visual de fatores de risco perinatal. Sua função é gerar diagnósticos gráficos que evidenciem a correlação estatística entre variáveis sociodemográficas/assistenciais e desfechos neonatais adversos (prematuridade e baixo peso ao nascer), servindo como etapa de validação de hipóteses prévia à modelagem preditiva.

Arquitetura e Dependências

O script opera como um componente satélite do pipeline de risco perinatal. Para garantir a consistência dos dados, ele não implementa lógica própria de ingestão; em vez disso, importa e

reutiliza as funções `carregar_dados_sinasc` e `preparar_dados` do módulo `src.modelagem.modelo_risco_perinatal`.

O fluxo de processamento consiste em:

1. **Ingestão e Preparação:** Carregamento dos microdados do SINASC e geração da variável alvo binária `ALVO_RISCO` (1 = Baixo Peso ou Prematuro; 0 = Normal).
2. **Enriquecimento Semântico:** Aplicação de dicionários de mapeamento (ex: códigos de escolaridade para rótulos textuais) e discretização de variáveis contínuas (ex: *binning* de idade materna).
3. **Agregação Estatística:** Agrupamento dos dados pelas variáveis preditoras e cálculo da média da variável alvo. Em um vetor binário, a média representa a **prevalência do evento** (taxa de risco) naquele grupo específico.
4. **Visualização de Dados:** Geração de gráficos de barras (*Barplots*) utilizando a biblioteca `Seaborn` para ilustrar as tendências.

Componentes de Visualização

O módulo contém funções específicas para a geração de três visualizações estratégicas:

1. `plotar_risco_por_consultas`**:**
   - **Análise:** Relaciona o número absoluto de consultas de pré-natal com a prevalência de risco perinatal.
   - **Objetivo:** Evidenciar visualmente o efeito protetor do acompanhamento pré-natal adequado (curva de dose-resposta inversa).
2. `plotar_risco_por_idade_mae`**:**
   - **Análise:** Segmenta a idade materna em faixas etárias de interesse epidemiológico (Menos de 20, 20-34, 35+).
   - **Objetivo:** Identificar o risco aumentado nos extremos da vida reprodutiva (gravidez na adolescência e idade materna avançada).
3. `plotar_risco_por_escolaridade`**:**
   - **Análise:** Mapeia os códigos numéricos de escolaridade (`ESCMAE2010`) para categorias descritivas e ordena os níveis educacionais.
   - **Objetivo:** Demonstrar o gradiente social da saúde, onde menores níveis de escolaridade tendem a correlacionar-se com maiores taxas de desfechos adversos.

Parâmetros de Execução (CLI)

O script é executado via terminal e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas a serem analisadas.
- `--anos` : Lista de anos de referência para os dados do SINASC.
- `--dir_saida` : Diretório onde os arquivos de imagem (.png) gerados serão salvos.

Artefatos Gerados (Outputs)

A execução bem-sucedida resulta na persistência de arquivos de imagem no diretório de saída especificado, tipicamente:

- `grafico_risco_por_consultas.png`
- `grafico_risco_por_idade.png`
- `grafico_risco_por_escolaridade.png`

Estes artefatos são utilizados para composição de relatórios técnicos e validação visual da qualidade dos dados antes do treinamento de modelos de Machine Learning.

Módulo de Análise de Fluxo de Pacientes (`src/modelagem/analise_fluxo_pacientes.py`)

Este módulo implementa um pipeline de **Inteligência Geoespacial** focado no mapeamento da mobilidade de pacientes dentro da rede do SUS. Sua função é identificar e visualizar os padrões de deslocamento entre o município de residência do paciente e o município de internação, permitindo a análise da rede de referência, a identificação de polos de atendimento e a detecção de vazios assistenciais para grupos específicos de doenças (CIDs).

Arquitetura e Metodologia

O script opera como uma ferramenta de ETL (Extração, Transformação e Carga) acoplada a um motor de visualização cartográfica. O processo analítico baseia-se na construção de uma **Matriz de Origem-Destino** (OD), seguindo o fluxo lógico abaixo:

1. **Ingestão de Dados (SIH/SUS):**
   - Utiliza a biblioteca `PySUS` para o download e leitura dos arquivos de Autorização de Internação Hospitalar (AIH).
   - Consolida múltiplos arquivos (várias UFs e meses) em um único *DataFrame* para análise sistêmica.
2. **Filtragem Nosológica:**
   - Aplica filtros baseados na coluna `DIAG_PRINC` (Diagnóstico Principal) para isolar a coorte de interesse (ex: Neoplasias - Capítulo C, ou doenças Cardiovasculares).
3. **Processamento de Fluxos (Matriz OD):**
   - Extrai as colunas `MUNIC_RES` (Origem) e `MUNIC_MOV` (Destino).
   - **Exclusão de Auto-laços:** Remove registros onde a Origem é igual ao Destino (atendimento local), isolando assim o fenômeno da migração para saúde.
   - **Agregação:** Contabiliza o volume de pacientes únicos para cada par `(Origem, Destino)`.
   - **Limiar de Relevância:** Aplica um filtro (`min_pacientes_fluxo`) para descartar fluxos esporádicos ou residuais, focando nas rotas de referência consolidadas.
4. **Enriquecimento Geográfico:**
   - Realiza o cruzamento (*Join*) da matriz OD com uma base de referência (`municipios.csv`) para atribuir coordenadas de latitude e longitude aos pontos de origem e destino.
5. **Visualização Vetorial:**

- Utiliza `Geopandas` e `Matplotlib` para renderizar a malha territorial.
- Plota vetores (linhas) conectando os centroides dos municípios.
- Aplica escala logarítmica (`LogNorm`) para definir a cor e a espessura das linhas, permitindo a visualização simultânea de fluxos de alta e baixa intensidade sem saturação visual.

Dependências Externas

Para a execução correta, este módulo requer a presença de arquivos de referência espacial no diretório do projeto:

1. **Base de Coordenadas:** Arquivo CSV contendo códigos IBGE, latitude e longitude dos municípios.
2. **Malha Digital (Shapefile):** Arquivos `.shp` (e auxiliares) contendo a geometria dos municípios brasileiros para a renderização do mapa de fundo.

Parâmetros de Execução (CLI)

O script é executado via terminal e expõe argumentos para definição do escopo da análise:

- `--ufs` : Lista de UFs para análise (ex: `TO PA`).
- `--anos` : Anos de competência a serem processados.
- `--cids` : Lista de prefixos da CID-10 para filtrar os diagnósticos (ex: `C` para Câncer, `I` para Circulatório).
- `--nome` : Rótulo descritivo da doença para nomear os arquivos de saída.
- `--min_pacientes` : (Opcional) Número mínimo de pacientes para que uma rota seja plotada (padrão: 5). Filtra ruído visual.
- `--municipios_csv` : Caminho para o arquivo de coordenadas.
- `--shapefile` : Caminho para o arquivo de malha digital.
- `--dir_saida` : Diretório de destino para os artefatos.

Artefatos Gerados (Outputs)

O módulo produz dois tipos de saída no diretório especificado:

1. **Relatório de Fluxo (`.csv`):** Arquivo tabular contendo a matriz de arestas da rede.
   - Colunas: `MUNIC_RES` , `MUNIC_MOV` , `N_PACIENTES` , `NOME_ORIGEM` , `NOME_DESTINO` , coordenadas.
   - **Utilidade:** Permite a importação dos dados em softwares de análise de redes complexas (com

- Introdução
- Visão Geral da Arquitetura
  - Engenharia de Atributos (Feature Engineering)
  - Modelagem Avançada (Modeling & Analytics)
- Estrutura do Documento
- Módulos de Extração de Características (src/features/)
  - Arquitetura e Funcionamento
  - Descrição dos Módulos por Fonte de Dados
    - 1. Epidemiologia e Agravos (Fonte: SINAN)
    - 2. Estatísticas Vitais - Mortalidade (Fonte: SIM)
    - 3. Estatísticas Vitais - Nascimentos (Fonte: SINASC)
    - 4. Infraestrutura e Capacidade (Fonte: CNES)
    - 5. Morbidade Hospitalar (Fonte: SIH)
  - Orquestração
  - Utilidade e Propósito do Diretório src/features/
    - 1. Encapsulamento de Regras de Negócio (Lógica Epidemiológica)
    - 2. Normalização e Comparabilidade Demográfica
    - 3. Abstração da Complexidade de ETL (Extração, Transformação e Carga)
    - 4. Modularidade e Escalabilidade Horizontal
- Orquestrador de Integração de Dados (src/integrarindicadores.py)
  - Arquitetura e Responsabilidades
  - Fluxo de Processamento
  - Parâmetros de Execução (Interface CLI)
  - Artefato de Saída
- Orquestrador de Clusterização e Tipologia (src/modelagem/clusters.py)
  - Arquitetura e Metodologia
  - Componentes Chave
  - Parâmetros de Execução (CLI)
  - Saídas Geradas (Outputs)
- Módulo de Análise Exploratória Visual (src/modelagem/analiseexploratoriarisco.py)
  - Arquitetura e Dependências
  - Componentes de Visualização
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Análise de Fluxo de Pacientes (src/modelagem/analisefluxopacientes.py)
  - Arquitetura e Metodologia
  - Dependências Externas
  - Parâmetros de Execução (CLI)
  - Artefatos Gerados (Outputs)
- Módulo de Inferência Causal (src/modelagem/analiseimpactocausal.py)

o Gephi) ou ferramentas de BI.

2. **Mapa de Mobilidade (`.png`):** Imagem de alta resolução visualizando a rede de saúde.
   - **Visualização:** O mapa destaca os vetores de deslocamento sobrepostos à geografia dos estados, com codificação de cores indicando a intensidade do fluxo. Evidencia visualmente os polos de atração (Hubs) e as regiões dependentes.

Módulo de Inferência Causal (`src/modelagem/analise_impacto_causal.py`)

Este módulo implementa um pipeline estatístico avançado baseado na metodologia de **Escore de Propensão (*Propensity Score Matching* - PSM)**. Sua função é estimar o efeito causal de intervenções em saúde (ex: realização de pré-natal adequado) sobre desfechos clínicos (ex: óbito infantil), mitigando o viés de seleção inerente a estudos observacionais.

Diferente dos modelos preditivos (que buscam correlação), este módulo busca isolar a **causalidade**, permitindo afirmar se uma política pública ou intervenção clínica é a causa direta da melhoria nos indicadores.

Arquitetura e Metodologia

O script utiliza a biblioteca `causalinference` (e rotinas auxiliares do `scipy`) para simular um Experimento Randomizado Controlado (RCT) a partir de dados observacionais. O fluxo metodológico segue as etapas abaixo:

1. **Definição do Modelo Causal:**
   - **Tratamento (D):** Variável binária que representa a intervenção (ex: `PRENATAL_ADEQUADO`, definido como 7 ou mais consultas).
   - **Desfecho (Y):** Variável de resultado (ex: `OBITO_INFANTIL`).
   - **Covariáveis de Confusão (X):** Conjunto de variáveis que influenciam tanto a probabilidade de receber o tratamento quanto o desfecho (ex: Escolaridade Materna, Idade, Raça/Cor, Paridade). Controlar estas variáveis é essencial para eliminar o viés.
2. **Estimativa do Escore de Propensão:**
   - Utiliza uma regressão logística para calcular a probabilidade condicional de cada indivíduo receber o tratamento, dadas as suas covariáveis: $P(D = 1|X)$.
3. **Pareamento (*Matching*):**
   - Executa o algoritmo de pareamento (tipicamente *Nearest Neighbor* ou *Kernel*) para associar cada indivíduo do grupo "Tratado" a um ou mais indivíduos do grupo "Controle" que possuam escores de propensão estatisticamente idênticos.
   - Este processo cria um **Grupo de Controle Sintético**, garantindo que os grupos comparados sejam homogêneos em termos de características basais.
4. **Estimativa do Efeito (ATT):**
   - Calcula o **Efeito Médio do Tratamento nos Tratados (ATT)**.
   - Realiza testes de hipótese (Cálculo de Z-score e Valor-P) para determinar a significância estatística do resultado.

Componentes Chave

- `preparar_dados_causal`**:** Função de ETL responsável por binarizar as variáveis de tratamento e desfecho, tratar valores nulos e selecionar o vetor de covariáveis (X) para o modelo.
- `executar_psm`**:** Núcleo de processamento estatístico.
  - Instancia o objeto `CausalModel`.
  - Executa a estimativa dos escores (`est_propensity_s`) e o pareamento (`est_via_matching`).

- Implementa o cálculo manual de significância estatística (Valor-P bilateral) baseado no erro padrão da estimativa, contornando limitações de versões específicas da biblioteca base.
- Gera gráficos de densidade para visualização da sobreposição (*Overlap*) dos escores de propensão, essenciais para validar a qualidade do pareamento.

Parâmetros de Execução (CLI)

O script é executado via terminal e aceita os seguintes argumentos:

- `--input-csv` : Caminho para o arquivo de dados consolidado (coorte de nascimentos/óbitos), geralmente gerado pelo módulo `analise_sobrevida_infantil.py` .
- `--dir-saida` : Diretório onde os artefatos gráficos e resultados serão salvos.

Artefatos Gerados (Outputs)

A execução do módulo resulta em dois tipos de saída:

1. **Relatório Estatístico (Console/Log):**
   - **ATT (Average Treatment Effect on the Treated):** A magnitude do impacto. Um valor negativo (ex: -0.015) indica redução do risco do desfecho (fator de proteção).
   - **Valor-P:** Indica a probabilidade de o resultado observado ser fruto do acaso. Valores < 0.05 indicam significância estatística.
   - **Conclusão Semântica:** Interpretação automática dos resultados (ex: "A intervenção reduz o risco em X%").
2. **Gráfico de Validação (`propensity_score_overlap.png`):**
   - Visualização da distribuição dos escores de propensão entre os grupos de Tratamento e Controle.
   - **Utilidade:** Permite avaliar visualmente se o pareamento foi bem-sucedido. Uma boa sobreposição das curvas indica que o grupo de controle sintético é comparável ao grupo tratado.

Módulo de Análise de Redes Complexas (`src/modelagem/analise_rede_hospitalar.py`)

Este módulo aplica fundamentos da **Teoria dos Grafos** (*Graph Theory*) e da **Ciência de Redes** (*Network Science*) para modelar o Sistema Único de Saúde (SUS) como uma estrutura reticular complexa. O objetivo é mapear a topologia de referência hospitalar, identificando fluxos de pacientes, polos de atração e gargalos logísticos que não são visíveis através de estatísticas descritivas tradicionais.

Arquitetura e Fundamentação Metodológica

O sistema modela a rede de saúde como um **Grafo Direcionado e Ponderado** $G = (V, E, W)$, onde:

- **Vértices (V):** Representam os municípios (nós da rede).
- **Arestas (E):** Representam o fluxo de pacientes entre um município de residência (Origem) e um município de internação (Destino). A direção do vetor é sempre $Origem \rightarrow Destino$.

- **Pesos (W):** Representam o volume absoluto de pacientes transferidos em um determinado período, indicando a força da conexão entre dois nós.

Fluxo de Processamento Detalhado

O *pipeline* de execução é segmentado em quatro etapas lógicas:

1. Ingestão e Mapeamento de Fluxos (ETL)

- **Extração:** O script utiliza a biblioteca `PySUS` para extrair dados brutos do Sistema de Informações Hospitalares (SIH).
- **Normalização:** Padroniza os códigos de município (`MUNIC_RES` e `MUNIC_MOV`) para o formato IBGE de 6 dígitos, garantindo a interoperabilidade com outras bases de dados.
- **Agregação (Matriz OD):** Constrói uma matriz de Origem-Destino através do agrupamento dos registros individuais, somando o volume de internações para cada par único de municípios.
- **Filtragem Topológica:** Remove os **auto-laços** (arestas onde $Origem == Destino$). Esta etapa é crítica para isolar o fenômeno da "referência intermunicipal", descartando o atendimento local para focar exclusivamente na dependência e na atração regional.

2. Construção do Grafo e Cálculo de Métricas

Utilizando a biblioteca `NetworkX` , o módulo instancia o grafo e calcula métricas avançadas de centralidade para cada nó (município):

- **Grau de Entrada Ponderado (*Weighted In-Degree*):** Somatório dos pacientes recebidos de outros municípios. Define a **Capacidade de Atração** e identifica os polos de atendimento.
- **Grau de Saída Ponderado (*Weighted Out-Degree*):** Somatório dos pacientes enviados para fora. Define o nível de **Dependência Externa** ou baixa resolutividade local.
- **Centralidade de Intermediação (*Betweenness Centrality*):** Calcula a frequência com que um nó atua como ponte no caminho mais curto entre dois outros nós. Identifica municípios estratégicos que conectam microrregiões isoladas aos grandes centros; falhas nestes nós podem fragmentar a rede.
- ***PageRank*:** Aplica o algoritmo recursivo de classificação (originalmente desenvolvido para indexação web) para medir a influência de um nó. Um município possui alto *PageRank* não apenas pelo volume de pacientes, mas por receber fluxos de outros municípios que também são importantes na rede. Isso revela a **Hierarquia Real** de referência (ex: Alta Complexidade).

3. Classificação de Perfis

Com base nas métricas calculadas, o algoritmo classifica automaticamente os municípios em perfis funcionais:

- **Polo Regional:** Municípios com *PageRank* elevado (acima do percentil 50 da rede).
- **Referência Local:** Municípios que recebem mais pacientes do que enviam (*In-Degree* > *Out-Degree*), mas não possuem alto *PageRank*.
- **Município Dependente:** Municípios que enviam mais pacientes do que recebem (*Out-Degree* > *In-Degree*).

4. Visualização e Persistência

- **Renderização de Grafo:** Utiliza o algoritmo de layout *Fruchterman-Reingold* (Spring Layout), que simula um sistema físico de molas e forças eletrostáticas para posicionar os nós. Isso aproxima visualmente os municípios fortemente conectados, revelando *clusters* naturais de atendimento que podem não corresponder às divisões administrativas oficiais.
- **Exportação Interoperável:** Gera um arquivo `.graphml` contendo toda a estrutura topológica e os atributos calculados, permitindo a importação e análise aprofundada em softwares especializados de visualização de redes (como Gephi ou Cytoscape).

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando, aceitando os seguintes argumentos:

- `--fluxo-csv` : (Opcional) Caminho para um arquivo CSV pré-processado contendo a matriz de fluxo. Se não fornecido, o script buscará baixar os dados brutos.
- `--ufs` : Lista de Unidades Federativas para compor a rede (ex: `TO`).
- `--anos` : Lista de anos de competência para análise.
- `--dir_saida` : Diretório para gravação dos artefatos resultantes.

Artefatos Gerados (Outputs)

1. **Relatório de Métricas de Rede (`metricas_rede_sus.csv`):** Tabela analítica contendo os indicadores de centralidade e a classificação do perfil de cada município.
2. **Arquivo de Grafo (`rede_sus.graphml`):** Arquivo XML estruturado representando a topologia completa da rede, arestas ponderadas e atributos dos nós.
3. **Mapa da Rede Hospitalar (`mapa_rede_hospitalar.png`):** Visualização estática de alta resolução do grafo, com codificação de cores e tamanhos baseada nas métricas calculadas.

Módulo de Análise de Risco Geográfico (`src/modelagem/analise_risco_geografico.py`)

Este módulo implementa um *pipeline* de **Machine Learning Espacial** e **Modelagem Multinível**. Seu objetivo é refinar a predição do risco de mortalidade infantil ao integrar variáveis clínicas individuais (do recém-nascido) com variáveis ecológicas e contextuais (do território de residência), permitindo a identificação de áreas geográficas prioritárias para intervenção.

Arquitetura e Fundamentação Metodológica

O sistema adota uma abordagem de determinantes sociais da saúde, partindo da premissa de que o desfecho clínico não é isolado, mas influenciado pelo ambiente. A arquitetura de dados unifica três dimensões:

1. **Dimensão Individual:** Dados biológicos e do parto (SINASC).
2. **Dimensão Socioeconômica:** Índice de Vulnerabilidade Social (IVS - IPEA).
3. **Dimensão de Infraestrutura:** Capacidade instalada de Atenção Primária (CNES).

O modelo preditivo utiliza algoritmos de *Gradient Boosting* (LightGBM) devido à sua capacidade de lidar com dados heterogêneos, valores nulos e relações não-lineares entre variáveis clínicas e geográficas.

Fluxo de Processamento Detalhado

O *pipeline* de execução é segmentado em quatro etapas lógicas de processamento e análise:

1. Engenharia de Atributos Contextuais (Enriquecimento)

- **Ingestão de Dados Externos:**
  - Carrega as bases de dados do **Índice de Vulnerabilidade Social (IVS)**, mapeando indicadores de infraestrutura urbana, capital humano e renda.
  - Processa dados do **CNES** (Cadastro Nacional de Estabelecimentos de Saúde) para contabilizar o número de Unidades Básicas de Saúde (UBS) por município.
- **Cálculo de Indicadores Derivados:**
  - Gera a métrica `DENSIDADE_UBS_10K`: Número de UBS disponíveis por 10.000 habitantes, servindo como *proxy* para acesso à Atenção Primária.
- **Vinculação (*Linkage*):**
  - Realiza a junção (*Merge*) destes dados contextuais com a base de coorte de nascimentos (gerada previamente), utilizando o código do município de residência (`CODMUNRES`) como chave primária.

2. Otimização e Pré-processamento

- **Gestão de Memória:** Aplica técnicas de *downcasting* numérico (ex: conversão de `float64` para `float32`), reduzindo drasticamente o consumo de memória RAM para permitir o processamento de grandes volumes de dados estaduais ou nacionais.
- **Tratamento de Valores Ausentes (Imputação):** Implementa estratégias estatísticas para preenchimento de nulos (mediana) em variáveis contínuas como IVS e Densidade de UBS, garantindo a integridade da matriz de treino.

3. Modelagem Preditiva (Supervisionada)

- **Treinamento:** Ajusta um classificador **LightGBM** (*LGBMClassifier*).
  - **Variável Alvo (Y):** Ocorrência de Óbito Infantil (Binário).
  - **Matriz de Características (X):** Combinação de vetores clínicos (Peso, Apgar, Idade Mãe) e vetores espaciais (IVS, Densidade UBS).
- **Balanceamento de Classes:** Aplica o hiperparâmetro `scale_pos_weight` calculado dinamicamente com base na proporção de casos positivos/negativos, mitigando o viés causado pela raridade do evento óbito (~1%).
- **Explicabilidade Global (SHAP):** Calcula os *SHapley Additive exPlanations* para quantificar a contribuição marginal de cada variável (clínica ou geográfica) na predição do risco, permitindo interpretar se o risco é guiado por fatores biológicos ou ambientais.

4. Análise e Visualização Espacial

- **Inferência em Lote:** Aplica o modelo treinado a toda a população de nascidos vivos para gerar uma probabilidade de risco individual (`RISCO_PREVISTO_OBITO`).
- **Agregação Territorial:** Calcula a média do risco predito por município.
- **Mapeamento Coroplético:** Utiliza `Geopandas` para vincular os riscos médios à malha digital municipal (`.shp`), gerando mapas de calor que evidenciam visualmente as regiões de maior vulnerabilidade epidemiológica.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando, aceitando os seguintes argumentos:

- `--sobrevida-csv` : Caminho para o arquivo CSV de coorte (nascimentos + óbitos vinculados).
- `--ivs-dir` : Diretório contendo os arquivos CSV do Índice de Vulnerabilidade Social (por UF).
- `--shapefile` : Caminho para o arquivo vetorial (`.shp`) da malha municipal brasileira.
- `--ufs` : Lista de siglas das Unidades Federativas alvo da análise.
- `--ano` : Ano de referência da coorte.
- `--dir_saida` : Diretório para persistência dos resultados.

Artefatos Gerados (Outputs)

1. **Painel Geográfico Enriquecido (`.csv`):** Dataset final contendo dados individuais acrescidos das variáveis de contexto (IVS, UBS) e do risco predito.
2. **Gráfico de Importância de Atributos (`feature_importance_geografico.png`):** Visualização baseada em SHAP que hierarquiza os fatores de risco.
3. **Mapas de Risco (`mapa_risco_{uf}.png`):** Imagens de alta resolução projetando o risco médio de mortalidade infantil sobre o território, utilizadas para priorização de gestão.
4. **Dataset para Análise Causal (`dados_para_analise_causal.csv`):** Subconjunto de dados limpos e preparados, exportado para consumo posterior pelo módulo de inferência causal.

Módulo de Mineração de Sequências Clínicas (`src/modelagem/analise_sequencia_eventos.py`)

Este módulo implementa um *pipeline* de **Mineração de Processos em Saúde** (*Healthcare Process Mining*). Sua função é reconstruir a trajetória longitudinal dos pacientes através dos sistemas fragmentados do SUS, integrando dados ambulatoriais (SIA) e hospitalares (SIH) para identificar padrões de utilização de serviços que antecedem um desfecho clínico grave (internação).

Arquitetura e Fundamentação Metodológica

O sistema aborda o desafio da fragmentação de dados (silos de informação) através de uma abordagem de **Vinculação Determinística de Registros** (*Record Linkage*) baseada em chaves compostas, seguida por algoritmos de análise de frequência de sequências (*N-Grams*).

O objetivo analítico é mapear a **Linha de Cuidado Real** *versus* a Linha de Cuidado Prescrita, permitindo a identificação de falhas na Atenção Primária (ex: "Vazio Assistencial") que culminam

em hospitalizações evitáveis.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em três etapas críticas de processamento:

1. Harmonização de Identificadores e Resolução de Entidades

Dada a ausência de um identificador único universal (como CPF) consistente nas bases públicas abertas, o módulo implementa uma heurística de identificação:

- **Geração de Pseudo-ID:** Cria uma chave composta determinística: `ANO_NASCIMENTO + SEXO + COD_MUNICIPIO_RESIDENCIA`.
- **Tratamento de Tipagem Robusta:** Implementa rotinas específicas de limpeza para mitigar erros de formatação comuns em arquivos grandes (ex: conversão de anos formatados como notação científica `1.99E+3` para inteiros `1990`), garantindo a integridade do *linkage* entre as bases SIH e SIA.
- **Normalização Geográfica:** Padroniza os códigos municipais para 6 dígitos em ambas as bases para assegurar a correspondência exata.

2. Definição de Coorte e Reconstrução Temporal (*Backtracking*)

- **Definição do Desfecho (Pivô):** O sistema identifica uma coorte de pacientes que sofreram a internação alvo (definida via parâmetro CID, ex: I50 - Insuficiência Cardíaca).
- **Busca Retroativa:** Para cada paciente identificado no evento pivô, o algoritmo realiza uma varredura na base ambulatorial (SIA) buscando registros em uma janela temporal anterior (tipicamente 12 meses antes da internação).
- **Ordenação Cronológica:** Os procedimentos ambulatoriais encontrados são ordenados por data de competência, formando uma cadeia de eventos (a "Jornada do Paciente").
- **Detecção de Vazio Assistencial:** Caso a busca retroativa não retorne registros, o algoritmo classifica a jornada explicitamente como "SEM_REGISTRO_AMBULATORIAL", evidenciando falha de cobertura ou acesso prévio.

3. Mineração de Padrões e Frequência

- **Agregação de Sequências:** Utiliza a estrutura de dados `Counter` para contabilizar a frequência de ocorrência de cada caminho único (tupla de procedimentos).
- **Tradução Semântica:** Mapeia os códigos de procedimentos (SIGTAP) para descrições legíveis (ex: '0301' $\rightarrow$ 'Consulta Médica'), facilitando a interpretação clínica.
- **Ranking:** Ordena as jornadas da mais frequente para a menos frequente, destacando os padrões dominantes de acesso ao sistema.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando, aceitando argumentos para definição das fontes de dados e do escopo clínico:

- `--sih-parquet`: Caminho para o arquivo de dados hospitalares otimizado (formato Parquet).

- `--sia-parquet` : Caminho para o arquivo de dados ambulatoriais otimizado (formato Parquet).
- `--cid-coorte` : Código CID-10 que define o grupo populacional de interesse.
- `--cid-evento` : Código CID-10 que define o evento desfecho (gatilho da análise).

Artefatos Gerados (Outputs)

1. **Relatório de Sequências (Console/Log):** Listagem das N jornadas mais frequentes, com suas respectivas contagens absolutas e percentuais.
   - *Exemplo de Output:* `Freq: 450 (30.5%) | Consulta -> Exame Lab -> Consulta -> INTERNAÇÃO`.
2. **Arquivo de Resultados (`.csv` - Opcional):** Dependendo da configuração de saída, pode persistir a tabela de frequências para análise posterior em ferramentas de *Business Intelligence* (BI).

Módulo de Análise de Sobrevida Infantil (`src/modelagem/analise_sobrevida_infantil.py`)

Este módulo implementa um *pipeline* completo de **Ciência de Dados Aplicada à Epidemiologia**, abrangendo desde a ingestão e vinculação de bases de dados governamentais distintas (*Record Linkage*) até o treinamento de modelos de **Aprendizado Supervisionado** para classificação de risco. Seu objetivo é prever a probabilidade de óbito infantil com base nas características do nascimento e da infraestrutura hospitalar.

Arquitetura e Fundamentação Metodológica

O sistema resolve o problema da desconexão entre os dados de nascimento e óbito no SUS. A Declaração de Nascido Vivo (SINASC) contém ricos detalhes sobre a gestação e o parto, mas desconhece o desfecho da criança. A Declaração de Óbito (SIM) contém o desfecho, mas carece de detalhes clínicos do nascimento.

O módulo unifica essas informações através de técnicas de **Linkage Determinístico e Probabilístico**, construindo uma coorte histórica rotulada. Para a modelagem preditiva, utiliza-se o algoritmo **LightGBM** devido à sua eficiência em dados tabulares e capacidade nativa de lidar com o desbalanceamento de classes (o evento óbito é raro, tipicamente ~1% da amostra).

Fluxo de Processamento Detalhado

O *pipeline* é executado em cinco etapas sequenciais de processamento de dados e modelagem:

1. Ingestão e Coorte Temporal

- **SINASC:** Carrega os dados de nascimentos para o ano de referência (Ano T).
- **SIM (Janela de Observação):** Carrega os dados de mortalidade para o ano de referência (T) e o ano subsequente ($T+1$). Isso é necessário para capturar a mortalidade infantil completa (óbito < 1 ano de idade) de crianças nascidas no final do ano T.

2. Vinculação de Registros (*Record Linkage*)

Implementa uma estratégia hierárquica para identificar quais nascidos vivos evoluíram para óbito:

- **Chave Forte (Determinística):** Utiliza o número da Declaração de Nascido Vivo (`NUMERODN`), presente em ambas as bases. Este é o método de maior precisão.
- **Chave Fraca (Probabilística/Heurística):** Para registros onde o `NUMERODN` está ausente ou inconsistente no SIM, constrói uma chave composta por `DATA_NASCIMENTO + COD_MUNICIPIO_RESIDENCIA + SEXO` .
- **Definição do Alvo ($Target$):** Cria a variável binária `OBITO_INFANTIL` : `1` se houve correspondência no SIM, `0` caso contrário (censura/sobrevivência).

3. Enriquecimento e Engenharia de Atributos

- **Integração com CNES:** Realiza o cruzamento com o Cadastro Nacional de Estabelecimentos de Saúde utilizando o código da unidade (`CODESTAB` / `CNES`). Adiciona variáveis de infraestrutura crítica ao modelo, como:
  - `QTLEIT39` : Quantidade de leitos de UTI Neonatal.
  - `ATIVIDAD` : Indicador de atividade de ensino/pesquisa.
- **Seleção de *Features*:** Filtra e converte variáveis preditoras essenciais, incluindo fatores biológicos (Peso, Apgar, Idade Gestacional) e sociodemográficos (Escolaridade Materna, Idade Materna, Raça/Cor).

4. Modelagem Preditiva (Classificação)

- **Divisão de Dados:** Segrega os dados em conjuntos de Treino e Teste, mantendo a estratificação pela variável alvo para preservar a proporção de óbitos.
- **Treinamento:** Ajusta um classificador `LGBMClassifier` .
  - Utiliza o hiperparâmetro `scale_pos_weight` (calculado como a razão entre negativos/positivos) para penalizar erros na classe minoritária (óbito), garantindo que o modelo não tendenciosa para a classe majoritária (sobrevivência).
- **Avaliação:** Calcula métricas de desempenho, com foco na **AUC-ROC** (*Area Under the Curve*), que avalia a capacidade de discriminação do modelo independentemente do ponto de corte.

5. Explicabilidade do Modelo (XAI)

- **Cálculo SHAP:** Aplica o método *SHapley Additive exPlanations* para decompor a predição do modelo.
- **Interpretação:** Gera visualizações que hierarquizam as variáveis por impacto no risco global, permitindo validar se o modelo aprendeu relações clínicas coerentes (ex: Baixo peso aumenta o risco).

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas para compor a coorte.
- `--ano` : Ano de referência dos nascimentos (início da coorte).

- `--pop` : Caminho para o arquivo de população (utilizado para normalizações demográficas, se necessário).
- `--dir_saida` : Diretório para persistência dos modelos e relatórios.

Artefatos Gerados (Outputs)

1. **Painel de Dados Enriquecido (`.csv`):** O *dataset* final processado, contendo todas as características do nascimento, dados do hospital e o *flag* de óbito real. Útil para validação cruzada e análises externas.
2. **Modelo Serializado (`.joblib`):** O objeto do modelo treinado, salvo em formato binário para ser carregado posteriormente pela API de inferência ou simuladores.
3. **Relatório de Importância (`shap_summary_bar.png`):** Gráfico de barras indicando quais fatores (ex: Peso, Apgar, UTI) foram mais determinantes para a predição do risco.
4. **Relatório de Distribuição de Impacto (`shap_summary_beeswarm.png`):** Gráfico de dispersão que mostra como os valores altos ou baixos de cada variável impactam o risco (positiva ou negativamente).

Módulo de Análise de Sobrevivência e Readmissão (`src/modelagem/analise_sobrevida_readmissao.py`)

Este módulo implementa um *pipeline* estatístico e de **Análise de Sobrevivência** (*Survival Analysis*) para modelar a dinâmica temporal das readmissões hospitalares. Diferente de modelos de classificação binária (que preveem apenas se o evento ocorrerá), este sistema modela o **tempo até o evento** (*Time-to-Event*), permitindo estimar o risco instantâneo e a probabilidade de não-readmissão ao longo do tempo.

Arquitetura e Fundamentação Metodológica

O sistema trata a readmissão hospitalar como um evento estocástico sujeito à censura. A metodologia baseia-se em dois pilares estatísticos:

1. **Modelo de Riscos Proporcionais de Cox (*Cox Proportional Hazards*):** Uma técnica de regressão semi-paramétrica multivariada utilizada para quantificar o impacto de diversas covariáveis (idade, uso de UTI, diagnóstico) na taxa de risco (*hazard rate*).
2. **Estimador Kaplan-Meier:** Um método não-paramétrico utilizado para estimar e visualizar a função de sobrevivência $S(t)$ da população ou de estratos específicos.

O sistema lida nativamente com **Dados Censurados à Direita** (*Right-Censored Data*), ou seja, pacientes que não retornaram ao hospital até o fim do período de observação do estudo (não significa que nunca retornarão, apenas que o evento não foi observado na janela temporal).

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de processamento de dados longitudinais e modelagem:

1. Ingestão e Estruturação Longitudinal

- **Carregamento de Dados:** Ingestão dos microdados de internação (SIH) e dados cadastrais de estabelecimentos (CNES).
- **Identificação de Pacientes:** Gera um *Pseudo-ID* determinístico para rastrear o mesmo indivíduo através de múltiplas internações ao longo do tempo.
- **Ordenação Cronológica:** Organiza os registros por Paciente e Data de Internação (`DT_INTER`).
- **Cálculo de Delta-T e Evento:**
  - Utiliza operações de deslocamento (*Shift*) para comparar a data de alta da internação atual com a data de admissão da próxima internação.
  - Calcula a variável `dias_ate_readmissao` (tempo até o evento).
  - Define a variável de censura binária (`ocorreu_readmissao`): `1` se houve retorno, `0` caso contrário.
  - Para casos censurados, calcula o tempo decorrido entre a alta e a data final do estudo.

2. Pré-processamento e Engenharia de Atributos

- **Enriquecimento:** Agrega dados do CNES (Natureza Jurídica, Tipo de Unidade) à internação base.
- **Tratamento de Categóricas:** Converte variáveis categóricas (Sexo, Diagnóstico, Procedimentos) utilizando *One-Hot Encoding* (`get_dummies`), gerando variáveis *dummy* binárias para inclusão no modelo de regressão.
- **Limpeza de Variância:** Implementa uma rotina de verificação para remover covariáveis com variância zero (constantes) ou nula, que impediriam a convergência matemática do modelo de Cox.

3. Modelagem Multivariada (Regressão de Cox)

- **Ajuste do Modelo:** Utiliza a biblioteca `lifelines` para ajustar o modelo `CoxPHFitter` .
- **Cálculo de *Hazard Ratios*:** Estima a razão de risco para cada variável.
  - *Interpretação:* Um HR > 1.0 indica que a variável aumenta o risco de readmissão precoce (acelera o evento). Um HR < 1.0 indica um fator protetor (retarda o evento).

4. Visualização e Análise Estratificada

- **Curvas de Sobrevivência:** Gera gráficos de Kaplan-Meier comparando grupos distintos (ex: Pacientes que utilizaram UTI vs. Pacientes que não utilizaram). O eixo Y representa a probabilidade de o paciente permanecer "livre de readmissão" após t dias (Eixo X).
- **Forest Plot:** Gera uma visualização gráfica dos coeficientes e intervalos de confiança do modelo de Cox, facilitando a identificação visual dos fatores de risco significativos.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas para análise.
- `--anos` : Janela temporal de dados históricos (recomenda-se múltiplos anos para capturar readmissões de longo prazo).

- `--dir_saida`: Diretório para persistência dos resultados.

Artefatos Gerados (Outputs)

1. **Modelo Serializado (`modelo_cox_readmissao.joblib`):** Objeto contendo o modelo ajustado, capaz de realizar predições de curvas de sobrevivência para novos pacientes individuais via API.
2. **Relatório Estatístico (Console/Log):** Sumário detalhado contendo coeficientes, *Hazard Ratios*, intervalos de confiança, estatísticas-z e valores-p para cada covariável.
3. **Gráfico de Fatores de Risco (`fatores_risco_cox.png`):** Representação visual dos *Hazard Ratios*.
4. **Curvas de Kaplan-Meier (`curva_kaplan_meier_uti.png`):** Gráfico comparativo da dinâmica de readmissão entre estratos clínicos.

Módulo de Modelagem de Tópicos em Texto (`src/modelagem/analise_topicos_sintomas_nlp.py`)

Este módulo aplica técnicas de **Processamento de Linguagem Natural (NLP)** e **Aprendizado de Máquina Não Supervisionado** para extrair inteligência de dados não estruturados (campos de texto livre) presentes nas fichas de notificação do SINAN.

Sua função é identificar padrões latentes de sintomatologia e descrições clínicas que não são capturados pelos campos estruturados (checkboxes), permitindo a descoberta de novas síndromes, falhas de registro ou especificidades locais de agravos (ex: Acidentes por Animais Peçonhentos).

Arquitetura e Fundamentação Metodológica

O sistema utiliza o algoritmo **LDA (*Latent Dirichlet Allocation*)** para realizar a modelagem de tópicos. O LDA é um modelo probabilístico generativo que assume que:

1. Cada documento (notificação) é uma mistura de diferentes tópicos.
2. Cada tópico é uma distribuição de probabilidade sobre palavras.

O objetivo do algoritmo é reverter este processo gerador para inferir os tópicos latentes a partir dos textos observados. A representação textual utilizada é o modelo **Bag-of-Words (BoW)**, que desconsidera a ordem gramatical, focando na frequência de ocorrência dos termos.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de processamento textual:

1. Ingestão e Concatenação

- **Extração:** Carrega os microdados do SINAN (focando no agravo configurado, ex: `ANIM` - Animais Peçonhentos).
- **Fusão de Campos:** Concatena múltiplas colunas de texto livre (ex: `CLI_LOCA_1` e `CLI_OUTR_3`) em um único documento de texto por paciente (`NOTA_CLINICA`), consolidando toda a informação descritiva disponível.

- **Filtragem:** Descarta registros que não possuem conteúdo textual, otimizando o processamento.

2. Pré-processamento de Texto (Limpeza)

Aplica uma série de transformações para normalizar o texto e reduzir o ruído:

- **Normalização:** Conversão de todo o texto para caixa baixa (*lowercase*).
- **Higienização:** Remoção de pontuação, caracteres especiais e dígitos numéricos via Expressões Regulares (*Regex*).
- **Remoção de *Stopwords*:** Utiliza o corpus da biblioteca `NLTK` para eliminar palavras funcionais de alta frequência e baixo valor semântico (artigos, preposições, conjunções) em português.
- **Filtragem de Tamanho:** Remove tokens (palavras) muito curtos que geralmente resultam de erros de digitação ou abreviações não padronizadas.

3. Vetorização

- **CountVectorizer:** Transforma o corpus textual limpo em uma **Matriz Documento-Termo** (DTM) esparsa.
- **N-Grams:** Configurado para capturar unigramas (1 palavra) e bigramas (2 palavras consecutivas), permitindo identificar termos compostos (ex: "picada aranha", "dor local").
- **Pruning:** Aplica filtros de frequência mínima (`min_df`) e máxima (`max_df`) para descartar termos muito raros (erros) ou onipresentes (que não ajudam a diferenciar tópicos).

4. Decomposição e Inferência (LDA)

- **Treinamento:** O algoritmo LDA decompõe a matriz DTM em duas matrizes menores: Distribuição de Tópicos por Documento e Distribuição de Palavras por Tópico.
- **Atribuição:** Para cada notificação, o modelo calcula qual é o "Tópico Dominante" (aquele com maior probabilidade), permitindo classificar semanticamente o registro (ex: "Caso com Necrose", "Caso com Sintomas Sistêmicos").

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de UFs para compor o corpus textual.
- `--anos` : Janela temporal de análise.
- `--n-topicos` : Hiperparâmetro K que define o número de grupos semânticos a serem encontrados.
- `--n-palavras` : Número de palavras-chave mais relevantes a serem exibidas para descrever cada tópico no relatório.

Artefatos Gerados (Outputs)

1. **Relatório Semântico (Console/Log):**
   - Listagem dos K tópicos descobertos.

- Para cada tópico, apresenta as N palavras-chave com maior peso vetorial (ex: *Tópico 1: [dor, edema, local, dedo, mão]*).
- Exibe exemplos reais de relatos ("snippets") classificados naquele tópico para validação humana.

2. **Enriquecimento de Dados (Em memória/Pipeline):**
   - O *DataFrame* processado ganha a coluna `TOPICO_DOMINANTE`, permitindo que análises estatísticas subsequentes cruzem esses "clusters semânticos" com variáveis estruturadas (ex: O Tópico 1 tem maior taxa de letalidade que o Tópico 2?).

Módulo de Auditoria Ambulatorial Inteligente (`src/modelagem/auditoria_ambulatorial.py`)

Este módulo implementa um *pipeline* híbrido de **Auditoria de Contas Médicas**, combinando sistemas baseados em regras (*Rule-Based Systems*) com métodos estatísticos de detecção de anomalias. Sua função é processar massas de dados de produção ambulatorial (SIA/SUS) para identificar inconsistências de faturamento, desvios de protocolo clínico e *outliers* volumétricos que sugerem erros administrativos ou fraudes.

Arquitetura e Fundamentação Metodológica

O sistema opera sob uma arquitetura de **Validação em Camadas**, onde os dados são submetidos a múltiplos motores de verificação sequenciais. A metodologia integra dados transacionais (produção) com dados cadastrais (infraestrutura), permitindo a aplicação de regras de negócio contextuais.

A abordagem analítica divide-se em:

1. **Auditoria Determinística:** Verificação de conformidade contra regras de negócio rígidas (ex: Tabela SIGTAP, compatibilidades).
2. **Auditoria Estatística:** Identificação de desvios de padrão baseada na distribuição normal dos dados (Escore-Z).

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas lógicas:

1. Ingestão e Enriquecimento Cadastral (ETL)

- **Carregamento:** Ingestão dos microdados de Produção Ambulatorial (SIA - Grupo PA) e dos dados cadastrais de estabelecimentos (CNES - Grupo ST).
- **Integração (Join):** Realiza o cruzamento das bases utilizando as chaves `PA_CODUNI` (Código da Unidade no SIA) e `CNES` (Código no Cadastro Nacional).
- **Normalização:** Padroniza tipagens de dados (conversão de códigos para *string*, valores monetários e quantidades para *float*) para garantir a integridade das comparações lógicas.

2. Motor de Regras de Negócio (Auditoria Clínica/Administrativa)

Aplica filtros lógicos para detectar incompatibilidades explícitas:

- **Compatibilidade de Complexidade:** Identifica a realização de procedimentos de Alta Complexidade em unidades classificadas como Atenção Básica (ex: Postos de Saúde), indicando erro de classificação ou faturamento indevido.
- **Compatibilidade de Sexo:** Cruza o sexo do paciente (`PA_SEXO`) com o procedimento realizado (`PA_PROC_ID`) para detectar inconsistências biológicas (ex: Procedimentos ginecológicos/obstétricos em pacientes do sexo masculino).

3. Motor Estatístico (Detecção de *Outliers* Volumétricos)

Analisa a distribuição da quantidade aprovada (`PA_QTDAPR`) para cada tipo de procedimento:

- **Segmentação:** Agrupa os dados por Código do Procedimento (`PA_PROC_ID`).
- **Cálculo de Parâmetros:** Calcula a média (μ) e o desvio padrão (σ) da produção para cada procedimento na amostra analisada.
- **Cálculo do Z-Score:** Aplica a fórmula $Z = (X - \mu)/\sigma$ para cada registro.
- **Filtragem de Anomalias:** Isola registros cujo *Z-Score* ultrapassa um limiar pré-definido (padrão: 3.5), indicando que o volume produzido está estatisticamente muito acima do padrão da rede, sugerindo *overcoding* ou digitação errônea.

4. Consolidação e Reporte

- Unifica os alertas gerados pelos motores determinístico e estatístico em um único *DataFrame*.
- Atribui uma etiqueta descritiva (`MOTIVO_ALERTA`) a cada registro flagrado, facilitando a triagem por auditores humanos.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas a serem auditadas.
- `--anos` : Janela temporal da auditoria.
- `--dir_saida` : Diretório para persistência do relatório final.

Artefatos Gerados (Outputs)

1. **Relatório de Auditoria Consolidado (`relatorio_auditoria.csv`):** Arquivo tabular contendo a lista de procedimentos suspeitos.
   - **Colunas Chave:** Identificação do Prestador, Código do Procedimento, Quantidade/Valor Apresentado e o Motivo do Alerta (ex: "Procedimento incompatível com sexo", "Volume estatisticamente anômalo").
   - **Utilidade:** Serve como insumo direto para equipes de regulação e auditoria do SUS, permitindo a priorização de contas médicas para revisão manual (glosa ou aprovação).

Módulo de Inferência de Risco Individual (`src/modelagem/forecast_internacoes.py`)

Este módulo atua como o **Motor de Inferência** (*Inference Engine*) via linha de comando (CLI) para o modelo de risco de readmissão hospitalar. Sua função é operacionalizar o modelo treinado

(`LightGBM`), permitindo a realização de predições pontuais para novos pacientes, acompanhadas de uma explicação detalhada dos fatores que contribuíram para aquele risco específico.

Arquitetura e Fundamentação Metodológica

O componente foi desenhado para simular um cenário de produção clínica, onde dados de um paciente singular são submetidos ao modelo para obtenção de um escore de risco em tempo real.

A arquitetura baseia-se em dois pilares:

1. **Consumo de Modelos Serializados:** Carrega artefatos persistidos (`.joblib` e `.parquet`) para garantir que a inferência utilize exatamente os mesmos hiperparâmetros e estrutura de dados definidos durante a etapa de treinamento.
2. **Explicabilidade Local (Local XAI):** Diferente dos módulos anteriores que focavam na importância global das variáveis, este módulo utiliza **SHAP Waterfall Plots** para decompor a predição de *uma única instância*. Ele quantifica matematicamente quanto cada atributo do paciente (ex: Idade, UTI) contribuiu para elevar ou diminuir o risco em relação à média populacional (Base Value).

Fluxo de Processamento Detalhado

O fluxo de execução segue o padrão de um microsserviço de predição:

1. Carregamento de Artefatos (Inicialização)

- **Modelo Preditivo:** Deserializa o arquivo `lgbm_readmission_model.joblib` contendo o classificador treinado.
- **Schema de Referência:** Carrega o arquivo `X_test.parquet` (ou similar) para reconstruir o objeto `TreeExplainer` do SHAP e garantir o alinhamento dos tipos de dados.

2. Ingestão e Alinhamento de Esquema (*Schema Enforcement*)

- **Parsing de Argumentos:** Recebe os dados clínicos do paciente via argumentos de linha de comando.
- **Vetorização:** Converte os inputs escalares em um *DataFrame* de uma única linha.
- **Alinhamento:** Executa a operação de `reindex` para garantir que o vetor de entrada possua exatamente as mesmas colunas, na mesma ordem e com os mesmos tipos de dados (ex: conversão para `category` do pandas) que o modelo espera. Esta etapa previne erros de dimensão ou tipagem durante a inferência.

3. Inferência Probabilística

- Executa o método `predict_proba` do modelo LightGBM.
- Extrai a probabilidade da classe positiva (`1`), retornando o **Score de Risco de Readmissão** (0 a 100%).

4. Geração de Explicabilidade (SHAP)

- Calcula os *SHAP values* para a instância fornecida.

- Constrói um gráfico do tipo **Waterfall**, que visualiza o "Valor Base" (risco médio da população) e como cada característica do paciente empurrou esse risco para cima (vermelho) ou para baixo (azul), resultando no score final.

Parâmetros de Execução (CLI)

O script aceita argumentos que representam o vetor de características do paciente:

- `--idade` : Idade do paciente (inteiro).
- `--dias_perm` : Tempo de permanência da internação atual.
- `--comorbidades` : Número de diagnósticos secundários.
- `--usou_uti` : Indicador binário (1=Sim, 0=Não).
- `--sexo` : Código do sexo (1=Masc, 3=Fem).
- `--raca_cor` : Código da raça/cor.
- `--car_int` : Caráter da internação (Eletivo/Urgência).
- `--diag_princ` : CID-10 do diagnóstico principal.
- `--hosp_natureza` : Código da natureza jurídica do hospital.
- `--hosp_leitos` : Capacidade do hospital.
- `--hosp_ensino` : Indicador se é hospital de ensino.
- `--dir_saida` : Diretório para persistência dos resultados da inferência.

Artefatos Gerados (Outputs)

1. **Log de Predição (** `previsao_paciente_individual.csv` **):** Arquivo contendo os dados de entrada e o `RISCO_READMISSAO_30D` calculado.
2. **Dados Explicativos (** `shap_explanation_data.csv` **):** Tabela contendo os valores SHAP brutos para cada *feature*, permitindo renderização em *front-ends* customizados.
3. **Visualização de Decisão (** `previsao_paciente_individual.png` **):** Gráfico *Waterfall* estático, pronto para ser anexado a um prontuário eletrônico ou relatório clínico, justificando a decisão do algoritmo para a equipe médica.

Módulo de Geração de Painel Geográfico (`src/modelagem/gerar_painel_geografico.py`)

Este módulo atua como um componente de **ETL (Extração, Transformação e Carga)** dedicado à construção de um *datamart* de contexto territorial. Sua função é consolidar indicadores socioeconômicos e de infraestrutura de saúde em nível municipal, criando um substrato de dados ecológicos que será utilizado para enriquecer modelos preditivos individuais ou para análises espaciais.

Arquitetura e Fundamentação Metodológica

O sistema implementa uma abordagem de **Enriquecimento de Dados** (*Data Enrichment*), integrando fontes externas heterogêneas para caracterizar o território. A arquitetura baseia-se na fusão de duas dimensões analíticas:

1. **Dimensão Social (Vulnerabilidade):** Utiliza o Índice de Vulnerabilidade Social (IVS), que agrega indicadores de capital humano, renda e trabalho, e infraestrutura urbana.
2. **Dimensão de Oferta (Capacidade Instalada):** Utiliza dados do Cadastro Nacional de Estabelecimentos de Saúde (CNES) para quantificar a presença física do Estado (Atenção Primária) no território.

O resultado é um painel transversal (*cross-sectional*) indexado pelo Código IBGE do município.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em três etapas de processamento e integração:

1. Ingestão e Padronização do IVS

- **Varredura de Diretório:** O script itera sobre os arquivos CSV contidos no diretório de referência (`--ivs-dir`), buscando correspondências para as UFs solicitadas. Implementa lógica de tratamento para variações de nomenclatura de arquivos (maiúsculas/minúsculas).
- **Normalização de Chaves:** Identifica e padroniza a coluna de identificação municipal (Código IBGE), convertendo-a para o formato de 6 dígitos (`CODMUNRES`) para garantir a interoperabilidade com as bases do SUS.

2. Mineração de Infraestrutura (CNES)

- **Extração:** Utiliza a biblioteca `PySUS` para baixar os microdados do grupo `ST` (Estabelecimentos) do CNES referente ao mês de dezembro do ano analítico.
- **Filtragem de Tipologia:** Aplica um filtro rígido para selecionar apenas estabelecimentos classificados como **Unidade Básica de Saúde** (`TP_UNID == '02'`). Esta escolha metodológica foca na capilaridade da Atenção Primária como *proxy* de acesso à saúde.
- **Agregação:** Executa uma operação de contagem (`groupby().size()`) para determinar o número absoluto de UBSs ativas por município.

3. Integração e Consolidação

- **Junção (*Merge*):** Realiza o cruzamento da base socioeconômica (IVS) com a base de infraestrutura (Contagem de UBS) utilizando o Código IBGE como chave.
- **Tratamento de Nulos:** Preenche com zero (`0`) a contagem de UBS para municípios que não apresentaram registros no CNES, assumindo ausência de infraestrutura do tipo filtrado.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ivs-dir` : Caminho para o diretório contendo os arquivos CSV brutos do IVS (segregados por UF).
- `--ufs` : Lista de siglas das Unidades Federativas a serem processadas.
- `--ano` : Ano de referência para a extração do "retrato" da infraestrutura no CNES.
- `--dir_saida` : Diretório para persistência do painel consolidado.

Artefatos Gerados (Outputs)

1. **Painel Geográfico Consolidado (`.csv`):** Arquivo tabular contendo uma linha por município.
   - **Colunas:** Código IBGE (`CODMUNRES`), Indicadores do IVS (ex: `ivs` , `ivs_infraestrutura`) e Contagem de UBS (`N_UBS`).
   - **Utilidade:** Este artefato serve como entrada direta para o módulo `analise_risco_geografico.py` e para o módulo `pipeline_alocacao_recursos_aps.py` , fornecendo as covariáveis ambientais necessárias para a modelagem multinível.

Módulo de Predição de Abandono do Tratamento de Tuberculose (`src/modelagem/modelo_risco_abandono_tb.py`)

Este módulo implementa um *pipeline* de **Aprendizado Supervisionado** focado na aderência terapêutica. Sua função é processar dados de notificação de Tuberculose (SINAN) para treinar um classificador capaz de identificar, no momento do diagnóstico, pacientes com alta probabilidade de abandonar o tratamento antes da cura.

Arquitetura e Fundamentação Metodológica

O sistema modela o problema como uma **Classificação Binária**. A arquitetura baseia-se na premissa de que o abandono não é um evento aleatório, mas influenciado por determinantes sociais, clínicos e comportamentais registráveis na ficha de notificação.

- **Algoritmo:** Utiliza o **LightGBM** (*Light Gradient Boosting Machine*), selecionado por sua robustez no tratamento de variáveis categóricas esparsas (comuns no SINAN) e eficiência computacional.
- **Definição de Escopo:** O modelo isola o comportamento de aderência filtrando o *dataset* apenas para os desfechos de interesse: **Cura** (Sucesso) *versus* **Abandono** (Falha de Aderência). Óbitos, transferências e falhas terapêuticas por resistência medicamentosa são excluídos desta análise específica para não introduzir ruído na modelagem do comportamento de abandono.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas críticas:

1. Ingestão e Filtragem de Escopo

- **Extração:** Carrega os microdados do SINAN-Tuberculose para a janela temporal definida.
- **Filtro de Desfecho:** Seleciona apenas os registros onde a situação de encerramento (`SITUA_ENCE`) é classificada como 'Cura' ou 'Abandono'.
- **Binarização do Alvo:** Cria a variável dependente `ALVO_ABANDONO` :
  - `1` (Classe Positiva): Abandono.
  - `0` (Classe Negativa): Cura.

2. Pré-processamento e Engenharia de Atributos

- **Decodificação de Idade:** Implementa uma rotina de *parsing* para a variável `NU_IDADE_N` do SINAN, que utiliza um formato composto (código de unidade temporal + valor), convertendo-a para uma idade numérica contínua em anos.
- **Seleção de *Features*:** Isola variáveis preditoras relevantes, incluindo:
  - **Sociodemográficas:** Sexo, Raça/Cor, Escolaridade.
  - **Comorbidades e Agravos:** Alcoolismo, Diabetes, Doença Mental, HIV, Tabagismo (`AGRA_*`).
  - **Clínicas/Operacionais:** Forma clínica (Pulmonar/Extrapulmonar), Baciloscopia, Raio-X e regime de Tratamento Supervisionado (TDO).
- **Codificação Categórica:** Aplica *One-Hot Encoding* (`get_dummies`) para transformar variáveis qualitativas em vetores numéricos binários, tratando adequadamente valores ausentes ou ignorados (comum em fichas de notificação).

3. Treinamento e Balanceamento

- **Estratégia de Divisão:** Segrega os dados em conjuntos de Treino e Teste (70/30), mantendo a estratificação pela classe de abandono.
- **Ponderação de Classes:** Calcula e aplica o hiperparâmetro `scale_pos_weight` para compensar o desbalanceamento natural do *dataset* (onde a taxa de cura é tipicamente muito superior à de abandono), forçando o modelo a dar maior importância aos erros de classificação na classe de abandono.

4. Avaliação e Explicabilidade (XAI)

- **Métricas de Desempenho:** Avalia o modelo utilizando a área sob a curva ROC (**AUC-ROC**), métrica que independe do limiar de decisão.
- **Interpretação Global (SHAP):** Gera um gráfico de importância de variáveis (*Summary Plot*) que permite visualizar não apenas quais fatores são mais relevantes (ex: Alcoolismo, TDO), mas também a direção do impacto (ex: TDO reduz o risco, Alcoolismo aumenta).

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--anos` : Lista de anos de notificação para compor a base histórica de treinamento.
- `--dir_saida` : Diretório para persistência dos gráficos de análise e métricas.

Artefatos Gerados (Outputs)

1. **Relatório de Performance (Console/Log):** Exibe a acurácia, precisão, recall, f1-score e AUC do modelo no conjunto de teste.
2. **Gráfico de Importância de Fatores (`feature_importance_abandono_tb.png`):** Visualização baseada em SHAP *values* que hierarquiza os determinantes do abandono. Este artefato é crucial para gestores, pois evidencia alavancas de intervenção (ex: reforçar a estratégia de Tratamento Supervisionado em grupos de risco identificados).

Módulo de Predição de Risco Perinatal (`src/modelagem/modelo_risco_perinatal.py`)

Este módulo implementa um *pipeline* de **Aprendizado de Máquina Supervisionado** focado na saúde materno-infantil. Sua função é processar dados da Declaração de Nascido Vivo (SINASC) para treinar um classificador capaz de prever a probabilidade de ocorrência de desfechos neonatais adversos imediatos.

Arquitetura e Fundamentação Metodológica

O sistema aborda o risco perinatal como um problema de **Classificação Binária**. Diferente da predição de óbito (que é um evento raro e extremo), este modelo foca em indicadores intermediários de fragilidade que são estatisticamente mais frequentes e clinicamente acionáveis.

- **Definição do Alvo (*Target*):** O modelo utiliza um desfecho composto. A variável alvo `ALVO_RISCO` é definida como positiva (`1`) se o recém-nascido apresentar **Baixo Peso ao Nascer** (< 2.500g) **OU Prematuridade** (Idade Gestacional < 37 semanas).
- **Algoritmo:** Utiliza o **LightGBM** (*Light Gradient Boosting Machine*), escolhido pela alta performance em dados tabulares e capacidade de modelar interações não lineares entre variáveis obstétricas e sociodemográficas.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de processamento de dados e modelagem:

1. Ingestão e Consolidação de Dados

- **Extração:** Utiliza a biblioteca `PySUS` para baixar e processar os microdados do SINASC para as UFs e anos solicitados.
- **Consolidação:** Concatena múltiplos arquivos anuais/estaduais em um único *DataFrame* pandas para compor uma base de treinamento robusta.

2. Pré-processamento e Engenharia de Atributos

- **Seleção de *Features*:** Isola variáveis preditoras disponíveis no momento do nascimento, incluindo:
  - **Fatores Maternos:** Idade, Escolaridade, Estado Civil, Raça/Cor.
  - **Histórico Obstétrico:** Paridade (filhos vivos/mortos), Tipo de Gravidez.
  - **Assistência Pré-Natal:** Número de consultas, Mês de início do acompanhamento.
  - **Parto:** Via de parto (Vaginal/Cesáreo).
- **Limpeza de Dados:** Remove registros com valores ausentes (`NaN`) nas colunas críticas para o treinamento, garantindo a integridade matemática da matriz de entrada.
- **Construção do Alvo:** Implementa a lógica booleana para derivar a coluna `ALVO_RISCO` a partir das colunas `PESO` e `GESTACAO`.

3. Treinamento e Otimização

- **Estratégia de Divisão:** Segrega os dados em conjuntos de Treino e Teste (70/30), utilizando amostragem estratificada para manter a proporção original de casos de risco em ambos os *sets*.
- **Tratamento de Desbalanceamento:** Calcula automaticamente a razão entre as classes negativa e positiva para ajustar o hiperparâmetro `scale_pos_weight`, instruindo o algoritmo a penalizar mais severamente os erros de classificação nos casos de risco.

4. Avaliação e Persistência

- **Métricas de Performance:** Avalia o modelo utilizando o relatório de classificação (Precisão, Revocação, F1-Score) e a área sob a curva ROC (**AUC-ROC**).
- **Explicabilidade (SHAP):**
  - Calcula os valores SHAP para o conjunto de teste.
  - Gera visualizações (*Bar Plot* e *Beeswarm Plot*) para interpretar quais fatores (ex: Baixa escolaridade, Poucas consultas) estão impulsionando a predição de risco.
- **Serialização:** Exporta o modelo treinado para um arquivo `.joblib`, permitindo que ele seja reutilizado posteriormente por módulos de inferência (calculadoras de risco) ou simulação de políticas públicas.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs`: Lista de siglas das Unidades Federativas para compor a base de treino.
- `--anos`: Lista de anos de referência dos dados do SINASC.
- `--dir_saida`: Diretório para persistência do modelo serializado e dos gráficos explicativos.

Artefatos Gerados (Outputs)

1. **Modelo Serializado (`modelo_risco_perinatal.joblib`):** Objeto binário contendo o classificador treinado. Este é o artefato central consumido por outros scripts do sistema (como `usar_calculadora_risco.py` e `simulador_impacto_politicas.py`).
2. **Relatório de Importância Global (`shap_summary_bar.png`):** Gráfico hierarquizando as variáveis pelo impacto médio absoluto no risco.
3. **Relatório de Impacto Direcional (`shap_summary_beeswarm.png`):** Gráfico detalhando como valores altos ou baixos de cada variável influenciam a probabilidade (ex: demonstrar visualmente que o aumento no número de consultas pré-natal reduz a probabilidade de risco).

Módulo de Classificação de Risco de Readmissão (`src/modelagem/modelo_risco_readmissao_classificacao.py`)

Este módulo implementa um *pipeline* de **Aprendizado de Máquina Supervisionado** otimizado para ambientes com restrição de recursos computacionais. Sua função é prever a probabilidade de um paciente retornar ao sistema hospitalar (readmissão) em um intervalo de curto a médio prazo (90 dias) após a alta, permitindo a estratificação de risco para gestão de altas.

Arquitetura e Fundamentação Metodológica

O sistema aborda a readmissão como um problema de **Classificação Binária**. A arquitetura destaca-se pela ênfase na engenharia de dados eficiente e na construção de uma visão longitudinal a partir de registros transacionais.

- **Definição do Alvo (*Target*):** A variável dependente `ALVO_READMITIDO_90D` é binária:
  - `1` : O paciente foi internado novamente em qualquer hospital da rede no intervalo $0 \leq \Delta t \leq 90$ dias.
  - `0` : O paciente não retornou ou retornou após 90 dias (considerando uma janela de observação válida para evitar censura espúria).
- **Algoritmo:** Utiliza o **LightGBM**, configurado para alta performance em *datasets* volumosos.
- **Gestão de Recursos:** Implementa rotinas agressivas de *downcasting* de tipos numéricos para minimizar o *footprint* de memória RAM durante o processamento de grandes estados ou séries históricas longas.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de manipulação de dados e modelagem:

1. Ingestão e Otimização de Memória

- **Carregamento Otimizado:** Ingestão iterativa dos dados do SIH e CNES.
- **Downcasting Automático:** A função `otimizar_memoria` analisa a amplitude de valores de cada coluna numérica e converte os tipos de dados para a menor representação possível (ex: `int64` $\rightarrow$ `int16`, `float64` $\rightarrow$ `float32`). Isso reduz o consumo de memória em até 70%, viabilizando a análise de grandes coortes.

2. Construção da Coorte Longitudinal

- **Identificação de Pacientes:** Gera um *Pseudo-ID* determinístico (`NASC + SEXO + MUN_RES`) para vincular internações distintas ao mesmo indivíduo.
- **Ordenação Temporal:** Classifica os registros por paciente e data de internação.
- **Cálculo de Intervalos (*Lead/Lag*):** Utiliza operações de deslocamento vetorial (`shift`) para acessar a data da **próxima** internação na linha da internação **atual**.
- **Engenharia do Alvo:** Calcula a diferença em dias entre a alta atual e a próxima admissão. Registros sem retorno ou com retorno > 90 dias são classificados como negativos, respeitando a janela de censura (garantindo que houve tempo de observação suficiente).

3. Enriquecimento e Preparação

- **Integração Cadastral:** Realiza *Merge* com dados do CNES para agregar características do prestador (Tipo de Unidade, Natureza Jurídica) ao perfil clínico do paciente.
- **Seleção de *Features*:** Consolida variáveis demográficas (Idade, Sexo), clínicas (Diagnóstico Principal, Uso de UTI) e administrativas (Tempo de Permanência, Caráter da Internação).

4. Treinamento e Interpretação

- **Modelagem:** Treina o classificador LightGBM com estratégia de divisão Treino/Teste (70/30) estratificada.

- **Avaliação de Desempenho:** Mensura a eficácia do modelo através de métricas de precisão, revocação e AUC-ROC.
- **Importância de Atributos:** Extrai a importância intrínseca das variáveis (*Feature Importance* baseada em ganho de informação ou frequência de *split* do LightGBM) para identificar os principais preditores de readmissão (ex: Idade, Dias de Permanência).

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas para análise.
- `--anos` : Janela temporal de dados históricos.
- `--dir_saida` : Diretório para persistência dos resultados gráficos.

Artefatos Gerados (Outputs)

1. **Relatório de Métricas (Console/Log):** Exibe a performance do classificador (Relatório de Classificação do Scikit-Learn e AUC).
2. **Gráfico de Importância (** `feature_importance_readmissao_90d.png` **):** Gráfico de barras (*Barplot*) que hierarquiza as variáveis preditoras, permitindo aos gestores focar em fatores de risco monitoráveis (ex: pacientes com longas estadias prévias têm maior risco de retorno).

Módulo de Otimização de Referência e Contra-referência (`src/modelagem/otimizador_referencia_pacientes.py`)

Este módulo implementa um sistema de **Análise Prescritiva** e **Simulação de Cenários** (*What-If Analysis*). Sua função é auxiliar na tomada de decisão regulatória, identificando qual unidade hospitalar de destino maximiza a probabilidade de sobrevida de pacientes de alto risco, considerando restrições geográficas e econômicas.

Arquitetura e Fundamentação Metodológica

O sistema opera como um **Sistema de Apoio à Decisão (SAD)** que combina modelagem preditiva com técnicas de otimização multicritério.

- **Simulação Contrafactual:** O algoritmo utiliza um modelo de *Machine Learning* previamente treinado para estimar desfechos clínicos sob diferentes condições. Para cada paciente, o sistema simula: *"Qual seria a probabilidade de óbito se este paciente fosse atendido no Hospital A? E no Hospital B?"*. Isso é realizado injetando as características estruturais do hospital candidato (ex: presença de UTI Neonatal, vínculo de ensino) no vetor de características do paciente.
- **Otimização Multiobjetivo:** Calcula um *score* final de adequação baseado em três dimensões conflitantes:
  1. **Desfecho Clínico:** Probabilidade de Sobrevida (Maximizar).
  2. **Custo Operacional:** Custo estimado da internação (Minimizar).
  3. **Logística:** Distância geográfica entre origem e destino (Minimizar).

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de integração de dados espaciais e inferência estatística:

1. Carregamento de Artefatos e Georreferenciamento

- **Desserialização:** Carrega o modelo de risco de óbito (`.joblib`) treinado pelo módulo de sobrevida.
- **Processamento Vetorial:** Ingestão de arquivos GeoJSON municipais. Utiliza a biblioteca `Geopandas` para calcular os centroides geométricos de cada município, convertendo polígonos em coordenadas pontuais (Latitude/Longitude) para cálculo de distâncias.

2. Definição de Cenário (Origem e Destino)

- **Definição da Coorte:** Seleciona do painel de dados histórico um subconjunto de pacientes de alto risco (ex: prematuros) residentes no município de origem especificado.
- **Mapeamento da Oferta:** Identifica todos os hospitais nas UFs de referência que possuem capacidade instalada compatível (ex: `QTLEIT39 > 0` para UTI Neonatal), constituindo o espaço de busca de soluções.

3. Motor de Simulação (*Inference Loop*)

Para cada par `(Paciente, Hospital Candidato)` , o sistema executa:

- **Fusão de Atributos:** Constrói um vetor de *features* sintético combinando os dados biológicos do paciente (imutáveis) com os dados estruturais do hospital candidato (variáveis).
- **Predição de Risco:** Submete este vetor ao modelo `LightGBM` para obter a probabilidade de óbito (P_{obito}).
- **Cálculo de Distância:** Aplica a fórmula de **Haversine** para calcular a distância geodésica (em km) entre o centroide do município de residência e a localização do hospital.

4. Ranking e Pontuação (*Scoring*)

- **Normalização:** Aplica escala Min-Max para transformar probabilidades, custos e distâncias em valores adimensionais entre 0 e 1.
- **Função de Custo Ponderada:** Calcula o `SCORE_FINAL` utilizando pesos pré-definidos para cada dimensão (Padrão: 60% Sobrevida, 20% Custo, 20% Distância).
- **Ordenação:** Gera uma lista classificada das melhores opções de referência.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--painel-base-csv` : Caminho para o *dataset* contendo os dados históricos dos pacientes.
- `--modelo-sobrevida` : Caminho para o arquivo binário do modelo preditivo.
- `--geojson-dir` : Diretório contendo os arquivos de geometria municipal.
- `--municipio-origem` : Código IBGE (6 dígitos) do município foco da análise (ponto de partida).
- `--ufs-referencia` : Lista de UFs onde o sistema deve buscar hospitais receptores.

- `--dir_saida`: Diretório para exportação do relatório.

Artefatos Gerados (Outputs)

1. **Relatório de Ranking (`ranking_referencia_*.csv`):** Tabela analítica ordenada por *score*, contendo:
   - Identificação do Hospital de Destino e Município.
   - Probabilidade média simulada de sobrevida para a coorte naquele hospital.
   - Distância média de deslocamento.
   - Custo estimado e Score Final.

Utilidade: Este artefato serve como insumo direto para as Centrais de Regulação de Leitos, fornecendo evidência matemática para justificar a transferência de pacientes para unidades que, embora mais distantes ou custosas, ofereçam ganho marginal significativo na probabilidade de sobrevivência.

Módulo de Priorização de Alocação de Recursos (`src/modelagem/pipeline_alocacao_recursos_aps.py`)

Este módulo implementa um *pipeline* de **Análise de Decisão Multicritério** (*Multi-Criteria Decision Analysis* - MCDA) focado no planejamento estratégico de saúde. Sua função é processar indicadores epidemiológicos, demográficos e sociais para gerar um **Índice de Priorização de Investimento**, permitindo que gestores identifiquem objetivamente quais municípios apresentam a maior necessidade de expansão da cobertura da Atenção Primária à Saúde (APS).

Arquitetura e Fundamentação Metodológica

O sistema baseia-se na construção de um **Índice Composto**. A metodologia aborda o problema da incomparabilidade entre diferentes métricas (ex: taxas de mortalidade *versus* cobertura de equipes) através de técnicas de normalização estatística.

A arquitetura é flexível e orientada a configuração: as dimensões de análise, os indicadores constituintes e suas respectivas polaridades (se "quanto maior, melhor" ou "quanto maior, pior") são definidos externamente em um arquivo JSON, desacoplando a regra de negócio da lógica de processamento.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em três etapas de integração de dados e cálculo aritmético:

1. Ingestão e Fusão de Dados (*Data Fusion*)

- **Carregamento:** Ingere dois *datamarts* principais:
  1. **Painel de Indicadores:** Contendo taxas epidemiológicas e de cobertura (saída de `integrar_indicadores.py`).
  2. **Painel Geográfico:** Contendo dados de vulnerabilidade social (IVS) e infraestrutura (saída de `analise_risco_geografico.py` ou similar).

- **Integração:** Realiza a união (*Merge*) das bases utilizando o código IBGE do município e o ano de referência como chaves primárias, consolidando todas as dimensões de análise em uma única estrutura tabular.

2. Normalização e Tratamento de Polaridade

Para permitir a agregação de indicadores com unidades de medida distintas, o módulo aplica o algoritmo **Min-Max Scaling** com intervalo de [0, 100]:

- **Transformação Linear:** Converte os valores brutos de cada indicador para uma escala centesimal relativa à distribuição observada na amostra.
- **Inversão de Polaridade:**
  - Para indicadores onde "Alto é Bom" (ex: Cobertura de Equipes): Mantém a escala original.
  - Para indicadores onde "Alto é Ruim" (ex: Mortalidade Infantil, IVS): Inverte a escala ($100 - Score$), garantindo que, no índice final, valores altos sempre representem **maior necessidade/prioridade**.

3. Agregação Dimensional e Ranking

- **Cálculo de Dimensões:** Calcula a média aritmética dos scores normalizados dos indicadores que compõem cada dimensão (ex: Dimensão Epidemiológica, Dimensão Social), gerando sub-índices.
- **Índice Geral:** Calcula a média dos sub-índices para gerar o `INDICE_PRIORIZACAO_FINAL`.
- **Ordenação:** Gera um *ranking* decrescente dos municípios, onde o topo da lista representa os territórios com a combinação mais crítica de alta vulnerabilidade e baixa cobertura assistencial.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--indicadores-csv` : Caminho para o arquivo CSV do painel de indicadores mestre.
- `--geo-csv` : Caminho para o arquivo CSV com dados geográficos/contextuais (IVS).
- `--ano` : Ano de referência para o corte transversal da análise.
- `--dimensoes-arquivo-json` : Caminho para o arquivo de configuração que define a estrutura do índice (indicadores e pesos).
- `--dir-saida` : Diretório para persistência do relatório de priorização.

Artefatos Gerados (Outputs)

1. **Relatório de Priorização (`ranking_priorizacao_aps_*.csv`):** Arquivo tabular ordenado por prioridade.
   - **Colunas:** Identificação do Município, População, Scores Normalizados de cada indicador individual, Scores das Dimensões Agregadas e o Índice de Priorização Final.
   - **Utilidade:** Ferramenta tática para alocação de verbas (emendas parlamentares, portarias de expansão) e distribuição de novas equipes de Saúde da Família, garantindo equidade na distribuição de recursos baseada em critérios técnicos.

Módulo de Análise de Jornada do Paciente Crônico (`src/modelagem/pipeline_doencas_cronicas.py`)

Este módulo implementa um *pipeline* de **Engenharia de Dados** e **Modelagem Preditiva Longitudinal**. Sua função primordial é solucionar o problema da fragmentação de dados no SUS, integrando registros ambulatoriais (SIA) e hospitalares (SIH) para reconstruir a linha de cuidado de pacientes com condições crônicas. O objetivo analítico é predizer o risco de hospitalização futura com base no histórico de utilização de serviços de atenção primária e secundária.

Arquitetura e Fundamentação Metodológica

O sistema adota uma arquitetura de **Vinculação de Registros** (*Record Linkage*) determinística baseada em heurísticas, necessária devido à ausência de um identificador único universal (como CPF) nas bases públicas.

A metodologia de modelagem utiliza janelas temporais deslizantes (*Sliding Windows*):

1. **Janela de Observação (Histórico):** Período anterior (ex: Ano $T-1$) onde são coletadas as variáveis preditoras (número de consultas, exames, internações prévias).
2. **Janela de Predição (Alvo):** Período futuro (ex: Ano T) onde verifica-se a ocorrência do desfecho (nova internação).

Fluxo de Processamento Detalhado

O *pipeline* é executado em quatro etapas críticas de processamento, otimização e análise:

1. Ingestão e Otimização de Recursos (*Memory Optimization*)

- **Extração e Caching:** O script implementa um mecanismo de persistência intermediária. Na primeira execução, baixa os dados brutos do DATASUS, aplica rotinas de limpeza e salva versões otimizadas em formato **Parquet**. Nas execuções subsequentes, carrega diretamente os arquivos Parquet, reduzindo drasticamente o tempo de I/O.
- **Downcasting Numérico:** Aplica a função `otimizar_memoria` para reduzir a precisão de tipos numéricos (ex: `float64` $\rightarrow$ `float32`, `int64` $\rightarrow$ `int16`) e converter colunas de alta cardinalidade para o tipo `category`, minimizando o consumo de RAM.

2. Harmonização e Identificação de Entidades

- **Geração de Pseudo-ID:** Constrói uma chave composta para rastrear o indivíduo entre as bases heterogêneas:
  - **SIH:** `ANO_NASCIMENTO + SEXO + MUNICIPIO_RESIDENCIA`.
  - **SIA:** Deriva o Ano de Nascimento subtraindo a Idade da Data de Atendimento. Normaliza os códigos de sexo e município para garantir compatibilidade com o SIH.
- **Definição de Coorte:** Identifica o universo de pacientes únicos que apresentaram o CID da doença crônica de interesse (ex: I50 - Insuficiência Cardíaca) em qualquer ponto do tempo, seja em consultas ambulatoriais ou internações.

3. Engenharia de Atributos Longitudinais

- **Agregação Temporal:** Para cada paciente da coorte, o algoritmo realiza varreduras nos dados históricos para calcular métricas de consumo de serviços:
  - `N_CONSULTAS_AMB_12M` : Frequência de atendimentos ambulatoriais no ano anterior.
  - `N_PROCEDIMENTOS_COMPLEXOS` : Contagem de procedimentos de alta complexidade.
  - `N_INTERNACOES_ANTERIORES` : Histórico de hospitalizações prévias (forte preditor de reincidência).
- **Definição do Alvo (Y):** A variável binária `ALVO_HOSPITALIZADO` recebe `1` se o paciente registrou uma internação hospitalar na Janela de Predição, e `0` caso contrário.

4. Modelagem e Interpretação

- **Treinamento:** Ajusta um classificador **LightGBM** para estimar a probabilidade de internação futura ($P(Y = 1|X)$).
- **Avaliação:** Utiliza métricas de classificação (AUC-ROC, Precision-Recall) adequadas para classes desbalanceadas.
- **Explicabilidade (SHAP):** Gera gráficos de importância de atributos para validar hipóteses clínicas (ex: verificar se a *falta* de consultas ambulatoriais está correlacionada com o *aumento* do risco de internação, indicando falha na Atenção Primária).

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de UFs para análise.
- `--anos` : Lista de anos que compõem tanto o histórico quanto o período alvo.
- `--ano-snapshot` : O ano de referência para a predição (o ano anterior será usado como histórico).
- `--cid` : Código CID-10 da condição crônica (ex: `I50` , `E11` , `J44`).
- `--dir-saida` : Diretório para persistência dos resultados.
- `--force-download` : *Flag* para ignorar o cache local (Parquet) e forçar novo download do DATASUS.

Artefatos Gerados (Outputs)

1. **Arquivos de Dados Otimizados (`.parquet`):** Bases do SIH e SIA processadas e comprimidas, salvas localmente para reuso rápido por outros módulos (como `analise_sequencia_eventos.py`).
2. **Relatório de Performance (Console/Log):** Métricas de acurácia do modelo preditivo.
3. **Gráfico de Fatores de Risco (`feature_importance_cronicos.png`):** Visualização SHAP detalhando quais comportamentos na jornada do paciente (ex: frequência de consultas) mais impactam o risco de hospitalização.

Módulo de Pipeline de Readmissão Hospitalar (`src/modelagem/pipeline_readmissao.py`)

Este módulo atua como um orquestrador *end-to-end* (ponta a ponta) para o desenvolvimento do modelo de risco de readmissão em 30 dias. Diferente de scripts isolados, este arquivo consolida

as etapas de ingestão de dados brutos, engenharia de atributos avançada (incluindo enriquecimento com dados de infraestrutura), treinamento de modelo e geração de artefatos de explicabilidade em um fluxo de execução único e reprodutível.

Arquitetura e Fundamentação Metodológica

O sistema implementa uma arquitetura de **Aprendizado de Máquina Supervisionado** aplicada a dados longitudinais de saúde.

- **Definição do Problema:** Classificação Binária. O objetivo é estimar a probabilidade $P(Y = 1|X)$, onde $Y = 1$ indica o retorno do paciente a uma unidade hospitalar em um intervalo ≤ 30 dias após a alta.
- **Integração de Domínios:** O modelo não considera apenas variáveis clínicas do paciente (SIH), mas enriquece o vetor de características com dados estruturais da unidade prestadora (CNES), partindo da premissa de que a qualidade e o perfil do hospital influenciam o desfecho da internação.
- **Algoritmo:** Utiliza o **LightGBM**, configurado com pesos balanceados para mitigar a desproporção entre pacientes que retornam e pacientes que não retornam.

Fluxo de Processamento Detalhado

O *pipeline* é segmentado em quatro etapas macro de processamento:

1. Aquisição e Persistência de Dados Brutos

- **Ingestão:** Utiliza a biblioteca `PySUS` para extrair dados do Sistema de Informações Hospitalares (SIH - Dados Clínicos) e do Cadastro Nacional de Estabelecimentos de Saúde (CNES - Dados de Infraestrutura).
- **Persistência Intermediária:** Salva os dados brutos em formato `.parquet` (`sih_raw_data.parquet` , `cnes_st_raw_data.parquet`). Isso cria um *checkpoint* que evita downloads redundantes em re-execuções do pipeline.

2. Engenharia de Atributos Aprimorada (*Advanced Feature Engineering*)

- **Construção Longitudinal:** Gera identificadores de pacientes (`Pseudo-ID`) e ordena os registros temporalmente para calcular a variável alvo (janela de 30 dias entre alta e nova admissão).
- **Enriquecimento Cadastral:** Realiza a fusão (*Merge*) entre as internações e o cadastro do hospital (CNES) utilizando as chaves `CNES` e `ANO` . Incorpora variáveis institucionais como:
  - **Natureza Jurídica:** Pública, Privada, Filantrópica.
  - **Capacidade:** Número de Leitos.
  - **Ensino:** Se a unidade é hospital de ensino/universitário.
- **Transformação de Variáveis:**
  - Criação de *proxies* de gravidade, como uso de UTI (`USOU_UTI`).
  - Agrupamento de diagnósticos pelo Capítulo da CID-10 para redução de dimensionalidade.
  - Conversão de variáveis categóricas para o tipo `category` (otimização nativa do LightGBM).

3. Treinamento e Validação do Modelo

- **Divisão de Dados:** Segrega o *dataset* analítico em conjuntos de Treino e Teste (75/25), mantendo a estratificação pela variável alvo.
- **Ajuste do Modelo:** Treina o classificador LightGBM.
- **Avaliação de Métricas:** Calcula indicadores de desempenho robustos para classes desbalanceadas:
  - **AUC-ROC:** Capacidade de discriminação global.
  - **AUPRC (Precision-Recall):** Focada na classe positiva (readmissão).

4. Interpretabilidade e Serialização

- **Serialização:** Exporta o modelo treinado (`.joblib`) e os dados de teste (`.parquet`) para uso futuro em APIs de inferência ou dashboards.
- **XAI Global (SHAP Summary):** Gera visualização dos fatores de risco mais impactantes no modelo geral.
- **XAI Local (SHAP Waterfall):** Simula a explicação de um caso de alto risco individual, demonstrando a aplicabilidade clínica do modelo.

Parâmetros de Configuração

Diferente dos módulos de execução via CLI, este script atua como uma definição de *pipeline* mestre. As configurações são definidas através de constantes globais no início do código:

- `UFS_PARA_ANALISE` : Lista de UFs alvo.
- `ANOS_PARA_ANALISE` : Janela temporal.
- `FORCE_DOWNLOAD` : *Flag* booleana para forçar a atualização dos dados brutos.

Artefatos Gerados (Outputs)

A execução bem-sucedida popula os diretórios de saída com uma estrutura completa de MLOps:

1. **Dados (`dados_brutos/` , `dados_processados/`):** Arquivos Parquet contendo desde a extração bruta até o *dataset* analítico final e conjuntos de validação (`X_test.parquet`).
2. **Modelos (`modelos/`):** Arquivo binário `lgbm_readmission_model.joblib` , pronto para *deployment*.
3. **Relatórios Visuais (`outputs/`):**
   - `shap_summary_plot_friendly.png` : Visão gerencial dos drivers de readmissão.
   - `shap_waterfall_plot_individual_friendly.png` : Exemplo de justificativa clínica para um paciente específico.

Módulo de Inferência de Risco Individual (`src/modelagem/predict_pacient.py`)

Este módulo atua como o **Motor de Inferência** (*Inference Engine*) via interface de linha de comando (CLI). Sua função é operacionalizar o modelo de *Machine Learning* treinado (`LightGBM`) em um cenário de produção simulado, permitindo a realização de predições pontuais

para novos pacientes (instância única). Além da estimativa de risco, o módulo fornece explicabilidade local detalhada para justificar a decisão algorítmica.

Arquitetura e Fundamentação Metodológica

O componente foi desenhado para garantir a **Consistência de Treino-Serviço** (*Training-Serving Skew prevention*), assegurando que os dados de entrada recebam o mesmo tratamento dado durante a modelagem.

A arquitetura baseia-se em três pilares:

1. **Consumo de Artefatos Serializados:** Carrega o modelo persistido (`.joblib`) e uma amostra do *dataset* de referência (`.parquet`) para inicializar os explicadores probabilísticos.
2. **Engenharia de Atributos em Tempo Real:** Implementa a função `engineer_features_for_patient`, que replica as transformações lógicas (ex: extração de Capítulo do CID) aplicadas no *pipeline* de treinamento, garantindo que o vetor de características de entrada seja compatível com o modelo.
3. **Explicabilidade Local (Local XAI):** Utiliza **SHAP Waterfall Plots** para decompor a predição de uma instância específica. Diferente da importância global (que mostra o que é relevante para o modelo em geral), esta técnica quantifica como cada atributo do paciente desviou o risco em relação ao valor base (*Base Value*), somando-se até a probabilidade final.

Fluxo de Processamento Detalhado

O fluxo de execução segue o padrão de um microsserviço de predição:

1. Inicialização e Carregamento

- **Desserialização:** Carrega o arquivo binário `lgbm_readmission_model.joblib`.
- **Contexto SHAP:** Carrega o arquivo `X_test.parquet` para instanciar o objeto `TreeExplainer`. Isso é necessário para calcular o valor esperado (média) do modelo, que serve de ponto de partida para o gráfico *Waterfall*.

2. Ingestão e Alinhamento de Esquema (*Schema Enforcement*)

- **Parsing:** Recebe os dados clínicos e demográficos via argumentos de CLI.
- **Transformação:** Aplica a engenharia de *features* (ex: cria `CAPITULO_CID` a partir de `DIAG_PRINC` e descarta a coluna original).
- **Alinhamento de Tipagem:** Executa a operação de `reindex` para forçar que o *DataFrame* de entrada possua exatamente as mesmas colunas do treino. Converte explicitamente variáveis categóricas para o tipo `category` do Pandas e numéricas para `float/int`, prevenindo erros de execução no LightGBM.

3. Inferência Probabilística

- Executa o método `predict_proba` no vetor de dados do paciente.
- Extrai a probabilidade da classe positiva (`1`), retornando o **Score de Risco de Readmissão em 30 dias** (0.0 a 1.0).

4. Geração de Artefatos de Decisão

- Calcula os *SHAP values* individuais.
- Gera um gráfico *Waterfall* que visualiza a contribuição marginal de cada variável (positiva ou negativa) para o score final.
- Persiste os dados brutos da explicação e a imagem para auditoria ou integração em *front-ends*.

Parâmetros de Execução (CLI)

O script aceita argumentos que representam o perfil clínico e administrativo do paciente:

- `--idade` : Idade do paciente (anos).
- `--dias_perm` : Tempo de permanência da internação atual.
- `--usou_uti` : Indicador binário de uso de Terapia Intensiva (1=Sim, 0=Não).
- `--sexo` : Código do sexo (1=Masc, 3=Fem).
- `--raca_cor` : Código da raça/cor (Padrão IBGE).
- `--car_int` : Caráter da internação (01=Eletivo, 02=Urgência).
- `--diag_princ` : Código CID-10 do diagnóstico principal (ex: I500).
- `--hosp_natureza` : Código da natureza jurídica do estabelecimento.
- `--hosp_leitos` : Capacidade instalada do hospital.
- `--dir_saida` : Diretório obrigatório para persistência dos resultados.

Artefatos Gerados (Outputs)

1. **Log de Predição (** `previsao_paciente_individual.csv` **):** Registro tabular contendo os dados de entrada processados e o risco calculado.
2. **Dados Explicativos (** `shap_explanation_data.csv` **):** Tabela detalhando o valor da *feature* e seu respectivo impacto SHAP, permitindo renderização customizada.
3. **Visualização de Decisão (** `previsao_paciente_individual.png` **):** Imagem estática do gráfico *Waterfall*, evidenciando os principais fatores que elevaram ou diminuíram o risco daquele paciente específico.

Módulo de Inferência de Sobrevivência Individual (`src/modelagem/prever_sobrevida_individual.py`)

Este módulo atua como um **Motor de Inferência de Análise de Sobrevivência**. Sua função é aplicar um modelo de Riscos Proporcionais de Cox (*Cox Proportional Hazards*) previamente treinado para estimar a função de sobrevivência individualizada $S(t|x)$ de um novo paciente.

Diferente dos classificadores binários, este sistema projeta uma curva probabilística contínua ao longo do tempo, permitindo estimar o risco de readmissão hospitalar em múltiplos horizontes temporais (30, 90, 180 dias).

Arquitetura e Fundamentação Metodológica

O sistema operacionaliza o modelo matemático de Cox para instâncias únicas (single-instance inference). A arquitetura enfrenta o desafio de reproduzir, em tempo de execução, o esquema de codificação utilizado no treinamento sem acesso ao *pipeline* de transformação original.

- **Reconstrução do Espaço de Atributos:** O *script* inspeciona os coeficientes do modelo carregado para identificar quais variáveis *dummy* (binárias) são esperadas.
- **Codificação Manual (*Hard-coded One-Hot Encoding*):** Implementa uma lógica de transformação para converter os *inputs* categóricos brutos (ex: CID "I50", Sexo "Feminino") nas colunas esparsas específicas exigidas pelo modelo (ex: `DIAG_PRINC_CAP_I=1`, `SEXO_3=1`), garantindo o alinhamento dimensional do vetor de entrada.

Fluxo de Processamento Detalhado

O *pipeline* de execução segue quatro etapas lógicas:

1. Carregamento de Artefatos

- **Desserialização:** Carrega o objeto `CoxPHFitter` da biblioteca `lifelines` a partir de um arquivo `.joblib`. Este objeto contém os coeficientes de regressão (β) e a função de risco base ($h_0(t)$).

2. Vetorização e Alinhamento

- **Inicialização:** Cria um *DataFrame* de uma única linha com todas as colunas esperadas pelo modelo, inicializadas com valor zero.
- **População de Dados:**
  - Preenche diretamente as variáveis numéricas contínuas (Idade, Dias de Permanência, Leitos).
  - Mapeia as variáveis categóricas de entrada para as respectivas colunas *dummy*, definindo o valor como `1` quando a categoria corresponde (ex: Se CID começa com 'I', define `DIAG_PRINC_CAP_I = 1`).

3. Predição da Função de Sobrevivência

- Executa o método `predict_survival_function` do objeto Cox.
- O resultado é uma série temporal decrescente onde o índice é o tempo (t em dias) e o valor é a probabilidade de **não ocorrência** do evento (não ser readmitido) até aquele momento.

4. Extração de Métricas e Visualização

- **Pontos de Corte:** Extrai probabilidades pontuais para horizontes clínicos relevantes (30, 90 e 180 dias) para exibição no console.
- **Renderização Gráfica:** Plota a curva completa de sobrevivência, permitindo a visualização da dinâmica de risco ao longo do tempo (ex: risco acentuado nos primeiros dias pós-alta vs. risco constante).

Parâmetros de Execução (CLI)

O script aceita argumentos que descrevem o perfil clínico e demográfico do paciente:

- `--idade` : Idade do paciente (anos).
- `--dias-perm` : Duração da internação índice.
- `--usou-uti` : Quantidade de dias em UTI (ou indicador de uso).
- `--sexo` : Código do sexo (1=Masc, 3=Fem).
- `--diag-princ` : Código CID-10 do diagnóstico principal.
- `--leitos-hosp` : Capacidade total de leitos do hospital.
- `--hosp-ensino` : Indicador binário de hospital de ensino (0 ou 1).
- `--modelo` : Caminho para o arquivo `.joblib` do modelo treinado.

Artefatos Gerados (Outputs)

1. **Relatório de Probabilidades (Console/Log):** Exibe as chances percentuais de o paciente **não** ser readmitido nos marcos de 30, 90 e 180 dias.
   - *Utilidade:* Apoio à decisão para definição de retornos ambulatoriais precoces ou inclusão em programas de monitoramento domiciliar.
2. **Curva de Sobrevivência (`curva_risco_readmissao_individual.png`):** Gráfico de linha representando a função $S(t)$, evidenciando a velocidade de decaimento da probabilidade de "sobrevivência" (não retorno) ao hospital.

Módulo de Inferência de Risco de Óbito Infantil (`src/modelagem/prever_sobrevida_infantil.py`)

Este módulo atua como o **Motor de Inferência** (*Inference Engine*) para o modelo de prognóstico de mortalidade infantil. Sua função é operacionalizar o classificador treinado (`LightGBM`), permitindo a avaliação de risco em tempo real para um recém-nascido específico, baseando-se em dados clínicos do parto e características maternas.

Além da estimativa probabilística, o módulo integra rotinas de **Inteligência Artificial Explicável (XAI)** para fornecer justificativas clínicas detalhadas sobre a predição.

Arquitetura e Fundamentação Metodológica

O sistema foi projetado para simular a aplicação do modelo em ambiente clínico (ex: sala de parto ou triagem neonatal). A arquitetura prioriza a consistência entre o ambiente de treinamento e o de produção:

1. **Reutilização de Lógica de ETL:** Importa e executa a função `preparar_dados_para_modelo` do módulo de treinamento original. Isso garante que o vetor de entrada do novo paciente sofra exatamente as mesmas transformações (tipagem, tratamento de nulos) que os dados históricos, prevenindo o *Training-Serving Skew*.
2. **Explicabilidade Local Baseada em Perturbação:** Utiliza o **SHAP** (*SHapley Additive exPlanations*) com uma amostra de fundo (*background dataset*) extraída dos dados de treino. Isso permite calcular como cada variável do recém-nascido (ex: "Peso=2200g") contribuiu marginalmente para aumentar ou diminuir o risco em relação à média da população base.

Fluxo de Processamento Detalhado

O *pipeline* de execução segue quatro etapas lógicas:

1. Carregamento de Artefatos

- **Modelo:** Desserializa o arquivo `.joblib` contendo o classificador treinado.
- **Dados de Referência:** Carrega o *dataset* de treinamento (`painel_sobrevida_*.csv`). Esta etapa é necessária para inicializar o `TreeExplainer` do SHAP, que requer uma distribuição de referência para calcular os valores esperados.

2. Vetorização da Instância

- **Parsing:** Recebe os parâmetros clínicos e sociodemográficos via interface de linha de comando (CLI).
- **Estruturação:** Converte os escalares em um *DataFrame* pandas de linha única, com as colunas nomeadas estritamente de acordo com o esquema exigido pelo modelo (ex: `PESO` , `APGAR5` , `QTLEIT39`).

3. Inferência e Classificação

- Executa o método `predict_proba` para obter a probabilidade da classe positiva (Óbito).
- Aplica regras de negócio para traduzir a probabilidade numérica em categorias semânticas de alerta (Baixo, Moderado ou Elevado Risco), facilitando a triagem rápida por profissionais de saúde.

4. Decomposição do Risco (SHAP Waterfall)

- Instancia o explicador SHAP utilizando a amostra de fundo.
- Calcula os valores SHAP para a instância atual.
- Gera e persiste um gráfico do tipo **Waterfall**, que visualiza graficamente a soma vetorial dos riscos: partindo do risco base ($E[f(x)]$), soma-se o impacto de cada variável até atingir o risco predito final ($f(x)$).

Parâmetros de Execução (CLI)

O script aceita argumentos que compõem o perfil do recém-nascido e da mãe:

- **Dados do Recém-Nascido:**
  - `--peso` : Peso ao nascer (em gramas).
  - `--gestacao` : Faixa de semanas de gestação.
  - `--apgar5` : Índice de Apgar no 5º minuto.
- **Dados Maternos:**
  - `--idade-mae` : Idade da mãe.
  - `--escolaridade` : Código de escolaridade.
  - `--raca-cor` : Código de raça/cor.
  - `--est-civil` : Estado civil.
- **Histórico Obstétrico:**
  - `--consultas-prenatal` : Número de consultas.

    - `--filhos-vivos` : Paridade prévia.
    - `--filhos-mortos` : Histórico de perdas fetais.
    - `--parto` : Tipo de parto (Vaginal/Cesáreo).
- **Dados do Estabelecimento:**
    - `--leitos-uti-neo` : Capacidade de UTI Neonatal do hospital.
    - `--hosp-ensino` : Indicador de hospital de ensino.
- **Configuração:**
    - `--modelo` : Caminho do arquivo de modelo.
    - `--dados-treino-csv` : Caminho do dataset de referência.

Artefatos Gerados (Outputs)

1. **Relatório de Risco (Console/Log):** Exibe a probabilidade percentual de risco e a classificação qualitativa (Avaliação de Risco).
2. **Visualização Explicativa (`explicacao_risco_obito_individual.png`):** Imagem estática do gráfico *Waterfall*.
    - **Utilidade:** Este artefato serve como "justificativa algorítmica" para a equipe clínica, permitindo entender, por exemplo, se o alto risco foi determinado primariamente pela prematuridade ou pela falta de infraestrutura hospitalar (ausência de UTI).

Módulo de Previsão de Custo de Internação (`src/modelagem/previsao_custo_internacao.py`)

Este módulo implementa um *pipeline* de **Aprendizado de Máquina Supervisionado (Regressão)** focado na gestão financeira hospitalar. Sua função é estimar o custo total (`VAL_TOT`) de uma Autorização de Internação Hospitalar (AIH) antes do faturamento final, baseando-se em características clínicas do paciente, procedimentos realizados e perfil estrutural do estabelecimento de saúde.

Arquitetura e Fundamentação Metodológica

O sistema aborda a previsão de custos como um problema de regressão em dados tabulares heterogêneos. A arquitetura lida com desafios estatísticos específicos de dados financeiros em saúde:

- **Distribuição Log-Normal:** Custos em saúde tipicamente seguem uma distribuição de cauda longa (muitos procedimentos baratos, poucos procedimentos extremamente caros). O modelo aplica uma **Transformação Logarítmica** ($y' = \ln(y+1)$) na variável alvo antes do treinamento para normalizar a distribuição e estabilizar a variância dos erros, revertendo a transformação ($\exp(y') - 1$) na fase de inferência.
- **Algoritmo:** Utiliza o **LightGBM Regressor**, selecionado por sua eficiência em lidar com variáveis categóricas de alta cardinalidade (como CIDs e Códigos de Procedimento) e por sua robustez a *outliers*.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de processamento de dados e modelagem estatística:

1. Ingestão e Integração de Fontes

- **Extração:** Utiliza a biblioteca `PySUS` para carregar dados do SIH (foco em custos e clínica) e do CNES (foco em infraestrutura e gestão).
- **Enriquecimento:** Realiza a fusão (*Merge*) das bases utilizando as chaves `CNES` e `ANO_COMPETENCIA`, agregando ao registro da internação atributos institucionais como Natureza Jurídica e Tipo de Gestão.

2. Pré-processamento e Engenharia de Atributos

- **Limpeza:** Remove registros com custos zerados ou nulos, que representam erros de faturamento ou lançamentos administrativos sem valor contábil.
- **Tipagem de Dados:** Converte explicitamente colunas categóricas (Sexo, Diagnóstico, Procedimento) para o tipo `category`, permitindo que o algoritmo de *Gradient Boosting* utilize métodos de *binning* otimizados para essas variáveis.
- **Transformação do Alvo:** Aplica a função `np.log1p` sobre a coluna `VAL_TOT`, convertendo o problema para uma escala logarítmica para reduzir o impacto de *outliers* extremos durante a otimização da função de perda.

3. Treinamento e Avaliação

- **Ajuste do Modelo:** Treina o regressor LightGBM no espaço logarítmico.
- **Avaliação na Escala Real:**
  - Realiza predições no conjunto de teste.
  - Aplica a transformação inversa (`np.expm1`) para retornar os valores à escala monetária original (Reais).
  - Calcula métricas de desempenho: **RMSE** (Raiz do Erro Quadrático Médio) para penalizar grandes desvios e **R^2** para medir a explicabilidade da variância.

4. Persistência e Explicabilidade

- **Schema Snapshot:** Salva o *DataFrame* de treino (`X_train_data.parquet`). Este artefato é crítico para garantir que o microsserviço de inferência (API) conheça a estrutura exata das categorias e tipos de dados esperados pelo modelo.
- **Interpretação Global (SHAP):** Gera um *Summary Plot* para identificar quais fatores (ex: Permanência em UTI, Complexidade do Procedimento) são os maiores *drivers* de custo.
- **Serialização:** Exporta o modelo treinado (`.joblib`) para *deployment*.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas para compor a base de treino.
- `--anos` : Janela temporal dos dados históricos.
- `--dir_saida` : Diretório para persistência dos modelos, dados e relatórios.

Artefatos Gerados (Outputs)

1. **Modelo de Regressão (`modelo_custo_internacao.joblib`):** Objeto binário do modelo treinado.
2. **Schema de Treinamento (`X_train_data.parquet`):** Arquivo contendo a estrutura de dados e metadados das categorias, essencial para evitar erros de alinhamento em produção.
3. **Relatório de Métricas (Console/Log):** Exibe RMSE e R^2 calculados na moeda local.
4. **Gráfico de Drivers de Custo (`shap_custo_internacao_friendly.png`):** Visualização que hierarquiza as variáveis pelo impacto financeiro na conta hospitalar.

Módulo de Previsão de Demanda Estratificada (`src/modelagem/previsao_demanda_estratificada.py`)

Este módulo implementa um *pipeline* híbrido que combina **Classificação Supervisionada** e **Previsão de Séries Temporais** (*Time Series Forecasting*). Sua função é projetar a demanda futura de nascimentos, segregando o volume total em estratos de risco clínico (Alto Risco vs. Risco Habitual).

O objetivo é fornecer insumos para o planejamento de capacidade hospitalar diferenciada, distinguindo a necessidade de leitos de UTI Neonatal (para o estrato de alto risco) da necessidade de leitos de Alojamento Conjunto (para o estrato de baixo risco).

Arquitetura e Fundamentação Metodológica

O sistema adota uma abordagem de **Previsão Hierárquica/Segmentada**, partindo da premissa de que subpopulações com perfis clínicos distintos possuem dinâmicas temporais e necessidades de recursos diferentes.

- **Classificação Retrospectiva:** Utiliza um modelo de *Machine Learning* pré-treinado (LightGBM) para realizar a inferência em lote (*Batch Inference*) sobre dados históricos, rotulando cada nascimento passado com um escore de risco perinatal.
- **Modelagem Temporal (Prophet):** Utiliza o algoritmo **Prophet** (modelo aditivo de regressão) para ajustar curvas de tendência e sazonalidade independentes para cada estrato de risco. O Prophet foi selecionado por sua robustez a *outliers* e capacidade de modelar sazonalidades anuais (ex: picos de nascimentos em meses específicos).

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de processamento e modelagem:

1. Ingestão de Série Histórica

- **Extração:** Utiliza a biblioteca `PySUS` para carregar um longo período histórico de dados do SINASC (recomendado ≥ 5 anos). A profundidade temporal é crítica para a detecção correta de padrões sazonais pelo algoritmo Prophet.
- **Consolidação:** Unifica os arquivos anuais em um único *DataFrame* contínuo.

2. Estratificação de Risco (*Batch Scoring*)

- **Carregamento de Modelo:** Desserializa o classificador de risco perinatal (`.joblib`) gerado pelo módulo `modelo_risco_perinatal.py` .
- **Inferência:** Aplica o modelo a todos os registros históricos para calcular a probabilidade de risco (P_{risco}).
- **Segmentação:** Classifica os registros em categorias ('Alto Risco' ou 'Baixo/Moderado Risco') com base em um limiar de corte (*threshold*) configurável.

3. Agregação e Modelagem de Séries Temporais

- **Resampling:** Converte os registros individuais em séries temporais de frequência mensal (*Monthly Frequency*), contabilizando o volume de nascimentos por mês para cada estrato.
- **Treinamento (Prophet):** Instancia e ajusta modelos de previsão independentes para cada grupo. O modelo decompõe a série em componentes de tendência (crescimento/decrescimento a longo prazo) e sazonalidade (padrões repetitivos anuais).
- **Projeção:** Gera o *forecast* para o horizonte futuro solicitado (ex: 12 meses), incluindo intervalos de confiança (`yhat_lower` , `yhat_upper`) que quantificam a incerteza da previsão.

4. Consolidação e Visualização

- **Fusão de Resultados:** Combina as previsões dos diferentes estratos em um único relatório tabular.
- **Plotagem Comparativa:** Gera gráficos que sobrepõem os dados históricos observados às curvas de previsão futura e seus intervalos de confiança, permitindo a validação visual da aderência do modelo.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas.
- `--anos-historico` : Lista de anos para compor a base de treinamento da série temporal.
- `--meses-futuros` : Horizonte de previsão (em meses).
- `--modelo-risco` : Caminho para o arquivo do classificador de risco perinatal (`.joblib`).
- `--limiar-risco` : Ponto de corte probabilístico (0.0 a 1.0) para definir "Alto Risco".
- `--dir-saida` : Diretório para persistência dos resultados.

Artefatos Gerados (Outputs)

1. **Relatório de Previsão (`previsao_demanda_estratificada.csv`):** Arquivo tabular contendo a série temporal futura.
   - **Colunas:** Data (`ds`), Previsão Central (`yhat`), Limite Inferior e Superior para cada estrato de risco.
   - **Utilidade:** Insumo direto para o **Simulador de Capacidade (Monte Carlo)**, fornecendo as taxas médias de chegada (λ) específicas para leitos de UTI e leitos comuns.
2. **Gráfico de Tendência (`grafico_previsao_demanda_estratificada.png`):** Visualização da série temporal que diferencia as curvas de demanda para pacientes de alto risco *versus* risco habitual, facilitando o planejamento estratégico de infraestrutura.

Módulo de Requalificação de Causa de Óbito via NLP (`src/modelagem/requalificador_causa_obito_nlp.py`)

Este módulo implementa um *pipeline* de **Processamento de Linguagem Natural (NLP)** e **Aprendizado Supervisionado** aplicado à qualificação de dados de mortalidade. Sua função é inferir a causa básica provável de óbitos originalmente classificados como "Mal Definidos" (Capítulo R da CID-10), utilizando como preditores os campos de texto livre preenchidos pelo médico na Declaração de Óbito.

Arquitetura e Fundamentação Metodológica

O sistema aborda a requalificação como um problema de **Classificação de Texto Multiclasse**. A premissa metodológica é que o texto descritivo da cadeia patológica (linhas A, B, C, D e II da DO) contém informações semânticas suficientes para deduzir o capítulo da CID-10, mesmo quando o código estruturado final foi genérico (ex: "Parada Cardíaca").

- **Representação Vetorial:** Utiliza a técnica **TF-IDF** (*Term Frequency-Inverse Document Frequency*) para converter os textos clínicos em vetores numéricos, ponderando a relevância de termos específicos em detrimento de termos comuns.
- **Algoritmo:** Emprega o **SGDClassifier** (*Stochastic Gradient Descent*), configurado com função de perda `modified_huber`. Esta escolha técnica permite o processamento eficiente de grandes volumes de texto e a extração de probabilidades calibradas (nível de confiança da predição), essenciais para filtrar alucinações do modelo.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de processamento textual e modelagem:

1. Ingestão e Estruturação do Corpus

- **Extração:** Carrega os microdados do Sistema de Informações sobre Mortalidade (SIM).
- **Engenharia de Texto:** Concatena as colunas `LINHAA`, `LINHAB`, `LINHAC`, `LINHAD` e `LINHAII` em um único documento textual por registro (`TEXTO_DO`).
- **Segmentação da Base:** Divide o *dataset* em dois subconjuntos lógicos:
  1. **Base de Treinamento (Conhecimento):** Óbitos com causa básica bem definida (Capítulos A-Q, S-Y). O modelo "aprende" a correlação entre o texto escrito e o código CID atribuído.
  2. **Base de Predição (Alvo):** Óbitos com causa mal definida (Capítulo R).

2. Pré-processamento e Vetorização

- **Normalização:** Aplica rotinas de limpeza textual (remoção de pontuação, conversão para caixa baixa, remoção de espaços excedentes) para reduzir a dimensionalidade do vocabulário.
- **Tokenização e N-Grams:** O vetorizador TF-IDF é configurado para capturar unigramas e bigramas (sequências de duas palavras), permitindo que o modelo entenda contextos compostos (ex: "insuficiencia respiratoria" tem significado distinto de apenas "insuficiencia").

3. Treinamento Supervisionado

- **Ajuste do Classificador:** Treina o modelo linear para maximizar a probabilidade de acerto do Capítulo da CID-10 com base nos vetores TF-IDF gerados a partir da Base de Treinamento.
- **Validação:** Avalia a acurácia do modelo em um subconjunto de teste dos dados bem definidos para garantir a generalização.

4. Inferência e Filtragem por Confiança

- **Predição:** Aplica o modelo treinado aos textos da Base de Predição (Causas Mal Definidas).
- **Cálculo de Confiança:** Estima a probabilidade associada à classe predita.
- **Corte de Qualidade (*Thresholding*):** Aplica um filtro rígido baseado no argumento `--limiar-confianca`. Apenas requalificações onde o modelo apresenta certeza superior ao limiar (ex: 80%) são aceitas e persistidas. Isso mitiga o risco de introduzir ruído nos dados epidemiológicos.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs`: Lista de siglas das Unidades Federativas para análise.
- `--anos`: Janela temporal de dados.
- `--dir_saida`: Diretório para persistência dos resultados.
- `--limiar-confianca`: Valor *float* (0.0 a 1.0) que define o nível mínimo de certeza exigido para considerar uma requalificação válida.

Artefatos Gerados (Outputs)

1. **Relatório de Requalificação (`obitos_requalificados_*.csv`):** Arquivo tabular contendo os registros originalmente mal definidos, acrescidos de:
   - `CAPITULO_PREVISTO`: A nova classificação sugerida pelo modelo.
   - `CONFIANCA_PREVISAO`: O score de certeza do algoritmo.
   - `TEXTO_DO`: O texto original utilizado para a inferência (para auditoria humana).
   - **Utilidade:** Permite que vigilâncias epidemiológicas recuperem informações perdidas e ajustem as taxas reais de mortalidade por causas específicas (ex: identificar óbitos cardiovasculares ocultos sob códigos de sintomas).

Módulo de Simulação de Impacto de Políticas Públicas (`src/modelagem/simulador_impacto_politicas.py`)

Este módulo implementa um sistema de **Análise Contrafactual** (*Counterfactual Analysis*) e **Simulação "What-If"**. Sua função é estimar o impacto potencial de intervenções de saúde pública (ex: universalização do acesso ao pré-natal) sobre desfechos clínicos populacionais, utilizando modelos preditivos treinados para projetar cenários alternativos.

Arquitetura e Fundamentação Metodológica

O sistema opera como uma ferramenta de **Avaliação Ex-Ante** de políticas. A arquitetura baseia-se na manipulação controlada de vetores de características de entrada de um modelo de *Machine Learning* já calibrado.

- **Princípio *Ceteris Paribus*:** A metodologia assume que, ao alterar apenas a variável de intervenção (tratamento) mantendo todas as outras variáveis constantes (idade, escolaridade, histórico), a variação resultante na predição do modelo reflete o impacto isolado daquela intervenção na população analisada.
- **Motor de Inferência:** Reutiliza o classificador `LightGBM` (treinado pelo módulo de risco perinatal) para recalcular as probabilidades de risco sob as novas condições simuladas.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de simulação e cálculo de impacto:

1. Carregamento e Definição de Linha de Base (*Baseline*)

- **Ingestão:** Carrega o *dataset* histórico de nascimentos (população real) e o modelo preditivo serializado (`.joblib`).
- **Pré-processamento:** Aplica as mesmas rotinas de limpeza e engenharia de atributos utilizadas no treinamento para garantir consistência.
- **Cenário Real:** Executa a inferência sobre os dados originais, contabilizando o número de casos classificados como "Alto Risco" (probabilidade > limiar) na situação atual.

2. Definição da Intervenção (Perturbação de Dados)

- **Identificação da População Alvo:** Seleciona o subconjunto de indivíduos elegíveis para a intervenção (ex: gestantes com número de consultas de pré-natal abaixo do recomendado).
- **Aplicação do Contrafactual:** Modifica artificialmente os valores da variável de interesse (`CONSPRENAT`) para refletir o cenário idealizado pela política pública (ex: atribui o código `4` - 7 ou mais consultas - para toda a população alvo).

3. Re-inferência (Cenário Simulado)

- Submete o *dataset* modificado ao modelo preditivo.
- Calcula as novas probabilidades de risco para cada indivíduo sob a hipótese de intervenção bem-sucedida.
- Contabiliza o novo número de casos de "Alto Risco".

4. Cálculo de Impacto

- **Redução Absoluta de Risco:** Calcula a diferença entre o número de casos de risco no Cenário Real e no Cenário Simulado (Casos Evitados).
- **Redução Relativa:** Calcula a porcentagem de redução de risco proporcionada pela intervenção, servindo como métrica de eficácia da política proposta.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--painel-base-csv` : Caminho para o arquivo CSV contendo os dados populacionais reais (Linha de Base).
- `--modelo-risco` : Caminho para o arquivo binário do modelo preditivo (`.joblib`) a ser utilizado na simulação.
- `--limiar-risco` : Valor de corte (*threshold*) de probabilidade (0.0 a 1.0) para classificar um caso como "Alto Risco".

Artefatos Gerados (Outputs)

1. **Relatório de Simulação (Console/Log):** Sumário executivo contendo:
   - **Diagnóstico Situacional:** Quantidade de casos de risco no cenário atual.
   - **Projeção de Cenário:** Quantidade de casos de risco após a intervenção.
   - **Métricas de Impacto:** Número absoluto de casos evitados e redução percentual do risco.
   - **Utilidade:** Fornece evidências quantitativas para justificar investimentos em programas preventivos (ex: demonstrar que investir na expansão da cobertura de pré-natal pode reduzir em X% a demanda por leitos de UTI Neonatal).

Módulo de Simulação de Custo de Internação (`src/modelagem/simular_custo_internacao.py`)

Este módulo atua como o **Motor de Inferência de Regressão** (*Regression Inference Engine*) via interface de linha de comando (CLI). Sua função é operacionalizar o modelo de previsão de custos (`LightGBM Regressor`) para estimar o impacto financeiro de uma única Autorização de Internação Hospitalar (AIH) hipotética ou real, antes do faturamento.

Além da estimativa monetária, o módulo fornece uma camada de **Explicabilidade Local**, detalhando quais fatores clínicos ou administrativos mais contribuíram para o custo final previsto.

Arquitetura e Fundamentação Metodológica

O sistema foi projetado para garantir a **Consistência de Esquema** (*Schema Consistency*) entre o ambiente de treinamento e o momento da inferência, um requisito crítico para modelos baseados em árvores que utilizam variáveis categóricas.

- **Inversão de Transformação:** O modelo opera internamente em escala logarítmica ($\ln(y+1)$) para estabilizar a variância. O módulo aplica automaticamente a transformação inversa exponencial (`np.expm1`) na saída para apresentar o resultado em unidade monetária real (R$).
- **Alinhamento de Metadados:** Utiliza o *snapshot* dos dados de treinamento (`X_train_data.parquet`) para impor a tipagem estrita das colunas de entrada, garantindo que variáveis categóricas (como CIDs e Procedimentos) sejam codificadas exatamente como o modelo espera.
- **Explicabilidade Aditiva (SHAP):** Decompõe o valor predito (Log-Custo) em uma soma de contribuições marginais de cada atributo, permitindo auditoria granular de contas médicas.

Fluxo de Processamento Detalhado

O *pipeline* de execução segue quatro etapas lógicas:

1. Inicialização e Carregamento de Artefatos

- **Modelo:** Desserializa o arquivo `.joblib` contendo o regressor treinado.
- **Schema de Referência:** Carrega o arquivo Parquet contendo a estrutura dos dados de treino. Isso é fundamental para recuperar os metadados das categorias (`pandas.CategoricalDtype`) utilizadas pelo LightGBM.

2. Vetorização e Tipagem Estrita

- **Parsing:** Recebe os parâmetros da internação via argumentos de CLI.
- **Estruturação:** Converte os *inputs* escalares em um *DataFrame* de linha única.
- **Coerção de Tipos:** Itera sobre as colunas do *schema* de referência e força a conversão dos dados de entrada. Se uma coluna era `category` no treino, o input é convertido para `category` com o mesmo conjunto de categorias permitidas, prevenindo erros de execução.

3. Inferência Numérica

- Executa o método `predict` do modelo para obter a estimativa em escala logarítmica.
- Aplica a função `numpy.expm1` para converter o resultado para o valor monetário final (Custo Estimado).

4. Geração de Explicabilidade (Waterfall Plot)

- Instancia o explicador `TreeExplainer`.
- Calcula o **Valor Base** (*Expected Value*), que representa o custo médio logarítmico de uma internação padrão.
- Calcula os **Valores SHAP** para a instância atual.
- Gera e persiste um gráfico *Waterfall*, demonstrando visualmente como cada fator (ex: "Uso de UTI", "Procedimento de Alta Complexidade") adicionou ou subtraiu valor monetário da estimativa base.

Parâmetros de Execução (CLI)

O script aceita argumentos que compõem o cenário da internação:

- **Dados do Paciente:**
  - `--idade` : Idade do paciente.
  - `--sexo` : Código do sexo.
- **Dados Clínicos/Procedimento:**
  - `--diag-princ` : Código CID-10 do diagnóstico principal.
  - `--proc-rea` : Código SIGTAP do procedimento realizado.
  - `--dias-perm` : Tempo de permanência (dias).
  - `--usou-uti` : Quantidade de diárias em UTI.
  - `--car-int` : Caráter da internação (Eletivo/Urgência).
  - `--morte` : Indicador de desfecho óbito.
- **Dados do Estabelecimento:**
  - `--hosp-leitos` : Capacidade do hospital.

- `--hosp-tipo` : Tipo de unidade (ex: Hospital Geral).
- `--hosp-natureza` : Código da natureza jurídica.
- `--hosp-ensino` : Indicador de hospital de ensino.

- **Configuração:**
  - `--modelo` : Caminho do arquivo de modelo.

Artefatos Gerados (Outputs)

1. **Estimativa Financeira (Console/Log):** Exibe o custo total previsto formatado como moeda (R$).
   - *Utilidade:* Apoio à auditoria de contas (identificação de *overpricing*) e planejamento orçamentário de procedimentos eletivos.
2. **Gráfico de Composição de Custo (** `explicacao_custo_individual.png` **):** Imagem estática do gráfico *Waterfall*.
   - *Utilidade:* Permite visualizar os *drivers* de custo específicos daquele caso (ex: o custo foi alto devido à longa permanência ou devido à complexidade intrínseca do procedimento?).

Módulo de Calculadora de Risco Perinatal (`src/modelagem/usar_calculadora_risco.py`)

Este módulo atua como um **Motor de Inferência de Instância Única** (*Single-Instance Inference Engine*). Sua função é operacionalizar o classificador de risco perinatal (`LightGBM`) em um ambiente de produção simulado, permitindo a avaliação individualizada de gestantes para predizer a probabilidade de desfechos adversos (Baixo Peso ao Nascer ou Prematuridade).

O sistema foi desenhado para funcionar como uma ferramenta de **Apoio à Decisão Clínica** na ponta (ex: durante o pré-natal), fornecendo não apenas o risco probabilístico, mas também a justificativa dos fatores determinantes para aquele caso específico.

Arquitetura e Fundamentação Metodológica

A arquitetura do módulo foca na portabilidade e na consistência entre o ambiente de treinamento e o ambiente de aplicação.

- **Inferência em Tempo Real:** Diferente dos módulos de processamento em lote (*batch*), este *script* processa um vetor de características isolado, retornando uma resposta imediata.
- **Alinhamento de Esquema (*Schema Enforcement*):** Implementa mecanismos para garantir que o vetor de entrada da nova gestante possua estritamente a mesma dimensionalidade e ordenação de colunas que a matriz utilizada no treinamento do modelo, prevenindo erros de execução e degradação de performance.
- **Explicabilidade Local (Local XAI):** Emprega a técnica **SHAP Waterfall**, que decompõe a predição de risco ($f(x)$) na soma do risco base da população ($E[f(x)]$) mais as contribuições marginais de cada variável da paciente (ϕ_i).

Fluxo de Processamento Detalhado

O *pipeline* de execução segue quatro etapas lógicas:

1. Carregamento de Artefatos

- **Desserialização:** Carrega o modelo treinado (`modelo_risco_perinatal.joblib`) para a memória.
- **Inicialização do Explicador:** Instancia o objeto `TreeExplainer` do SHAP associado ao modelo carregado.

2. Ingestão e Vetorização

- **Parsing:** Recebe os dados sociodemográficos e obstétricos via argumentos de linha de comando (CLI).
- **Estruturação:** Converte os inputs em um *DataFrame* pandas de linha única.
- **Reindexação:** Força o alinhamento das colunas do *DataFrame* de entrada com a lista de *features* esperada pelo modelo (`X_train_columns`). Colunas ausentes são preenchidas com `NaN` (tratadas nativamente pelo LightGBM), garantindo a robustez da inferência mesmo com dados parciais.

3. Inferência Probabilística

- Executa o método `predict_proba` sobre o vetor da paciente.
- Extrai a probabilidade da classe positiva (`1`), resultando no **Score de Risco Perinatal**.

4. Geração de Explicabilidade Visual

- Calcula os valores SHAP para a instância.
- Gera e persiste um gráfico *Waterfall*.
  - **Barras Vermelhas:** Indicam fatores que aumentam o risco (ex: Baixa escolaridade, poucas consultas).
  - **Barras Azuis:** Indicam fatores protetores (ex: Idade ideal, multiparidade).

Parâmetros de Execução (CLI)

O script aceita argumentos que compõem o perfil da gestante e da gestação atual:

- **Obrigatórios:**
  - `--idade` : Idade materna (anos).
  - `--consultas` : Número de consultas de pré-natal realizadas.
  - `--mes_prenatal` : Mês de gestação de início do pré-natal.
  - `--filhos-vivos` : Paridade (nascidos vivos anteriores).
- **Opcionais (com valores padrão):**
  - `--escolaridade` : Código de escolaridade (Padrão: 3 - Médio).
  - `--raca` : Código de raça/cor (Padrão: 4 - Parda).
  - `--filhos-mortos` : Histórico de perdas fetais (Padrão: 0).
  - `--parto` : Tipo de parto previsto/realizado (Padrão: 1 - Vaginal).
  - `--gravidez` : Tipo de gestação (Única/Múltipla).
  - `--est_civil` : Estado civil.

Artefatos Gerados (Outputs)

1. **Score de Risco (Console/Log):** Exibe a probabilidade percentual de a gestação resultar em prematuridade ou baixo peso.
2. **Gráfico de Decisão (`previsao_gestante_individual.png`):** Imagem estática do gráfico *Waterfall*.
   - **Utilidade:** Ferramenta visual para profissionais de saúde comunicarem riscos à paciente ou justificarem encaminhamentos para o Alto Risco.

Módulo de Validação Cruzada de Desfechos (`src/modelagem/validar_modelo_perinatal_com_sobrevida.py`)

Este módulo implementa um *pipeline* de **Validação Externa** e **Avaliação de *Proxy***. Sua função é verificar a capacidade de generalização clínica do modelo de "Risco Perinatal" (que prevê baixo peso e prematuridade) ao aplicá-lo para predizer um desfecho mais severo e não observado durante o treinamento original: o **Óbito Infantil**.

O objetivo técnico é validar a hipótese de que o escore de risco perinatal gerado pelo sistema é um indicador *proxy* robusto para a mortalidade, permitindo seu uso como ferramenta de triagem prioritária.

Arquitetura e Fundamentação Metodológica

O sistema executa uma **Inferência Cruzada entre Tarefas** (*Cross-Task Inference*).

- **Modelo de Origem:** Classificador treinado para prever morbidade (Baixo Peso/Prematuridade).
- **Dados de Validação (*Ground Truth*):** *Dataset* independente contendo o desfecho de mortalidade real, construído via *linkage* entre SINASC e SIM.
- **Métrica de Avaliação:** Utiliza a **AUC-ROC** (*Area Under the Receiver Operating Characteristic Curve*) para mensurar a capacidade do escore de risco (contínuo) em discriminar entre as classes binárias de sobrevivência e óbito.

Fluxo de Processamento Detalhado

O *pipeline* de execução integra componentes de dois fluxos distintos (Risco e Sobrevida) em quatro etapas:

1. Carregamento de Artefatos (Modelo)

- **Desserialização:** Carrega o arquivo `modelo_risco_perinatal.joblib`.
- **Introspecção:** Extrai a lista de *features* esperadas pelo modelo (`feature_name_`) para garantir o alinhamento do *dataset* de validação.

2. Construção do Dataset de Validação (ETL de Sobrevida)

Reutiliza as funções do módulo `analise_sobrevida_infantil.py` para construir uma base de teste "cega" para o modelo:

- **Ingestão:** Carrega dados brutos de nascimentos e óbitos.
- **Vinculação (*Linkage*):** Reconstrói a variável alvo real (`OBITO_INFANTIL`) através do cruzamento de bases.
- **Enriquecimento:** Adiciona dados do CNES (infraestrutura hospitalar).
- **Preparação:** Aplica as transformações de dados (limpeza e tipagem) necessárias para compatibilidade.

3. Inferência Cruzada

- **Alinhamento de Colunas:** Filtra o *dataset* de validação para manter apenas as colunas que o modelo de risco perinatal sabe processar.
- **Predição:** Submete os dados de validação ao modelo de risco perinatal.
  - *Nota:* O modelo não foi treinado para prever óbito, mas sim fragilidade. O sistema avalia se essa fragilidade predita correlaciona-se com o óbito real.

4. Avaliação de Performance

- **Cálculo de Métricas:** Computa a curva ROC comparando as probabilidades preditas (Risco Perinatal) contra os rótulos reais (Óbito).
- **Interpretação Automática:** Se AUC > 0.75, o sistema valida o modelo como um *proxy* eficaz. Caso contrário, indica divergência entre os fatores de risco de morbidade e mortalidade.

Parâmetros de Execução (CLI)

O script é executado via linha de comando e aceita os seguintes argumentos:

- `--dir_modelo` : Diretório contendo o modelo `.joblib` de risco perinatal previamente treinado.
- `--ufs` : Lista de UFs para construir a coorte de validação (pode ser diferente da coorte de treino para teste de generalização geográfica).
- `--ano_coorte` : Ano de referência dos dados de validação.

Artefatos Gerados (Outputs)

1. **Relatório de Validação (Console/Log):** Exibe o valor da AUC-ROC e uma conclusão semântica sobre a eficácia do modelo como preditor de mortalidade.
2. **Curva ROC (`roc_validacao_cruzada.png`):** Gráfico que ilustra a sensibilidade *versus* especificidade do modelo de risco perinatal ao tentar classificar óbitos.
   - **Utilidade:** Permite aos cientistas de dados e gestores visualizarem o quão bem o escore de risco consegue segregar os pacientes mais críticos, validando o uso da ferramenta para priorização de visitas domiciliares e cuidados intensivos.

Módulo de Previsão de Tempo de Permanência - LOS (`src/modelagem/previsao_tempo_permanencia.py`)

Este módulo implementa um sistema de **Inteligência Artificial Híbrida e Segmentada** para a predição do Tempo de Permanência Hospitalar (*Length of Stay* - LOS). Sua função é estimar a

duração da internação no momento da admissão do paciente, fornecendo insumos críticos para a gestão de giro de leitos e planejamento de alta.

A arquitetura distingue-se pela abordagem de **Segmentação por Especialidade** e pelo uso de uma **Topologia de Dois Estágios** (Classificação seguida de Regressão), superando as limitações de modelos de regressão simples em distribuições de dados assimétricas.

Arquitetura e Fundamentação Metodológica

O sistema modela o LOS não como um problema único, mas como um conjunto de subproblemas específicos de cada especialidade médica (Cirurgia, Obstetrícia, Clínica Médica, Pediatria).

Para cada segmento, o módulo instancia uma arquitetura hierárquica:

1. **Estágio 1 (Classificação de Risco):** Um classificador binário determina se a internação tende a ser de **Curta Permanência** (≤ 7 dias) ou **Longa Permanência** (> 7 dias). Isso captura a natureza bimodal de certas distribuições hospitalares.
2. **Estágio 2 (Regressão Especializada):** Com base na decisão do classificador, o paciente é encaminhado para um regressor treinado especificamente para aquele horizonte temporal, reduzindo a variância do erro.

- **Algoritmo:** Utiliza o **LightGBM** tanto para classificação quanto para regressão, devido à sua eficiência e suporte nativo a variáveis categóricas.
- **Transformação do Alvo:** Aplica transformação logarítmica ($\ln(y+1)$) na variável dependente (`DIAS_PERM`) para normalizar a distribuição de cauda longa típica de tempos de internação.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em cinco etapas de engenharia de dados e modelagem iterativa:

1. Ingestão e Enriquecimento

- **Extração:** Carrega dados do SIH (Internações) e CNES (Estabelecimentos).
- **Fusão de Dados:** Vincula as características da internação aos dados estruturais do hospital (Tipo de Gestão, Natureza Jurídica, Capacidade Instalada).

2. Engenharia de Atributos Avançada

- **Complexidade Econômica:** Cria a *feature* sintética `COMPLEXIDADE_MEDIA`, calculada como o valor médio histórico de faturamento do procedimento realizado. Esta variável atua como um *proxy* numérico para a complexidade clínica do caso.
- **Redução de Dimensionalidade:** Converte o código CID-10 completo para o seu Capítulo (primeira letra), reduzindo a esparsidade dos dados categóricos.
- **Tratamento de *Outliers*:** Remove registros com tempos de permanência acima do percentil 99, eliminando distorções causadas por casos extremos atípicos ou erros de digitação.

3. Modelagem Segmentada (Loop de Treinamento)

O script itera sobre o dicionário de especialidades mapeadas:

- **Filtro de Domínio:** Isola o *subset* de dados referente à especialidade (ex: Obstetrícia).
- **Treinamento do Classificador:** Ajusta um modelo para prever a probabilidade de Longa Permanência. Avalia via **AUC-ROC**.
- **Treinamento dos Regressores:** Ajusta dois modelos de regressão independentes (Curta e Longa), otimizando a métrica **RMSE** (*Root Mean Squared Error*) no espaço logarítmico.

4. Inferência Híbrida e Avaliação

- **Simulação de Produção:** Aplica o *pipeline* completo no conjunto de teste: o classificador direciona a instância para o regressor apropriado.
- **Reversão de Escala:** Aplica a função exponencial (`np.expm1`) para converter as predições de volta para a unidade "dias".
- **Cálculo de Métricas Reais:** Computa o **MAE** (Erro Médio Absoluto) e o **R^2** com base nos dias reais, fornecendo métricas de negócio tangíveis.

5. Explicabilidade Semântica (SHAP)

- Aplica um dicionário de mapeamento (`MAPA_ROTULOS`) para traduzir os nomes técnicos das colunas (ex: `UTI_MES_TO`) para termos de negócio (ex: "Dias de UTI").
- Gera gráficos *SHAP Summary* para o classificador, permitindo identificar quais fatores determinam o risco de "bloqueio de leito" (longa permanência) em cada especialidade.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas.
- `--anos` : Janela temporal de dados históricos.
- `--dir_saida` : Diretório raiz para persistência. O script cria subdiretórios automáticos para cada especialidade.

Artefatos Gerados (Outputs)

Para cada especialidade processada, o módulo gera:

1. **Modelos Serializados (`.joblib`):**
   - `classificador_permanencia.joblib`
   - `regressor_curta.joblib`
   - `regressor_longa.joblib`
2. **Gráficos de Performance (`.png`):**
   - `roc_curve.png` : Curva ROC do classificador.
   - `real_vs_previsto_hibrido.png` : Dispersão colorida mostrando a eficácia da separação entre curta e longa permanência.
3. **Gráfico de Interpretabilidade (`shap_drivers_longa_permanencia.png`):** Visualização dos fatores de risco com rótulos amigáveis.

Módulo de Simulação de Capacidade Hospitalar (`src/modelagem/simulador_capacidade_leitos.py`)

Este módulo implementa um motor de **Simulação de Monte Carlo** aplicado ao dimensionamento de recursos hospitalares. Sua função é modelar estocasticamente a dinâmica de ocupação de leitos sob incerteza, permitindo determinar a capacidade instalada ideal (número de leitos) necessária para atender a uma determinada demanda com um Nível de Serviço alvo (ex: 95% de atendimento sem filas).

Arquitetura e Fundamentação Metodológica

O sistema abandona as médias aritméticas simples (que falham em capturar picos de demanda) em favor de uma abordagem probabilística. A arquitetura baseia-se na simulação de cenários repetitivos para convergir em métricas robustas de risco.

- **Modelagem de Chegadas (Demanda):** Utiliza a **Distribuição de Poisson** para simular a chegada diária de pacientes, assumindo que os eventos são independentes e ocorrem a uma taxa média constante (λ), mas com variabilidade dia a dia.
- **Modelagem de Permanência (LOS):** Utiliza a **Distribuição Log-Normal** para gerar o tempo de permanência de cada paciente. Esta escolha metodológica é crítica, pois tempos de internação são assimétricos à direita (cauda longa) e não admitem valores negativos, tornando a distribuição Normal inadequada.
- **Método de Monte Carlo:** Executa milhares de iterações (anos virtuais) para cada cenário de capacidade, calculando a média dos resultados para suavizar a aleatoriedade e fornecer intervalos de confiança implícitos.

Fluxo de Processamento Detalhado

O *pipeline* de execução opera através de um loop de cenários e um loop de simulação temporal:

1. Definição de Cenários

- O script gera uma lista de configurações de capacidade a serem testadas (ex: 10, 15, 20... 50 leitos), definida pelos parâmetros de intervalo (`min`, `max`, `step`).

2. Execução da Simulação (Núcleo Estocástico)

Para cada cenário de capacidade (C), o algoritmo realiza N iterações (padrão: 1000). Em cada iteração:

- **Geração de Vetores:** Pré-calcula as chegadas diárias para todo o período de simulação (365 dias).
- **Simulação de Eventos Discretos Simplificada:**
  - Itera dia a dia.
  - **Alta:** Decrementa o tempo restante dos pacientes internados e libera leitos daqueles cujo tempo chegou a zero.
  - **Admissão:** Tenta alocar os novos pacientes gerados pela Poisson nos leitos vagos.

- **Cálculo de LOS:** Se admitido, sorteia um tempo de permanência da Log-Normal baseada na média e desvio padrão fornecidos.
- **Recusa (Fila/Perda):** Se a ocupação atingir a capacidade C, contabiliza o paciente como "Recusado" ou "Não Atendido".

3. Consolidação de Métricas

Ao final das iterações, calcula as médias das métricas operacionais para aquele cenário:

- **Taxa de Ocupação Média:** Eficiência do uso do recurso.
- **Probabilidade de Lotação:** Percentual de dias em que o hospital opera em 100% da capacidade (risco de colapso).
- **Taxa de Rejeição:** Percentual de demanda não atendida devido à falta de leitos.
- **Nível de Serviço:** Complemento da taxa de rejeição (ex: 98% dos pacientes atendidos).

4. Análise de Trade-off

- Gera visualizações que cruzam a **Eficiência** (Ocupação) com a **Segurança** (Nível de Serviço), permitindo a identificação visual do ponto ótimo de dimensionamento (onde o ganho marginal de adicionar um leito não justifica o custo de ociosidade).

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e requer parâmetros estatísticos derivados dos dados históricos (ou dos modelos preditivos anteriores):

- `--chegada-dia` : Taxa média de chegada de pacientes por dia (λ).
- `--los-medio` : Tempo médio de permanência em dias.
- `--los-std` : (Opcional) Desvio padrão do tempo de permanência. Se omitido, assume-se igual à média (distribuição exponencial).
- `--leitos-min` / `--leitos-max` / `--step` : Definem o intervalo de cenários de capacidade a serem simulados.
- `--nome-cenario` : Rótulo para identificação nos relatórios (ex: "UTI_Adulto").
- `--dir-saida` : Diretório para persistência dos resultados.

Artefatos Gerados (Outputs)

1. **Tabela de Simulação (`simulacao_*.csv`):** Arquivo contendo as métricas consolidadas para cada cenário de número de leitos testado.
2. **Gráfico de Trade-off (`tradeoff_capacidade_*.png`):** Visualização de duas curvas (Eixo Y1: Nível de Serviço, Eixo Y2: Taxa de Ocupação) em função do número de leitos (Eixo X).
   - **Utilidade:** Permite ao gestor visualizar o custo da segurança (quantos leitos "vazios" são necessários para garantir atendimento nos dias de pico).

Módulo de Detecção de Anomalias Financeiras (`src/modelagem/detector_anomalias_faturamento.py`)

Este módulo implementa um *pipeline* de **Aprendizado de Máquina Não Supervisionado** aplicado à auditoria de contas médicas. Sua função é varrer grandes volumes de Autorizações de Internação Hospitalar (AIH) para identificar registros que divergem estatisticamente do padrão esperado (outliers), sinalizando potenciais erros de faturamento, ineficiências operacionais ou fraudes.

Arquitetura e Fundamentação Metodológica

O sistema abandona a abordagem tradicional de auditoria por amostragem aleatória ou regras determinísticas simples, adotando uma abordagem baseada em densidade e isolamento.

- **Algoritmo:** Utiliza o **Isolation Forest** (*Floresta de Isolamento*). O princípio fundamental é que anomalias são dados "poucos e diferentes". O algoritmo isola observações criando cortes aleatórios nos atributos; anomalias são isoladas mais rapidamente (em menos cortes/profundidade de árvore) do que observações normais.
- **Multidimensionalidade:** O modelo avalia a anomalia considerando a correlação simultânea entre múltiplas variáveis (ex: Custo *versus* Tempo de Permanência *versus* Idade *versus* Uso de UTI), detectando padrões atípicos que regras lineares simples não capturariam.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em cinco etapas de processamento, modelagem e explicação:

1. Ingestão e Limpeza

- **Extração:** Carrega os microdados do SIH (Sistema de Informações Hospitalares) via biblioteca `PySUS`.
- **Tratamento de Tipos:** Converte colunas numéricas críticas (`VAL_TOT` , `DIAS_PERM` , `UTI_MES_TO`) garantindo a consistência aritmética.

2. Engenharia de Atributos para Auditoria

Cria *features* derivadas que potencializam a detecção de distorções:

- `CUSTO_DIA` : Custo médio diário da internação. Permite detectar internações curtas com custos exorbitantes.
- `PROP_UTI` : Proporção do tempo de internação gasto em UTI (0.0 a 1.0). Detecta inconsistências como tempo de UTI superior ao tempo total de permanência.
- **Frequency Encoding:** Transforma variáveis categóricas de alta cardinalidade (`PROC_REA` , `DIAG_PRINC`) em frequências numéricas. Isso permite que o modelo aprenda que procedimentos muito raros são inerentemente mais propensos a serem anômalos.
- **Corte de *Outliers* Extremos:** Remove o percentil 99.9 dos custos para evitar que erros grosseiros de digitação distorçam a escala dos dados antes da modelagem.

3. Detecção Não Supervisionada

- **Padronização:** Aplica `StandardScaler` para normalizar as escalas das variáveis, requisito importante para algoritmos baseados em distância ou espaço vetorial.

- **Treinamento:** Ajusta o `IsolationForest`. O hiperparâmetro `contamination` define a sensibilidade do modelo (proporção esperada de anomalias na base).
- **Scoring:** Atribui a cada AIH um *score* de anomalia e uma etiqueta binária (`-1` para anomalia, `1` para normal).

4. Explicação Heurística

Como o *Isolation Forest* é uma "caixa preta", o módulo implementa uma camada de pós-processamento para gerar justificativas humanas para a auditoria:

- Compara os valores da AIH anômala com as médias globais do *dataset*.
- Gera *tags* textuais automáticas baseadas em regras de desvio (ex: "Custo Excessivo (> 3x média)", "Diária Caríssima").

5. Visualização (Redução de Dimensionalidade)

- Utiliza **PCA (Principal Component Analysis)** para reduzir o espaço de características multidimensional para 2 componentes principais.
- Gera um gráfico de dispersão (*Scatter Plot*) projetando as anomalias em vermelho e os dados normais em azul, permitindo a validação visual da separação das classes.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de UFs para análise.
- `--anos` : Janela temporal dos dados.
- `--contaminacao` : Taxa estimada de anomalias no dataset (padrão: 0.01 ou 1%). Aumentar este valor torna o auditor mais rigoroso (marca mais casos).
- `--dir-saida` : Diretório para persistência dos relatórios.

Artefatos Gerados (Outputs)

1. **Lista de Auditoria (`lista_auditoria_ia.csv`):** Arquivo tabular contendo todas as AIHs classificadas como anômalas.
2. **Relatório Executivo (Console/Log):** Exibe as Top 10 AIHs mais suspeitas com as respectivas justificativas textuais geradas pela heurística.
3. **Mapa de Auditoria (`mapa_auditoria_anomalias.png`):** Visualização gráfica da distribuição das anomalias no espaço vetorial reduzido (PCA), demonstrando a distância entre os casos suspeitos e o comportamento padrão da rede.

Módulo de Otimização de Alocação de Recursos (`src/modelagem/otimizador_alocacao_recursos.py`)

Este módulo implementa um sistema de **Análise Prescritiva** baseado em **Programação Linear Inteira** (*Integer Linear Programming*). Sua função é resolver matematicamente o problema de alocação orçamentária em saúde pública, determinando a distribuição ótima de novas Equipes de

Saúde da Família (eSF) entre municípios para maximizar o impacto na redução de internações sensíveis à atenção primária (ICSAP), respeitando restrições financeiras e operacionais.

Arquitetura e Fundamentação Metodológica

O sistema modela o desafio de gestão como um problema de otimização combinatória, análogo ao clássico "Problema da Mochila" (*Knapsack Problem*), mas com múltiplas restrições de capacidade.

- **Paradigma:** Pesquisa Operacional (*Operations Research*).
- **Solver:** Utiliza a biblioteca `PuLP` com o solver **CBC** (*Coin-or branch and cut*), garantindo que as soluções encontradas (número de equipes) sejam números inteiros, refletindo a indivisibilidade do recurso humano.
- **Função Objetivo:** Maximizar o **Impacto em Saúde Ponderado**. O algoritmo prioriza a alocação em municípios onde a taxa de doenças crônicas é mais alta, assumindo que a utilidade marginal de uma nova equipe é maior em áreas mais vulneráveis.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de preparação de dados, modelagem matemática e resolução:

1. Ingestão e Normalização de Dados

- **Carregamento:** Ingere o painel de indicadores municipais consolidado.
- **Sanitização de Tipos:** Converte explicitamente colunas críticas (população, taxas) para tipos numéricos, garantindo a estabilidade das operações algébricas.
- **Normalização Temporal:** Implementa uma rotina de correção de dados para tratar a agregação temporal. Detecta e corrige valores de "Equipes Existentes" que representam somas anuais (12 meses), convertendo-os para a média mensal (capacidade real instalada), evitando a saturação artificial das restrições de capacidade.

2. Formulação do Problema (Modelagem)

Define as variáveis e equações do sistema linear:

- **Variáveis de Decisão (x_i):** Número inteiro de novas equipes a contratar para o município i.
- **Restrição Orçamentária:** A soma do custo de todas as novas equipes não pode exceder o orçamento global ($ \sum x*i* *cdot Custo*{eq} \le Orçamento $).
- **Restrições de Capacidade Técnica (Teto Populacional):** O número total de equipes (existentes + novas) não pode exceder a densidade recomendada de 1 equipe para cada 1.000 habitantes.
- **Restrições Administrativas (Trava de Expansão):** Limita o número máximo de novas contratações por município (ex: máximo 3 ou 10, dependendo do porte) para garantir a viabilidade operacional da expansão.

3. Resolução (*Solving*)

- Executa o algoritmo *Branch-and-Cut* para explorar o espaço de soluções e encontrar a combinação de x_i que maximiza a função objetivo global.
- Verifica o *status* da solução (Ótima, Inviável ou Ilimitada) para garantir a validade do plano proposto.

4. Consolidação e Relatório

- **Pós-processamento:** Incorpora as variáveis de decisão otimizadas de volta ao *DataFrame* original.
- **Cálculo de Impacto:** Estima o total de internações evitadas e o montante financeiro alocado.
- **Exportação:** Gera um plano de ação detalhado, listando apenas os municípios que devem receber investimento.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos para definição do cenário de investimento:

- `--indicadores-csv` : Caminho para o arquivo de dados municipais.
- `--uf` : Unidade Federativa alvo da otimização.
- `--ano` : Ano de referência dos dados.
- `--orcamento` : Valor total disponível para investimento (em moeda local).
- `--custo-equipe` : Custo unitário anual estimado de uma equipe de saúde.
- `--impacto-equipe` : Estimativa de redução de internações por equipe/ano (parâmetro de eficiência).
- `--dir-saida` : Diretório para persistência do plano de alocação.

Artefatos Gerados (Outputs)

1. **Plano de Alocação Ótima (`plano_otimo_alocacao.csv`):** Arquivo tabular contendo a lista de municípios beneficiados.
   - **Colunas:** Município, População, Taxa de Doenças, Equipes Atuais, **Novas Equipes Recomendadas** e Investimento Alocado.
   - **Utilidade:** Serve como guia prescritivo direto para secretários de saúde e gestores públicos, fundamentando a distribuição de verbas com critérios técnicos e equitativos.
2. **Relatório Executivo (Console/Log):** Resumo dos resultados agregados (Total Investido, Total de Equipes, Estimativa de Vidas Salvas).

Módulo de Detecção de Surtos e Anomalias (`src/modelagem/pipeline_deteccao_surtos.py`)

Este módulo implementa um **Sistema de Alerta Precoce** (*Early Warning System*) baseado em **Análise de Séries Temporais**. Sua função é monitorar o comportamento histórico de agravos de notificação compulsória (ex: Dengue, Chikungunya) para identificar desvios estatísticos significativos que sinalizem o início de uma epidemia ou surto localizado, permitindo uma resposta ágil da vigilância epidemiológica.

Arquitetura e Fundamentação Metodológica

O sistema moderniza o conceito clássico de "Canal Endêmico" (diagrama de controle) substituindo médias móveis estáticas por modelagem preditiva aditiva.

- **Algoritmo:** Utiliza o **Prophet** (desenvolvido pelo Facebook/Meta), um modelo de regressão aditiva decomponível. Ele foi selecionado por sua robustez a dados ausentes, capacidade de modelar múltiplas sazonalidades (anuais, mensais) e facilidade de interpretação.
- **Definição de Surto:** Um surto é detectado matematicamente quando o número de casos observados (y) ultrapassa o limite superior do **Intervalo de Confiança de 95%** ($\hat{y}_{upper}$) projetado pelo modelo, indicando um evento estatisticamente improvável sob condições basais.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de processamento temporal e inferência estatística:

1. Ingestão e Filtragem Geoespacial

- **Extração:** Utiliza a biblioteca `PySUS` para carregar microdados brutos do SINAN para o agravo específico solicitado (parâmetro `--agravo`).
- **Normalização Geográfica:** Implementa um mapeamento robusto entre Siglas de UF e Códigos IBGE para garantir a filtragem correta dos registros de notificação, independentemente do formato da coluna `SG_UF_NOT` no ano específico.

2. Construção de Séries Temporais

- **Agregação Temporal:** Converte os registros individuais (uma linha por paciente) em uma série temporal de frequência semanal (`W-SUN`), contabilizando o volume de notificações por Semana Epidemiológica.
- **Tratamento de Esparsidade:** Preenche semanas sem notificações com zero (`0`), garantindo a continuidade temporal necessária para a análise de sazonalidade.
- **Filtro de Relevância:** Aplica um limiar mínimo de volume histórico (> 50 casos) para evitar a modelagem de municípios com dados insuficientes ou ruído excessivo.

3. Modelagem e Inferência (Prophet)

- **Ajuste do Modelo:** Treina um modelo Prophet para cada município relevante, aprendendo os componentes de **Tendência** (crescimento/queda a longo prazo) e **Sazonalidade** (padrões cíclicos anuais).
- **Projeção do Canal Endêmico:** Gera previsões *in-sample* (para o período observado) para estabelecer o "comportamento esperado" e seus limites de confiança.
- **Detecção de Anomalias:** Compara os dados reais com o limite superior ($\hat{y}_{upper}$). Sinaliza como surto se:
  1. O valor real excede o limite superior.
  2. O valor real é superior a um piso mínimo de relevância epidemiológica (ex: > 5 casos), evitando alertas falsos em cenários de baixa endemicidade.

4. Visualização e Reporte

- **Geração de Gráficos:** Produz gráficos de séries temporais para os municípios em estado de alerta, plotando a curva esperada, a faixa de confiança e destacando os pontos anômalos (surtos) em vermelho.
- **Consolidação:** Exporta um relatório tabular contendo as semanas epidemiológicas críticas e a magnitude do excesso de casos.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas para monitoramento.
- `--anos` : Janela temporal histórica para treinamento do modelo (recomenda-se ≥ 3 anos para captura adequada da sazonalidade).
- `--agravo` : Código do agravo no SINAN (ex: `DENG` , `CHIK` , `ZIKA` , `INFL`).
- `--dir-saida` : Diretório para persistência dos alertas e gráficos.

Artefatos Gerados (Outputs)

1. **Relatório de Alertas (`relatorio_alertas_{agravo}.csv`):** Arquivo tabular listando os eventos detectados.
   - **Colunas:** Município, Semana Epidemiológica, Casos Observados, Limite Esperado (Teto do Canal) e Excesso de Casos.
   - **Utilidade:** Permite que a vigilância priorize municípios com desvios mais graves para intervenção imediata (fumacê, bloqueio de transmissão).
2. **Gráficos de Surto (`surto_{municipio}.png`):** Visualizações individuais para cada município crítico.
   - **Utilidade:** Prova visual do comportamento anômalo, facilitando a comunicação de risco para gestores e imprensa.

Módulo de Fenotipagem de Pacientes Complexos (`src/modelagem/clusterizacao_pacientes_complexos.py`)

Este módulo implementa um *pipeline* de **Aprendizado de Máquina Não Supervisionado** (*Unsupervised Learning*) focado na Gestão de Saúde Populacional. Sua função é segmentar a base de pacientes hospitalizados em grupos homogêneos (clusters), identificando perfis clínicos e de utilização de recursos ("Personas") que demandam estratégias de gestão diferenciadas (ex: Desospitalização, Cuidados Paliativos, Linha de Cuidado Cirúrgica).

Arquitetura e Fundamentação Metodológica

O sistema adota uma abordagem de **Redução de Dimensionalidade** seguida de **Agrupamento**. A arquitetura enfrenta o desafio da heterogeneidade dos dados de saúde, que misturam variáveis contínuas (custo, idade) com variáveis categóricas de alta cardinalidade (diagnósticos).

- **Pipeline de Transformação:** Utiliza a estrutura `Pipeline` do *Scikit-Learn* para encadear pré-processamento, redução e modelagem, garantindo a reprodutibilidade das transformações.

- **Redução de Dimensionalidade (PCA):** Aplica a **Análise de Componentes Principais** (*Principal Component Analysis*) para condensar a variância dos dados em 10 componentes latentes. Isso mitiga a "Maldição da Dimensionalidade" gerada pela codificação de variáveis categóricas e facilita a convergência do algoritmo de agrupamento.
- **Algoritmo de Agrupamento:** Utiliza o **K-Means**, um algoritmo de particionamento que minimiza a variância intra-cluster, ideal para identificar grupos compactos e distintos quando aplicado sobre componentes principais ortogonais.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em cinco etapas de manipulação de dados e análise estatística:

1. Ingestão e Filtragem de Escopo

- **Extração:** Carrega os microdados do SIH (Autorizações de Internação Hospitalar) via `PySUS`.
- **Definição de Coorte:**
  - Filtra apenas pacientes adultos (`IDADE >= 18`) para evitar a mistura de perfis pediátricos e geriátricos no mesmo espaço vetorial.
  - Remove *outliers* extremos de custo (acima do percentil 99) que poderiam distorcer o cálculo dos centroides e a escala do PCA.
- **Engenharia de Atributos:** Extrai o Capítulo da CID-10 (primeira letra) para reduzir a esparsidade da variável de diagnóstico.

2. Pré-processamento Híbrido

Utiliza um `ColumnTransformer` para aplicar tratamentos distintos conforme o tipo de dado:

- **Variáveis Numéricas (Idade, Valor, Dias, UTI):** Aplica `StandardScaler` (Z-Score) para normalizar as escalas, garantindo que variáveis com magnitudes maiores (como Valor Total) não dominem o cálculo de distância.
- **Variáveis Categóricas (Sexo, CID, Caráter, Desfecho):** Aplica `OneHotEncoder` para transformar categorias em vetores binários esparsos.

3. Modelagem (PCA + K-Means)

- **Compressão:** Projeta os dados transformados no espaço dos 10 primeiros Componentes Principais.
- **Clusterização:** Executa o K-Means sobre os componentes principais para atribuir um rótulo de cluster (0 a $K-1$) a cada paciente.

4. Perfilamento (*Profiling*) e Interpretação

- **Cálculo de Centroides:** Agrega os dados originais pelo rótulo do cluster para calcular as estatísticas descritivas médias (Idade Média, Custo Médio, Tempo de Permanência Médio).
- **Identificação de Modas:** Determina a categoria predominante (Moda) para Diagnóstico e Sexo em cada cluster.

- **Geração de Relatório:** Consolida essas estatísticas em uma tabela "Persona", permitindo a interpretação semântica dos grupos (ex: "Cluster de Alta Mortalidade e Alto Custo" vs. "Cluster de Giro Rápido e Baixa Complexidade").

5. Visualização em Espaço Reduzido

- Gera um gráfico de dispersão 2D plotando os dois primeiros Componentes Principais (PC1 vs. PC2).
- Colore os pontos conforme o cluster atribuído, permitindo a inspeção visual da separabilidade e coesão dos grupos formados.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--ufs` : Lista de siglas das Unidades Federativas.
- `--anos` : Janela temporal de análise.
- `--clusters` : Número de grupos (K) a serem formados (Hiperparâmetro do K-Means).
- `--dir-saida` : Diretório para persistência dos resultados.

Artefatos Gerados (Outputs)

1. **Relatório de Personas (** `relatorio_personas_clinicas.csv` **):** Tabela analítica que descreve o "DNA" de cada cluster.
   - **Colunas:** Cluster ID, Médias de Variáveis Numéricas, Modas de Variáveis Categóricas, Contagem e Percentual do total.
   - **Utilidade:** Base para definição de protocolos clínicos gerenciados (ex: Protocolo de Desospitalização para o cluster de longa permanência).
2. **Mapa de Clusters (** `mapa_clusters_pacientes.png` **):** Visualização da topologia dos pacientes no espaço latente, evidenciando a distinção entre perfis de alta e baixa complexidade.

Módulo de Mineração de Comorbidades Clínicas (`src/modelagem/mineracao_comorbidades.py`)

Este módulo implementa um *pipeline* de **Mineração de Regras de Associação** (*Association Rule Mining*) aplicado à epidemiologia clínica. Sua função é descobrir padrões latentes de multimorbidade, identificando quais diagnósticos ocorrem concomitantemente em internações hospitalares com frequência estatisticamente relevante, revelando "pacotes" de doenças (síndromes ou complicações) que frequentemente aparecem juntos.

Arquitetura e Fundamentação Metodológica

O sistema utiliza a técnica clássica de **Análise de Cesta de Compras** (*Market Basket Analysis*), adaptando o contexto transacional do varejo para o contexto clínico: cada internação é tratada como uma "transação" e cada código CID-10 (Doença) é tratado como um "item".

- **Algoritmo:** Utiliza o **Apriori**, um algoritmo determinístico para extração de conjuntos de itens frequentes (*frequent itemsets*) em bases de dados relacionais.
- **Métricas de Avaliação:**
  - **Suporte (*Support*):** A fração de internações que contém o conjunto de doenças.
  - **Confiança (*Confidence*):** A probabilidade condicional de ter a doença B dado que se tem a doença A ($P(B|A)$).
  - ***Lift*:** A medida de dependência ou correlação. Um *lift* > 1 indica que a presença de A aumenta a probabilidade de ocorrência de B mais do que o esperado pelo acaso (associação positiva).

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de transformação de dados e mineração de padrões:

1. Ingestão e Extração de Transações

- **Extração:** Carrega os microdados do SIH (Sistema de Informações Hospitalares).
- **Parsing de Diagnósticos:** Varre as colunas de diagnóstico principal (`DIAG_PRINC`) e as nove colunas de diagnósticos secundários (`DIAGSEC1` a `DIAGSEC9`).
- **Normalização:** Trunca os códigos CID-10 para os 3 primeiros caracteres (Categoria), agrupando subcategorias para aumentar a densidade estatística (ex: `I10.9` $\rightarrow$ `I10`).
- **Estruturação:** Constrói uma lista de listas (matriz de transações), onde cada linha representa o conjunto único de doenças presentes em uma AIH.

2. Vetorização (One-Hot Encoding)

- Utiliza o `TransactionEncoder` da biblioteca `mlxtend` para converter a lista de transações em uma matriz booleana esparsa, onde as colunas representam todas as doenças encontradas e as linhas indicam a presença (True) ou ausência (False) daquela doença na internação.

3. Mineração de Regras (Apriori)

- **Geração de Itemsets:** Identifica todas as combinações de doenças que satisfazem o limiar mínimo de **Suporte** definido pelo usuário.
- **Indução de Regras:** A partir dos *itemsets* frequentes, gera regras do tipo "Se {Antecedente} Então {Consequente}" que satisfazem o limiar mínimo de **Confiança**.
- **Ranking:** Ordena as regras descobertas pela métrica de *Lift*, priorizando as associações mais fortes e não-aleatórias.

4. Modelagem e Visualização de Redes

- **Construção de Grafo:** Converte as regras de associação em um Grafo Direcionado ($G = (V, E)$), onde os Vértices são as doenças e as Arestas representam a regra de implicação.
- **Ponderação:** O peso das arestas é definido pelo *Lift* da regra.

- **Renderização:** Utiliza `NetworkX` e `Matplotlib` para gerar uma representação visual da rede de comorbidades, permitindo a identificação visual de *clusters* de patologias (ex: Síndrome Metabólica, Complicações Cirúrgicas).

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos para calibrar a sensibilidade da mineração:

- `--ufs` : Lista de UFs para análise.
- `--anos` : Janela temporal.
- `--min-support` : Frequência mínima (0.0 a 1.0) para considerar um conjunto de doenças relevante. Valores baixos (ex: 0.001) são necessários para detectar doenças raras.
- `--min-confidence` : Probabilidade mínima (0.0 a 1.0) da regra ser verdadeira.
- `--dir-saida` : Diretório para persistência dos resultados.

Artefatos Gerados (Outputs)

1. **Tabela de Regras (`regras_associacao_clinica.csv`):** Arquivo tabular contendo as regras descobertas.
   - **Colunas:** Antecedentes, Consequentes, Suporte, Confiança, Lift.
   - **Utilidade:** Insumo para criação de protocolos clínicos de rastreio (ex: Se o paciente tem doença A, investigar automaticamente doença B).
2. **Rede de Comorbidades (`grafo_comorbidades.png`):** Visualização gráfica das conexões entre patologias.
   - **Utilidade:** Permite visualizar "ilhas" de doenças interconectadas, facilitando a compreensão de multimorbidades complexas.

Módulo de Simulação de Fluxo de Pronto Socorro (`src/modelagem/simulador_fluxo_ps.py`)

Este módulo implementa um **Gêmeo Digital Operacional** (*Digital Twin*) baseado em **Simulação de Eventos Discretos (DES)**. Sua função é modelar a dinâmica estocástica de um serviço de urgência e emergência, permitindo testes de estresse e dimensionamento de equipes (médicos e enfermeiros) e recursos físicos (leitos) antes da implementação no mundo real.

O sistema simula a jornada individual de cada paciente, minuto a minuto, para identificar gargalos ocultos, medir tempos de espera e prever taxas de desistência (*Left Without Being Seen* - LWBS).

Arquitetura e Fundamentação Metodológica

O sistema utiliza a biblioteca `SimPy` para orquestrar processos assíncronos e gerenciar recursos compartilhados em um ambiente de tempo contínuo.

- **Paradigma de Simulação:** Diferente de modelos de média (que assumem fluxo constante), o DES modela a variabilidade. Chegadas e tempos de atendimento seguem distribuições de probabilidade, capturando o efeito de "ondas" de demanda que causam filas mesmo quando a capacidade média teórica é suficiente.

- **Gestão de Recursos:** Modela enfermeiros, médicos e leitos como `Resources` (capacidade finita). Quando a demanda excede a capacidade, o sistema instancia filas de espera automaticamente.
- **Filas com Prioridade (*Priority Queues*):** Implementa a lógica do **Protocolo de Manchester**, onde a ordem de atendimento não é estritamente FIFO (*First-In, First-Out*), mas ordenada pela gravidade clínica (Vermelho > Amarelo > Verde).

Fluxo de Processamento Detalhado

A simulação executa um ciclo de vida para cada entidade "Paciente" gerada:

1. Geração de Demanda (Chegadas)

- **Processo Estocástico:** Utiliza uma **Distribuição Exponencial** para determinar o intervalo de tempo entre chegadas, simulando um Processo de Poisson com taxa média λ (pacientes/hora).

2. Triagem e Classificação

- **Alocação de Recurso:** O paciente solicita um recurso `Enfermeiro`. Se ocupado, entra em fila.
- **Classificação de Risco:** Após o atendimento (duração baseada em Distribuição Normal), um algoritmo probabilístico atribui uma cor de risco (Vermelho, Laranja, Amarelo, Verde, Azul) baseada nas proporções epidemiológicas configuradas.
- **Definição de Tolerância:** Baseado na cor, define-se o "tempo de paciência" (*Reneging Time*). Pacientes menos graves têm menor tolerância à espera antes de abandonar o sistema.

3. Atendimento Médico

- **Fila Prioritária:** O paciente entra na fila do recurso `Médico`. A posição na fila é determinada pelo nível de risco (menor valor numérico = maior prioridade).
- **Lógica de Desistência (LWBS):** Se o tempo de espera na fila exceder a tolerância do paciente, o processo é abortado, o paciente sai do sistema e o evento é contabilizado como "Desistência/Perda de Receita".
- **Serviço:** Se atendido, o tempo de consulta é gerado por uma Distribuição Exponencial, ajustada pela gravidade do caso (casos graves demandam maior tempo de processamento).

4. Decisão Clínica e Desfecho

- Após a consulta, uma árvore de decisão probabilística define o destino:
  - **Alta Imediata:** O paciente libera os recursos e sai do sistema.
  - **Observação/Medicação:** O paciente solicita um recurso `Leito/Poltrona`. O tempo de permanência segue uma Distribuição Normal (ex: tempo de correr um soro).
  - **Internação:** O paciente é transferido para a internação hospitalar (saída do escopo do PS).

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita argumentos para configuração do cenário (*Scenario Planning*):

- `--chegada` : Taxa de demanda (pacientes por hora).
- `--medicos` : Quantidade de médicos plantonistas (Capacidade do Servidor 1).
- `--enfermeiros` : Quantidade de enfermeiros de triagem (Capacidade do Servidor 2).
- `--leitos` : Quantidade de leitos de observação (Capacidade do Servidor 3).
- `--dir-saida` : Diretório para persistência dos logs.

Artefatos Gerados (Outputs)

1. **Relatório Operacional (Console/Log):**
   - **Métricas de Fluxo:** Total de pacientes atendidos, total de desistências (LWBS), total de internações.
   - **Métricas de Nível de Serviço:** Tempo médio de espera por classificação de risco (ex: "Vermelho: 0 min", "Verde: 120 min").
   - **Diagnóstico Automático:** O sistema emite alertas textuais de gargalo (ex: "🔴 GARGALO NO ATENDIMENTO MÉDICO").
2. **Log de Eventos (`simulacao_manchester.csv`):** Registro tabular granular de cada paciente simulado.
   - **Colunas:** ID, Classificação de Risco, Tempo de Espera, Status (Atendido/Desistiu), Tempo Total de Permanência (LOS no PS).
   - **Utilidade:** Permite auditoria detalhada da simulação e criação de histogramas de tempo de espera.

Módulo de Análise Espacial e Detecção de Clusters (`src/modelagem/analise_espacial_lisa.py`)

Este módulo implementa um *pipeline* de **Estatística Espacial** e **Análise Exploratória de Dados Espaciais (ESDA)**. Sua função é identificar padrões geográficos não aleatórios em indicadores de saúde, distinguindo aglomerados estatisticamente significativos (*Clusters*) de variações estocásticas (ruído).

O sistema utiliza o Índice de Moran Local (LISA) para classificar territórios em **Hotspots** (áreas de alto risco contíguas), **Coldspots** (áreas de proteção contíguas) e **Outliers Espaciais** (ilhas de divergência), fundamentais para o planejamento regionalizado de saúde.

Arquitetura e Fundamentação Metodológica

A análise baseia-se na Primeira Lei da Geografia de Tobler ("toda as coisas são parecidas, mas coisas mais próximas são mais parecidas"). O módulo quantifica essa dependência através da **Autocorrelação Espacial**.

- **Matriz de Ponderação Espacial (W):** Define a vizinhança topológica. O sistema utiliza o critério de **Contiguidade Rainha** (*Queen Contiguity*), onde dois municípios são considerados vizinhos se compartilham qualquer fronteira ou vértice.

- **Índice de Moran Global (I):** Estatística única que resume o grau de agrupamento espacial de todo o mapa. Testa a hipótese nula de aleatoriedade espacial completa.
- **Índice de Moran Local (I_i - LISA):** Decompõe o índice global para cada unidade espacial, permitindo a identificação de *clusters* locais e a avaliação da significância estatística (Valor-P) para cada município individualmente.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em três etapas de geoprocessamento e inferência estatística:

1. Ingestão e Vinculação Geoespacial (*Spatial Join*)

- **Carregamento de Atributos:** Ingere o painel de indicadores tabulares (CSV) contendo as métricas de saúde (ex: Taxa de Dengue).
- **Carregamento de Geometrias:** Ingere arquivos vetoriais (*Shapefiles*) contendo a malha digital municipal.
- **Harmonização de Chaves:** Resolve a incompatibilidade padrão entre códigos IBGE de tabelas de dados (muitas vezes 6 dígitos) e malhas cartográficas (7 dígitos, incluindo dígito verificador), truncando as chaves para garantir o pareamento perfeito dos dados (`Inner Merge`).

2. Cálculo de Autocorrelação (Modelagem)

- **Construção da Matriz W:** Gera a matriz de pesos espaciais normalizada por linha.
- **Inferência Global:** Calcula o Moran Global e executa testes de permutação para determinar a pseudo-significância do padrão geral.
- **Inferência Local (Classificação):** Calcula o LISA para cada município e classifica os resultados estatisticamente significantes ($p < 0.05$) em quatro quadrantes:
  1. **Alto-Alto (HH):** Município com valor alto cercado por vizinhos com valor alto (**Hotspot**).
  2. **Baixo-Baixo (LL):** Município com valor baixo cercado por vizinhos com valor baixo (**Coldspot**).
  3. **Alto-Baixo (HL):** Município com valor alto cercado por vizinhos com valor baixo (**Outlier**).
  4. **Baixo-Alto (LH):** Município com valor baixo cercado por vizinhos com valor alto (**Outlier**).

3. Renderização e Persistência

- **Filtragem de Significância:** Municípios cujo valor-p não atinge o limiar de confiança (95%) são classificados como "Não Significante" (ruído), evitando alarmes falsos em áreas onde o padrão pode ser aleatório.
- **Mapeamento Coroplético:** Gera visualizações cartográficas utilizando a biblioteca `splot` , colorindo as regiões conforme a categoria LISA.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--indicadores-csv` : Caminho para o arquivo CSV contendo os dados de saúde.

- `--shapefile` : Caminho para o arquivo `.shp` da malha municipal.
- `--uf` : Unidade Federativa foco da análise.
- `--ano` : Ano de referência dos dados.
- `--indicador` : Nome exato da coluna numérica a ser analisada (ex: `TAXA_INCIDENCIA_DENGUE`).
- `--dir-saida` : Diretório para persistência dos mapas e tabelas.

Artefatos Gerados (Outputs)

1. **Relatório Espacial (`analise_espacial_*.csv`):** Arquivo tabular enriquecido com os resultados estatísticos.
   - **Colunas:** Dados originais + `LISA_cluster` (Quadrante), `LISA_p_value` (Significância) e `LISA_label` (Rótulo Semântico).
   - **Utilidade:** Permite filtrar municípios prioritários para intervenção regional (HH) ou local (HL).
2. **Mapa de Clusters LISA (`mapa_lisa_*.png`):** Visualização cartográfica de alta resolução.
   - **Utilidade:** Ferramenta estratégica para gestão visual do território, diferenciando áreas de risco sistêmico (manchas vermelhas) de problemas isolados.

Módulo de Inferência Causal e Pareamento (`src/modelagem/analise_impacto_causal2.py`)

Este módulo implementa um *pipeline* estatístico robusto para **Inferência Causal** em dados observacionais de saúde. Sua função é estimar o Efeito Médio do Tratamento nos Tratados (**ATT** - *Average Treatment Effect on the Treated*) utilizando a técnica de **Pareamento por Escore de Propensão** (*Propensity Score Matching* - PSM).

O script foi desenhado para contornar limitações específicas de versões de bibliotecas científicas, implementando cálculos manuais de significância estatística para garantir a confiabilidade dos resultados.

Arquitetura e Fundamentação Metodológica

O sistema baseia-se no **Modelo de Resultados Potenciais de Rubin** (*Rubin Causal Model*). O objetivo é simular um experimento randomizado (RCT) *a posteriori*, criando um grupo de controle sintético que seja estatisticamente idêntico ao grupo de tratamento em todas as covariáveis observáveis, exceto na intervenção em si.

- **Tratamento (D):** Variável binária de intervenção (ex: Realização de Pré-natal Adequado).
- **Desfecho (Y):** Variável binária de resultado clínico (ex: Óbito Infantil).
- **Covariáveis (X):** Vetor de características de confusão (ex: Escolaridade, Idade Materna, Histórico Obstétrico).
- **Cálculo Estatístico Híbrido:** Utiliza a biblioteca `causalinference` para a estimação dos escores e pareamento, mas implementa rotinas nativas via `scipy.stats` para o teste de hipótese (Cálculo de Valor-P via Escore Z), garantindo precisão na inferência estatística.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em quatro etapas de preparação, modelagem e validação estatística:

1. Ingestão e Definição do Modelo Causal

- **Carregamento:** Ingere o *dataset* de coorte de nascimentos e óbitos (gerado pelo módulo de sobrevida).
- **Definição de Variáveis:**
  - Binariza a variável de tratamento (ex: `CONSPRENAT >= 7` $\rightarrow$ 1).
  - Define o vetor de covariáveis (X) responsável pelo viés de seleção.
- **Limpeza Estrita:** Remove registros com valores ausentes (*listwise deletion*) nas variáveis do modelo causal, requisito obrigatório para o cálculo da matriz de covariância.

2. Estimação do Escore de Propensão

- **Regressão Logística:** Ajusta um modelo Logit para estimar a probabilidade condicional de receber o tratamento: $e(x) = P(D = 1|X = x)$.
- **Geração de Escores:** Atribui a cada indivíduo (tratado ou controle) um valor de propensão entre 0 e 1.

3. Pareamento e Estimativa do Efeito

- **Matching:** Executa o algoritmo de *Nearest Neighbor Matching* (Pareamento pelo Vizinho Mais Próximo) com reposição e ajuste de viés (*Bias Adjustment*).
- **Cálculo do ATT:** Estima a diferença média de desfechos entre os grupos pareados.
- **Teste de Significância Manual:**
  - Extrai o ATT e o Erro Padrão (SE) do objeto do modelo.
  - Calcula o **Escore Z** ($Z = ATT/SE$).
  - Computa o **Valor-P Bilateral** utilizando a função de distribuição cumulativa da normal (`scipy.stats.norm.cdf`), superando limitações de acesso a atributos internos da biblioteca em diferentes versões.

4. Validação de Suposições (*Common Support*)

- **Análise de Sobreposição:** Gera gráficos de densidade (*Kernel Density Estimation*) comparando a distribuição dos escores de propensão entre os grupos Tratado e Controle.
- **Interpretação:** A visualização permite validar a suposição de suporte comum: para que a inferência seja válida, deve haver uma sobreposição substancial nas distribuições, indicando que existem "gêmeos estatísticos" comparáveis para os indivíduos tratados.

Parâmetros de Execução (CLI)

O módulo é executado via linha de comando e aceita os seguintes argumentos:

- `--csv-input` : Caminho para o arquivo CSV contendo a coorte de dados (padrão: saída do módulo de sobrevida).
- `--dir-saida` : Diretório para persistência dos resultados gráficos e logs.

Artefatos Gerados (Outputs)

1. **Relatório de Inferência Causal (Console/Log):**
   - Tabela com estimativas ATE (Efeito Médio), ATC (Efeito no Controle) e ATT (Efeito no Tratado).
   - Interpretação semântica do resultado (Significância Estatística baseada no Valor-P calculado e Magnitude do Impacto em pontos percentuais).
2. **Gráfico de Sobreposição (`propensity_score_overlap.png`):** Visualização da distribuição dos escores de propensão.
   - **Utilidade:** Ferramenta de diagnóstico para avaliar a qualidade do pareamento. Se as curvas não se sobrepuserem, o modelo indica que os grupos são incomparáveis.

Prova de Conceito: Módulo de Assistente de Protocolos Clínicos - RAG Lite (`src/modelagem/assistente_protocolos_rag.py`)

Este módulo implementa um **Protótipo Funcional (PoC)** de um sistema de **Busca Semântica** e **Recuperação de Informação** (*Information Retrieval*). Sua função é demonstrar a aplicação de Inteligência Artificial Generativa e Vetorial para a consulta de protocolos clínicos em linguagem natural, superando as limitações dos mecanismos de busca tradicionais baseados em palavras-chave exatas (*keyword matching*).

Nota Técnica: Esta implementação opera como uma arquitetura de referência "RAG Lite" (*Retrieval-Augmented Generation*), utilizando uma base de conhecimento simulada em memória para validar a viabilidade técnica da busca vetorial em ambiente local (CPU), sem dependência de APIs externas.

Arquitetura e Fundamentação Metodológica

O sistema baseia-se na representação de texto em Espaço Vetorial Latente.

- **Modelagem de Linguagem (Transformers):** Utiliza a arquitetura *Sentence-BERT* (modelo `all-MiniLM-L6-v2`), otimizada para gerar *embeddings* de sentenças densos e compactos. Este modelo converte conceitos médicos e descrições clínicas em vetores numéricos de 384 dimensões.
- **Busca Semântica:** Diferente da busca léxica, o sistema compara o **significado** da consulta com o significado dos documentos. Isso permite que uma busca por "boca torta" recupere corretamente o protocolo de "AVC Isquêmico", mesmo sem compartilhamento de vocabulário explícito.
- **Métrica de Similaridade:** Utiliza a **Similaridade de Cosseno** para calcular a proximidade angular entre o vetor da pergunta do usuário e os vetores dos documentos indexados.

Fluxo de Processamento Detalhado

O *pipeline* de execução é estruturado em três etapas de indexação e inferência:

1. Construção do Índice Vetorial (Em Memória)

- **Ingestão de Corpus:** Carrega a base de conhecimento clínica (atualmente estruturada como uma lista de dicionários estática para fins de prototipagem). Em um ambiente de produção, esta etapa seria substituída por conectores a bancos de dados vetoriais (ex: Milvus, Qdrant, pgvector).
- **Enriquecimento de Contexto:** Concatena os campos `Título`, `Texto` e `Tags` de cada protocolo para criar um documento denso de informação.
- **Vetorização (*Embedding*):** Submete o corpus textual ao modelo Transformer, gerando uma matriz de tensores (*tensors*) que representa matematicamente o conhecimento clínico disponível.

2. Processamento da Consulta (*Query*)

- **Normalização e Codificação:** Recebe a pergunta do usuário em linguagem natural, aplica pré-processamento básico e a converte em um vetor de consulta utilizando o mesmo modelo da indexação.

3. Recuperação e Ranking (*Retrieval*)

- **Cálculo de Similaridade:** Executa operações de álgebra linear (produto escalar normalizado) para medir a similaridade entre a consulta e todos os documentos da base.
- **Ordenação:** Classifica os resultados por pontuação (*score*) decrescente.
- **Filtragem:** Aplica um limiar de relevância (ex: score > 20%) para descartar resultados com baixa correlação semântica.

Parâmetros de Execução (CLI)

O script oferece dois modos de operação via linha de comando:

- `--pergunta`: Executa uma consulta pontual ("single-shot"). Recebe a string da dúvida clínica entre aspas.
- `--interativo`: Inicia um *loop* de conversação no terminal, permitindo múltiplas consultas sequenciais sem a necessidade de recarregar o modelo de IA na memória.

Artefatos Gerados (Outputs)

1. **Resultados da Busca (Console/Log):**
   - **Título do Protocolo:** Identificação do documento recuperado.
   - **Score de Relevância:** Grau de certeza da associação semântica (0 a 100%).
   - **Resumo do Conteúdo:** Trecho do texto clínico para validação rápida pelo profissional de saúde.

Limitações do Protótipo

Sendo uma prova de conceito, este módulo apresenta restrições deliberadas em relação a um sistema de produção:

- **Persistência:** O índice vetorial é recriado a cada execução (volátil), não havendo persistência em disco.

- **Escopo de Dados:** A base de conhecimento é limitada a exemplos ilustrativos (Dengue, IAM, AVC, Sepse) inseridos diretamente no código fonte.